Site Reliability Engineering

How Google Runs Production Systems

Edited by Betsy Beyer, Chris Jones, Jennifer Petoff & Niall Richard Murphy

Chapter-by-chapter reading notes

Foreword · Preface · Parts I–V · Chapters 1–34

How to read these notes

These notes are a first reading, built only from the book's own argument and words. Each Part, chapter and sub-heading appears with the book's exact title and in the book's order. Words in "curly quotes" are the book's exact words. The blue box opening each chapter says why the chapter and its title sit where they do; the gold box closing it says how it builds the authors' case. Notes are capped at a 1:4 ratio of notes to book, measured in characters per chapter, and are usually shorter. Appendices, bibliography and index are not summarised (see Contents).

Contents

The Architecture of the Book at a Glance 3
Foreword 4
Preface 5
Part I. Introduction — overview 7
1. Introduction 8
2. The Production Environment at Google, from the Viewpoint of an SRE 10
Part II. Principles — overview 12
3. Embracing Risk 13
4. Service Level Objectives 15
5. Eliminating Toil 17
6. Monitoring Distributed Systems 18
7. The Evolution of Automation at Google 21
8. Release Engineering 24
9. Simplicity 26
Part III. Practices — overview 28
10. Practical Alerting from Time-Series Data 29
11. Being On-Call 32
12. Effective Troubleshooting 34
13. Emergency Response 37
14. Managing Incidents 39
15. Postmortem Culture: Learning from Failure 41
16. Tracking Outages 43
17. Testing for Reliability 44
18. Software Engineering in SRE 47
19. Load Balancing at the Frontend 50
20. Load Balancing in the Datacenter 52
21. Handling Overload 54
22. Addressing Cascading Failures 56
23. Managing Critical State: Distributed Consensus for Reliability 59
24. Distributed Periodic Scheduling with Cron 63
25. Data Processing Pipelines 65
26. Data Integrity: What You Read Is What You Wrote 67
27. Reliable Product Launches at Scale 71
Part IV. Management — overview 74
28. Accelerating SREs to On-Call and Beyond 75
29. Dealing with Interrupts 77
30. Embedding an SRE to Recover from Operational Overload 79
31. Communication and Collaboration in SRE 81
32. The Evolving SRE Engagement Model 84
Part V. Conclusions — overview 87
33. Lessons Learned from Other Industries 88
34. Conclusion 90

Not summarised: Appendices A–F (Availability Table; A Collection of Best Practices for Production Services; Example Incident State Document; Example Postmortem; Launch Coordination Checklist; Example Production Meeting Minutes), Bibliography and Index — reference and example material, not authored argument.

The Architecture of the Book at a Glance

Part	The authors' stated aim	Chapters
I. Introduction	"what SRE is and why it is different from more conventional IT industry practices"	1–2
II. Principles	"the principles underlying how SRE teams typically work"	3–9
III. Practices	"the theory and practice of an SRE's day-to-day activity: building and operating large distributed computing systems"	10–27
IV. Management	"working together in a team, and working as teams"	28–32
V. Conclusions	how SRE's practices "compare to other industries where reliability is critically important", and SRE's evolution	33–34

The book's overall purpose, in the authors' words

The Editors describe the book as "a series of essays written by members and alumni of Google's Site Reliability Engineering organization," more like "conference proceedings" than a standard book. Its closing hope for readers is to bear "the SRE Way" in mind: "thoroughness and dedication, belief in the value of preparation and documentation, and an awareness of what could go wrong, coupled with a strong desire to prevent it."

Foreword

Place in the argument — why this chapter, why this title

Written by Mark Burgess, an outside reader (author of In Search of Certainty), this Foreword tells you how to approach the book before the editors speak: not as a manual but as one company's account of its own change of thinking.

Chapter opening — untitled section before the first sub-heading in the book

- Google's story is "a story of scaling up"; it "drew a line in the silicon" and created the revised role of SRE, formalized "using software and automation." Once "fiercely secretive," the book now lets SRE thinking "come out of the shadows."
- The essays are "personal accounts, written with pride," in varied styles, with "common themes, and common characters (software systems)" recurring. Some say "never do this, always do that"; others are tentative. Readers should come with humility, since they lack all the information.
- The reader's questions ("Why didn't they do X?") are "the real legacy of the volume."
- Core claim: "Implementations are ephemeral, but the documented reasoning is priceless." Scaling is "about scaling a business process, rather than just the machinery." The tale is "not for copying—though perhaps for emulating."

How this chapter builds the authors' case

What it sets up: a reading attitude for every later chapter: expect reasoning and candor over universal recipes, and expect the same systems and themes to recur across essays.

Preface

Place in the argument — why this chapter, why this title

Written by the editors, the Preface defines the discipline before any chapter uses it: it argues that operation is where most software effort lies, says what "Site Reliability Engineering" means, and tells readers why and how to use the book.

Chapter opening — untitled section before the first sub-heading in the book

- Software engineering is like having children: the labor after the birth is where most effort goes, yet the discipline talks mostly about the first period, "despite estimates that 40–90% of the total costs of a system are incurred after birth." The model of operational software being "stabilized" in production "is wrong."
- So there must be another discipline for "the whole lifecycle of software objects, from inception, through deployment and operation, refinement, and eventual peaceful decommissioning": Google calls it **Site Reliability Engineering**. The name is admittedly unclear; the editors unpack it in three parts.
- First, SREs are **engineers**: they apply computer science and engineering principles to large distributed systems, sometimes writing software with product developers, sometimes building pieces such as backups or load balancing (ideally reusable), sometimes applying existing solutions to new problems.
- Second, **reliability**: Ben Treynor Sloss, who originated the term, holds reliability to be "the most fundamental feature of any product." Reliability is defined as "The probability that … will perform a required function without failure under stated conditions for a stated period of time." Effort goes into it only up to a point: when systems are "reliable enough," effort moves to features and new products.
- Third, **operating services** built on distributed systems. "Site" originally meant keeping google.com running; now it covers many non-website services, from Bigtable to Google Cloud Platform.
- SRE arose in web services, where improving server-side software is comparatively contained and managing change is tightly coupled with failures. Unlike DevOps, reliability is the main focus, and SRE is oriented toward removing the necessity for operations (Chapter 7). Safety-critical software is not covered, though Chapter 33 compares other industries.
- The book separates general principles from specific practices, and adds orienting material: a description of the production environment and a mapping of internal to public software.
- Small organizations: "the earlier you care about reliability, the better"; it is cheaper to expand a structure later than to introduce one that is absent, so put lightweight reliability support in early. Part IV has training, communication and meeting practices usable at once. Mid-sized organizations probably already have someone doing SRE work unrecognized; recognize and reward them.
- The Apollo story: Margaret Hamilton is offered as the possible first SRE. Her daughter Lauren crashed a simulated mission by selecting P01 (which wipes navigation data). Margaret requested error-checking code; NASA's higher-ups judged it unnecessary, so she only updated the documentation to say not to select P01. On Apollo 8 Jim Lovell selected P01, and the documentation helped recover the mission. Her lesson: "a thorough understanding of how to operate the systems was not enough to prevent human errors," and the change was then approved.
- The **SRE Way**: "thoroughness and dedication, belief in the value of preparation and documentation, and an awareness of what could go wrong, coupled with a strong desire to prevent it."

How to Read This Book

- The book is "much more like conference proceedings" than a standard book; each chapter is part of a whole, but you can read what interests you, in any order.
- Suggested start: Chapters 2 and 3 (production environment; how SRE approaches risk). "Risk is, in many ways, the key quality of our profession."
- Cover-to-cover reading is possible: chapters group into Principles (Part II), Practices (Part III) and Management (Part IV), each with a short introduction. A companion site is at https://g.co/SREBook.

How this chapter builds the authors' case

What it sets up: the definition of SRE and of reliability used throughout, the claim that reliability is "enough" rather than maximal (taken up in the risk chapters), the Chapter 7 and Chapter 33 cross-references, and the advice to read Chapters 2 and 3 first.

PART I

INTRODUCTION

- The authors' own description: "This section provides some high-level guidance on what SRE is and why it is different from more conventional IT industry practices."

The theme of the Part

- What SRE is, and how it differs from the conventional way of running IT.

How the chapters build the theme

- Chapter 1: Ben Treynor Sloss, the senior VP overseeing technical operations at Google, gives his view of what SRE means, how it works, and how it compares to other ways of doing things.
- Chapter 2: a guide to Google's production environment, to "acquaint you with the wealth of new terms and systems you are about to meet in the rest of the book."

What the Part contributes to the book

- Definition first (Chapter 1), vocabulary second (Chapter 2); everything later assumes both.

CHAPTER 1

Introduction

Place in the argument — why this chapter, why this title

Written by Benjamin Treynor Sloss (founder of Google SRE), edited by Betsy Beyer. "Introduction" opens by posing the question of how a large system should be run. It sets out the sysadmin model and its flaws first, so SRE appears as the response; then defines SRE; then lists its tenets, which later chapters expand ("The rest of the book explores the SRE Way in detail").

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph: "Hope is not a strategy." "Systems do not run themselves," so how should a complex, large-scale system be run?

The Sysadmin Approach to Service Management

- The **sysadmin** approach assembles existing components into a service and staffs a team that responds to events; as traffic grows the team grows. Different skills split work into "development" and "operations."
- Advantages: easy to implement, familiar, ready talent pool, existing tools.
- **Direct costs**: manual change and event handling means the team "necessarily scales with the load." **Indirect costs** are subtler and often larger: the teams differ in skills, incentives, vocabulary, and assumptions about risk and stability, until the split reaches "trust and respect. This outcome is a pathology."
- The conflict centres on release speed: developers want launches adopted, ops want no breakage on their pager, and "most outages are caused by some kind of change." The result is "trench warfare": ops add launch gates; devs counter with "flag flips," "incremental updates," "cherrypicks," or sharding the product to dodge review.

Google's Approach to Service Management: Site Reliability Engineering

- "Conflict isn't an inevitable part of offering a software service." "SRE is what happens when you ask a software engineer to design an operations team." Treynor Sloss, a lifelong software engineer, designed the seven-person "Production Team" (2003) as he would want to work.
- Composition: 50–60% standard Google Software Engineers; 40–50% are 85–99% qualified plus rarer skills, most often UNIX internals and networking (Layer 1 to 3). No practical performance difference found. These people "quickly become bored by performing tasks by hand" and can write software to replace it.
- An ops-focused group scales linearly with service size, so the team "needs to code or it will drown": a **50% cap** on aggregate ops work (tickets, on-call, manual tasks), an upper bound, aiming at systems "automatic, not just automated." The other 50% is development; time is measured, and teams under 50% development shift ops burden to developers or add staff without adding ops duties.
- Gains: rapid innovation, acceptance of change, low cost, headcount that scales "sublinearly," cross-training with developers. Challenges: a small hiring pool, little industry guidance, and the need for management backing (e.g. halting releases once an error budget is depleted).
- DevOps or SRE? DevOps shares the principles (IT in every phase, automation, engineering applied to operations): "One could view DevOps as a generalization of several core SRE principles," or SRE as "a specific implementation of DevOps with some idiosyncratic extensions."

Tenets of SRE

- Responsibilities: availability, latency, performance, efficiency, change management, monitoring, emergency response, capacity planning, under rules that keep focus on engineering.
- **Durable focus on engineering**: ops work above 50% is redirected to developers (reassigned tickets, developers on-call) until load drops; this also pushes developers to build systems needing no manual intervention. Target at most two events per 8–12-hour on-call shift, so each gets handled and a postmortem written. Postmortems cover all significant incidents, including unpaged ones (they show monitoring gaps), blame-free.
- **Change velocity within the SLO**: the conflict of innovation versus stability is resolved by an **error budget**. "100% is the wrong reliability target for basically everything": users cannot tell 100% from

99.999% amid laptop, WiFi, ISP and power grid. The right target is a product question (user happiness, alternatives, usage at different levels). Budget is one minus the target (99.99% leaves 0.01%), spent "on anything we want, as long as we don't overspend it." Developers spend it on launches; phased rollouts and 1% experiments stretch it. SRE's goal is no longer "zero outages"; an outage is "an expected part of the process of innovation."

- **Monitoring**: a system requiring a human to read an email and decide "is fundamentally flawed." Valid outputs: **alerts** (act immediately), **tickets** (act within days), **logging** (recorded, read only if prompted).
- **Emergency response**: reliability depends on MTTF and MTTR; "Humans add latency." A recorded playbook gives "roughly a 3x improvement in MTTR" over "winging it," though it does not replace thinking engineers; Wheel of Misfortune exercises prepare people.
- **Change management**: roughly 70% of outages come from live changes; automate progressive rollouts, quick detection and safe rollback, raising both velocity and safety.
- **Demand forecasting and capacity planning**: forecast beyond acquisition lead time, include inorganic growth, load test regularly. Capacity is critical to availability, so SRE owns it and **provisioning**, which is costly and riskier than load shifting.
- **Efficiency and performance**: SRE controls provisioning, hence utilization, "a very, very big lever" on cost; slowdown equals lost capacity.

The End of the Beginning

- SRE "represents a significant break from existing industry best practices"; begun from familiarity, it became "a set of principles, a set of practices, a set of incentives, and a field of endeavor."

How this chapter builds the authors' case

it expands the Preface's definition (engineering applied to operations, reliability as focus) into the full programme in miniature. Its footnotes point ahead to "Disaster Role Playing" and "Communications: Production Meetings," and each tenet seeds a later chapter.

CHAPTER 2

The Production Environment at Google, from the Viewpoint of an SRE

Place in the argument — why this chapter, why this title

Written by JC van Winkel, edited by Betsy Beyer. The Preface advises starting with Chapters 2 and 3; this is the first. It describes Google datacenters, their "extra problems and opportunities," and "introduces terminology that is used throughout the book." It climbs from hardware to system software, software infrastructure and development environment, then ties them together in one sample service.

Hardware

- Compute sits mostly in Google-designed datacenters, hardware "the same across the board" (roughly).
- **Machine**: a piece of hardware (or perhaps a VM). **Server**: a piece of software implementing a service. Machines can run any server; Borg, the cluster operating system, allocates resources.
- Topology (Figure 2-1): tens of machines per rack; racks in a row; rows form a cluster; buildings house clusters; nearby buildings form a campus.
- Machines talk over a virtual switch of tens of thousands of ports (Clos fabric Jupiter, 1.3 Pbps bisection bandwidth); datacenters connect over the software-defined backbone B4.

System Software That "Organizes" the Hardware

- In a typical year thousands of machines and disks fail in a single cluster, so software must abstract failures away from users and service teams.
- **Managing Machines**: Borg runs jobs (servers or batch processes) of identical tasks, places them, monitors them, and kills and restarts malfunctioning ones, perhaps elsewhere. Since tasks move, BNS (Borg Naming Service) gives each a name resolving to IP:port. Jobs state resources; Borg bin-packs them across failure domains (not all tasks on one rack) and restarts tasks exceeding requests, since "a slowly crashlooping task is usually preferable."
- **Storage**: D (fileserver) under Colossus (cluster filesystem, successor to GFS) under database services: Bigtable (petabyte-scale NoSQL, eventually consistent replication), Spanner (SQL-like, global consistency), others with trade-offs (Chapter 26).
- **Networking**: OpenFlow SDN with cheap "dumb" switches and a central controller precomputing paths; BwE allocates bandwidth like Borg allocates compute; GSLB sends users to the closest datacenter with capacity at three levels: DNS (Chapter 19), user service, and RPC (Chapter 20).

Other System Software

- Chubby gives locks across locations via Paxos (Chapter 23), handles master election, and stores consistent data such as BNS mappings.
- Borgmon (Chapter 10) "scrapes" metrics for alerts, behavior comparison (did an update make it faster?) and capacity planning.

Our Software Infrastructure

- Code is heavily multithreaded; every server exposes an HTTP server for diagnostics.
- Services talk via Stubby RPC (open source: gRPC), even for local calls, easing refactoring. Protocol buffers are "3 to 10 times smaller, 20 to 100 times faster, and less ambiguous" than XML.

Our Development Environment

- "Development velocity is very important." A single shared repository; anyone can send a changelist (CL) to a component's owner; all software is reviewed.
- Parallel builds; each CL triggers tests on all dependents, notifying the owner if likely broken; some projects push-on-green.

Shakespeare: A Sample Service

- Finds where a word appears in Shakespeare: a batch MapReduce (map, shuffle, reduce) writing a Bigtable keyed by word, plus an always-up frontend.

- **Life of a Request** (Figure 2-4): DNS, GSLB, the Google Frontend reverse proxy, GSLB again to a Shakespeare frontend, RPC to a backend, Bigtable, then back, in “just a few hundred milliseconds.” Many failure points exist (a failing GSLB “would wreak havoc”); rigorous testing, careful rollout and graceful degradation keep it reliable.
- **Job and Data Organization**: a backend handles about 100 QPS; peak is about 3,470 QPS, so 35 tasks; with an update taking one and a machine failure another, 37, or **N + 2**. Peak by region: 1,430 North America, 290 South America, 1,400 Europe and Africa, 350 Asia and Australia; giving 17 tasks USA, 16 Europe, 6 Asia, but 4 (N + 1) in South America, accepting small latency risk to save 20% of hardware. Bigtable is replicated per region for resilience and latency; eventual consistency is fine since updates are rare.

How this chapter builds the authors’ case

it supplies vocabulary and systems later chapters assume, pointing forward to Chapters 10, 19, 20, 23 and 26. Its N + 1 versus N + 2 trade of hardware cost against latency risk previews the risk reasoning of Chapter 3.

PART II

PRINCIPLES

- The authors' own description: "This section examines the principles underlying how SRE teams typically work—the patterns, behaviors, and areas of concern that influence the general domain of SRE operations."

The theme of the Part

- The principles behind SRE work, with risk as the widest-angle lens: Chapter 3 is "the most important piece to read if you want to attain the widest-angle picture of what exactly SRE does, and how we reason about it."

How the chapters build the theme

- Reasoning and measurement: Chapter 3 (risk, error budgets, "usefully neutral approaches to service management"); Chapter 4 separates indicators from objectives from agreements; Chapter 6 on monitoring: "If you can't monitor a service, you don't know what's happening, and if you're blind to what's happening, you can't be reliable."
- Reducing work: Chapter 5 on toil, "mundane, repetitive operational work providing no enduring value, which scales linearly with service growth"; Chapter 7 on automation, with successes and failures.
- Change and design: Chapter 8 (release engineering, since "most outages result from pushing a change of some kind"); Chapter 9 (simplicity, "once lost, can be extraordinarily difficult to recapture").

What the Part contributes to the book

- The concepts (risk, SLOs, toil, monitoring, automation, releases, simplicity) that the practices in Part III apply.

CHAPTER 3

Embracing Risk

Place in the argument — why this chapter, why this title

Written by Marc Alvidrez (edited by Kavita Guliani); the section "Motivation for Error Budgets" is by Mark Roth. The Preface calls risk in many ways the key quality of the profession and tells readers to start with this chapter. The title signals that SRE accepts risk rather than fleeing it. The order runs from why not 100%, to how to manage risk, to how to measure it, to how services differ in tolerance, and ends by turning that tolerance into a working tool, the error budget.

Chapter opening — untitled section before the first sub-heading in the book

- Past a certain point, "increasing reliability is worse for a service (and its users) rather than better!" Extreme reliability limits how fast features ship and "dramatically increases their cost," which reduces the features a team can afford.
- Users cannot tell the difference: "a user on a 99% reliable smartphone cannot tell the difference between 99.99% and 99.999% service reliability!"
- So SRE seeks "to balance the risk of unavailability with the goals of rapid innovation and efficient service operations," optimizing users' overall happiness.

Managing Risk

- Cost is not linear: "an incremental improvement in reliability may cost 100x more than the previous increment." Two costs: **redundant machine/compute resources** (e.g. spare equipment for maintenance, parity blocks), and **opportunity cost** (engineers building risk-reducing rather than user-visible features).
- "We manage service reliability largely by managing risk," seen as a continuum. Equal weight goes to engineering reliability in and finding the right tolerance, enabling a cost/benefit placement for Search, Ads, Gmail, Photos. The goal is to align service risk with what the business will bear: "reliable enough, but no more reliable than it needs to be." The target is "both a minimum and a maximum"; exceeding 99.99% by much wastes chances to add features, clean up technical debt or cut operational cost. The payoff: "explicit, thoughtful risktaking."

Measuring Service Risk

- Google picks an objective metric. Failure effects (dissatisfaction, lost trust, revenue, reputation, press) are hard to measure, so the focus is **unplanned downtime**, expressed as the "nines" of availability; each additional nine is an order-of-magnitude step.
- Time-based availability (Equation 3-1) is uptime / (uptime + downtime); 99.99% allows 52.56 minutes down a year (see Appendix A).
- But for globally distributed services with fault isolation, some traffic is nearly always served, so availability is defined by **request success rate** (Equation 3-2): successful requests / total requests over a rolling window. Example: 2.5M requests a day at 99.99% allows 250 errors. Requests are not equal, yet the aggregate approximates unplanned downtime from the user's view.
- It also suits nonserving systems (batch, pipeline, storage, transactional): a batch job counts records processed successfully or not.
- Targets are usually quarterly, tracked weekly or daily to fix "meaningful deviations" (Chapter 4).

Risk Tolerance of Services

- At Google risk tolerance is "less clearly defined" than in safety-critical systems. SREs work with product owners to turn business goals into explicit engineering objectives. Consumer and infrastructure services are treated in turn.
- **Consumer services** usually have a product team, the best resource on reliability needs.
- Target availability depends on expected user level, revenue link, paid or free, competitors, consumer or enterprise. Google Apps for Work: an outage hits the enterprises too, so about 99.9% external quarterly target, a stronger internal one, and contract penalties. YouTube (2006): consumer-focused and growing rapidly, so a lower target because rapid feature development mattered more.
- Failure types: is a constant low failure rate or an occasional full outage worse? In a contacts app, failed profile pictures differ from showing one user's contacts to another; the latter justifies taking it

down while debugging. At the other end, the Ads Frontend accepted regular scheduled maintenance windows, counted as planned, not unplanned, downtime.

- Cost is often the key factor. Worked example: raising 99.9% to 99.99% (0.09% more availability) on $1M revenue is worth $1M * 0.0009 = $900; below $900 cost is worthwhile. Without a revenue translation, use the ISP background error rate (measured 0.01% to 1%): errors below that fall within the noise.
- Other metrics: AdWords latency must not slow search, treated as "an invariant"; AdSense need only avoid slowing the publisher's page, so ads can be "hundreds of milliseconds slower," allowing consolidation into fewer locations.
- **Infrastructure services** have multiple clients with differing needs. Bigtable: low-latency users want queues almost always empty; offline-analysis users want queues never empty for throughput. "Success for the low-latency user is failure for the user concerned with offline analysis." Ultra-reliable infrastructure is too expensive; instead partition into low-latency clusters (slack capacity) and throughput clusters (run hot), the latter at perhaps 10–50% of the cost. Key strategy: "explicitly delineated levels of service," which externalize cost so clients choose the cheapest adequate level (e.g. Google+ putting privacy-critical data in a Spanner-like store and optional data in a Bigtable-like one).
- Frontend infrastructure (reverse proxies, load balancers) is engineered for extreme reliability: "If a request never makes it to the application service frontend server, it is lost."

Motivation for Error Budgets

- Product development is judged on velocity; SRE on reliability, with information asymmetry amplifying the tension. It appears in fault tolerance, testing, push frequency, and canary duration and size. Decisions should not be driven by "politics, fear, or hope" ("Hope is not a strategy"); an objective, jointly agreed metric is needed.
- **Forming your error budget**: both teams define a quarterly budget from the SLO (Chapter 4). Product Management defines the SLO; a neutral third party, monitoring, measures actual uptime; the difference is the remaining "unreliability"; releases proceed while budget remains. Example: SLO 99.999% gives a 0.001% budget; failing 0.0002% spends 20%.
- **Benefits**: a common incentive for balancing innovation and reliability. If the budget is spent, releases halt while testing and resilience are funded (subtler options: slow releases or roll back). With a large budget developers take risks; with a nearly drained one they push for testing, becoming "self-policing" (this relies on SRE having authority to stop launches). Network and datacenter failures also consume budget. A team struggling to launch may loosen the SLO.
- **Key insights**: managing reliability is managing risk, which is costly; 100% is probably never the right target; error budgets align incentives and defuse outage discussions "without rancor."

> **How this chapter builds the authors' case**
> it develops Chapter 1's claim that 100% is the wrong reliability target into a full cost/benefit argument and introduces the error budget Chapter 1 mentions. It builds on Chapter 2's redundancy and cluster vocabulary, and points ahead to Chapter 4 for SLOs and Appendix A for availability tables.

CHAPTER 4

Service Level Objectives

Place in the argument — why this chapter, why this title

Written by Chris Jones, John Wilkes and Niall Murphy with Cody Smith, edited by Betsy Beyer. Chapter 3 introduced the error budget as derived from an SLO, and this chapter supplies the SLO and its relatives. Its premise: a service cannot be managed "correctly, let alone well" without knowing which behaviors matter and how to measure them. The order follows the three terms (indicators, objectives, agreements), first defined, then applied "in practice," using the Shakespeare service from Chapter 2.

Chapter opening — untitled section before the first sub-heading in the book

- **SLIs**, **SLOs** and **SLAs** say which metrics matter, what values we want, and how we react if we cannot deliver. Good metrics "drive the right action if something goes wrong."

Service Level Terminology

- "SLA" is overloaded in common use, so the terms are separated.
- **Indicators**: an SLI is "a carefully defined quantitative measure of some aspect of the level of service." Common: request latency, error rate, throughput, usually aggregated over a window into a rate, average or percentile. **Availability** is the fraction of time a service is usable, often well-formed requests that succeed ("yield"). "Nines": Google Compute Engine's published target is "three and a half nines," 99.95%.
- **Objectives**: an SLO is "a target value or range of values for a service level that is measured by an SLI", shaped SLI ≤ target or lower ≤ SLI ≤ upper. Example: average search latency under 100 milliseconds. You do not always choose the value (QPS is set by users), and SLIs interact: higher QPS often raises latency. Published SLOs set expectations; without them users form their own beliefs, causing over-reliance (as with Chubby) or under-reliance.
- **The Global Chubby Planned Outage** (Marc Alvidrez): global Chubby outages were so rare that service owners assumed it would never go down. The fix: SRE makes sure it "meets, but does not significantly exceed, its service level objective," synthesizing a controlled outage in any quarter without a true failure, to flush out unreasonable dependencies.
- **Agreements**: an SLA is "an explicit or implicit contract with your users that includes consequences of meeting (or missing) the SLOs." Test: ask "what happens if the SLOs aren't met?"; no explicit consequence means an SLO. SRE does not typically build SLAs (tied to business decisions) but helps avoid triggering consequences and defines the SLIs. Google Search has no public SLA, yet defining SLIs and SLOs is valuable with or without one.

Indicators in Practice

- **What Do You and Your Users Care About?** Do not use every trackable metric; too many hides what matters, too few leaves gaps. "A handful of representative indicators" suffice. User-facing serving systems: availability, latency, throughput. Storage: latency, availability, durability (Chapter 26). Pipelines: throughput and end-to-end latency. All: correctness, usually not SRE's to meet.
- **Collecting Indicators**: mostly server-side (Borgmon, Prometheus, log analysis), but client-side collection catches what server metrics miss, e.g. slow page JavaScript.
- **Aggregation**: even requests per second hides burst rates inside the window (200 requests/s in even seconds and 0 otherwise averages 100/s). Averaged latencies hide a long tail. "Most metrics are better thought of as distributions rather than averages." Figure 4-1: a typical request takes about 50 ms while 5% are 20 times slower, which averages miss. High percentiles show a plausible worst case, the 50th the typical case; people prefer a slightly slower system to one with high variance, so some teams track only high percentiles.
- **A Note on Statistical Fallacies**: prefer percentiles to the mean. Data are skewed (no response under 0 ms; a timeout caps successes), so mean and median can differ widely; do not assume normality unchecked.
- **Standardize Indicators**: reusable templates fixing aggregation interval and region, measurement frequency, which requests count, how data is acquired, and data-access latency.

Objectives in Practice

- Start from what users care about, not what is easy to measure, which gives less useful SLOs; working backward from objectives to indicators often works better.
- **Defining Objectives**: state how measured and when valid, e.g. "99% of Get RPC calls will complete in less than 100 ms." Use several targets for the curve's shape, and separate ones per workload class.
- Insisting on 100% "unrealistic and undesirable": it slows innovation and forces overly conservative solutions. Instead allow an error budget, "a rate at which the SLOs can be missed," tracked daily or weekly. "An error budget is just an SLO for meeting other SLOs!" The violation rate gives early warning and informs release decisions.
- **Choosing Targets** is not purely technical. Lessons: don't pick a target from current performance (it may lock in heroics); keep it simple; avoid absolutes; have as few SLOs as possible (if quoting one never wins a priorities argument, it is not worth having); perfection can wait, since a loose target tightened beats a strict one relaxed. "SLOs are a massive lever: use them wisely."
- **Control Measures**: monitor SLIs, compare to SLOs, decide if action is needed, work out and take it.
- **SLOs Set Expectations**: publishing them tells users whether the service fits their use. Two tactics: **keep a safety margin** (tighter internal than advertised SLO, giving room to respond to chronic problems and to trade performance for cost or maintainability) and **don't overachieve** (users build on reality, so deliberately take the system offline occasionally as Chubby did, throttle requests, or avoid being faster under light load).

Agreements in Practice

- Business and legal teams choose consequences; SRE explains the likelihood and difficulty of meeting the SLOs. Much SLO advice applies. Be conservative in what you advertise: "the broader the constituency, the harder it is to change or delete SLAs."

> **How this chapter builds the authors' case**
> it supplies the SLO that Chapter 3's error budget is defined against ("Motivation for Error Budgets"), reuses Chapter 2's Shakespeare service, and builds on Chapter 3's reasoning that a service should not exceed what is needed (the Chubby planned outage).

CHAPTER 5

Eliminating Toil

Place in the argument — why this chapter, why this title

Written by Vivek Rau, edited by Betsy Beyer. Chapter 1 set the 50% cap on "ops" work; this chapter names the misleading "operational work" as **toil** and defends the cap. The order: definition, case against toil, what counts as engineering, toil's limited virtues.

Chapter opening — untitled section before the first sub-heading in the book

- SRE wants time on long-term engineering, not operational work; because that term "may be misinterpreted," the book uses toil.

Toil Defined

- Toil is not "work I don't like to do" nor administrative chores, which are **overhead**. Grungy work with long-term value is not toil.
- Toil is work tied to running a production service that tends to be **manual**, **repetitive**, **automatable**, **tactical**, with **no enduring value**, and "O(n) with service growth." The closer work matches these, the likelier it is toil.
- Manual: running a script still costs hands-on time. Repetitive: first or second time is not toil. Automatable: a machine could do it, or the need could be designed away. Tactical: interrupt-driven and reactive; handling pager alerts is toil. No enduring value: the service is unchanged afterward. O(n): work grows linearly with size; a well-designed service grows an order of magnitude with zero additional work.

Why Less Toil Is Better

- Goal: toil below 50% of each SRE's time; at least 50% on engineering that reduces future toil or adds features. Toil "tends to expand if left unchecked and can quickly fill 100% of everyone's time." Engineering lets SRE scale sublinearly.
- **Calculating Toil**: on-call sets a floor: one week primary and one secondary per cycle gives 2/6 = 33% in a 6-person rotation, 2/8 = 25% in an 8-person one. Surveys show about 33% average toil, ranging from 0% to 80%.

What Qualifies as Engineering?

- Engineering is "novel and intrinsically requires human judgment," brings permanent improvement, follows a strategy, and lets the team handle more with the same staffing.
- Categories: software engineering; systems engineering; toil; overhead.
- At least 50% engineering, averaged over a few quarters or a year; toil is spiky, but a long-run average well below 50% means something is wrong.

Is Toil Always Bad?

- Toil can calm and give quick wins, and some is unavoidable: "fine in small doses," but "toxic when experienced in large quantities."
- Individually it causes career stagnation and low morale. For the organization it creates confusion about SRE's role, slows progress, sets precedent (devs shift operational tasks to SRE), promotes attrition and causes breach of faith with new hires.

Conclusion

- Eliminating "a bit of toil each week" steadily cleans up services and frees effort for scale and next-generation architecture: "Let's invent more, and toil less."

How this chapter builds the authors' case

it gives the reasoning behind Chapter 1's 50% cap and points forward to Chapter 11 for primary and secondary on-call.

CHAPTER 6

Monitoring Distributed Systems

Place in the argument — why this chapter, why this title

Written by Rob Ewaschuk, edited by Betsy Beyer. Chapter 4 defined the SLIs and SLOs a service is held to; this chapter covers the monitoring and alerting that tell the team when it is failing, with "guidelines for what issues should interrupt a human via a page." It runs from vocabulary and purpose, to realistic expectations, to the core distinctions (symptoms versus causes, black-box versus white-box), to what to measure, to simplicity, a checklist philosophy, and long-term case studies.

Chapter opening — untitled section before the first sub-heading in the book

- Google SRE has basic principles for monitoring and alerting; the chapter is about what deserves a page.

Definitions

- **Monitoring**: collecting, processing, aggregating and displaying real-time quantitative data. **White-box**: metrics from the system's internals (logs, handlers). **Black-box**: "testing externally visible behavior as a user would see it." **Alert**: a notification for a human, classed as tickets, email alerts or pages. **Root cause**: a defect that, if repaired, "instills confidence that this event won't happen again in the same way"; an incident may have several. Also defined: dashboard, node and machine, push.

Why Monitor?

- Reasons: long-term trends; comparison over time or experiment groups; alerting; dashboards (normally with the four golden signals); ad hoc debugging.
- Never alert just because "something seems a bit weird." Paging is "a quite expensive use of an employee's time"; too many pages lead people to ignore even a real one, and noise prolongs outages. Effective alerting has "good signal and very low noise."

Setting Reasonable Expectations for Monitoring

- Monitoring is a significant engineering endeavor: a 10–12-member team typically has one or two people mainly building it, yet SRE avoids anyone having to "stare at a screen to watch for problems."
- Google trends toward simpler, faster monitoring with better post hoc tools, avoiding "magic" that learns thresholds or detects causality. Complex dependency hierarchies have had limited success because infrastructure is constantly refactored.
- Some ideas are aspirational; the path from onset through page to debugging must be simple, and pager rules "simple to understand and represent a clear failure."

Symptoms Versus Causes

- Monitoring must answer what is broken (symptom) and why (a possibly intermediate cause). Table 6-1: serving HTTP 500s versus database servers refusing connections; slow responses versus overloaded CPUs or a crimped cable; private content world-readable versus a push that lost ACLs.
- This "what" versus "why" is "one of the most important distinctions in writing good monitoring with maximum signal and minimum noise."

Black-Box Versus White-Box

- Google combines heavy white-box with "modest but critical" black-box monitoring. Black-box is symptom-oriented and shows active problems; white-box also detects imminent problems and failures masked by retries.
- In layers, one person's symptom is another's cause (slow database reads: symptom for the database SRE, cause for the frontend SRE).
- White-box is essential for debugging; for paging, black-box forces discipline to nag only when a problem is ongoing and causing symptoms, but is "fairly useless" for imminent problems.

The Four Golden Signals

- "The four golden signals of monitoring are latency, traffic, errors, and saturation." If you can measure only four metrics of a user-facing system, use these.

- **Latency**: separate successful from failed requests but track error latency, since "a slow error is even worse than a fast error!" **Traffic**: demand in a system-specific metric (e.g. HTTP requests per second). **Errors**: explicit (500s), implicit (200 with wrong content) or by policy (slower than a committed one second). **Saturation**: how "full" the service is, emphasizing the most constrained resource; systems degrade before 100%, so a utilization target is essential. Rising 99th percentile latency is an early signal; saturation also covers predictions (a disk full in 4 hours).
- Page when one is problematic (for saturation, nearly) and the service is "at least decently covered."

Worrying About Your Tail (or, Instrumentation and Performance)

- Means mislead: at 100 ms average and 1,000 requests per second, 1% of requests might take 5 seconds, and "the 99th percentile of one backend can easily become the median response of your frontend."
- Collect request counts in latency buckets (roughly exponential, factors of about 3), not raw latencies.

Choosing an Appropriate Resolution for Measurements

- Granularity varies: CPU load over a minute hides spikes that drive tail latency, but for 99.9% annual uptime (about 9 hours downtime), probing more than once or twice a minute is unnecessary.
- Cheap high resolution: sample CPU each second into 5% buckets and aggregate every minute.

As Simple as Possible, No Simpler

- Requirements pile up, and monitoring can become "so complex that it's fragile." Keep rules that catch real incidents simple and reliable; remove rarely exercised configuration (less than once a quarter) and signals in no dashboard or alert.
- Keep collection, alerting and dashboards a standalone system; blending in profiling, debugging, log analysis and the like makes it fragile. Prefer "clear, simple, loosely coupled points of integration."

Tying These Principles Together

- Ask of each alert: does it detect an otherwise undetected condition that is urgent, actionable and actively or imminently user-visible? Can I ever ignore it? Does it definitely mean users are affected? Can I act, and is it urgent, automatable, a fix or a workaround? Are others paged too?
- Philosophy: urgency only a few times a day before fatigue; every page actionable and requiring intelligence ("If a page merely merits a robotic response, it shouldn't be a page"); pages about novel problems. Then white-box versus black-box matters little, but "it's better to spend much more effort on catching symptoms than causes."

Monitoring for the Long Term

- "Every page that happens today distracts a human from improving the system for tomorrow," which can justify a short-term hit to availability.
- **Bigtable SRE: A Tale of Over-Alerting**: the SLO rested on a synthetic client's mean, driven by a slow worst 5%; alerts fired voluminously and real problems were missed. The team improved Bigtable, temporarily used the 75th percentile as the SLO, and disabled email alerts, gaining room for real fixes.
- **Gmail: Predictable, Scriptable Responses from Humans**: alerts on single de-scheduled tasks were unmaintainable; the team built a tool to "poke" the scheduler and debated automating the loop, fearing it would delay a real fix. Managers must prioritize long-term fixes. "Pages with rote, algorithmic responses should be a red flag."
- **The Long Run**: heroics lead to burnout; a controlled short-term availability decrease can be a strategic trade. Page frequency is reviewed quarterly with management.

Conclusion

- A healthy pipeline is simple, pages primarily on symptoms, and keeps cause-oriented heuristics as debugging aids. Email alerts are of limited value; favor a dashboard.
- Long-term success means alerting on symptoms or imminent real problems, achievable targets, and monitoring that supports rapid diagnosis.

> **How this chapter builds the authors' case**
> it supplies the monitoring that makes Chapter 4's SLOs and Chapter 3's error budgets measurable,

echoes Chapter 1's alerts/tickets/logging tenet, and applies Chapter 5's warning against rote work to paging. It reuses Chapter 4's percentile argument.

CHAPTER 7

The Evolution of Automation at Google

Place in the argument — why this chapter, why this title

Written by Niall Murphy with John Looney and Michael Kacirek. Part II introduces the chapter as SRE's approach to automation, walked through case studies, both successful and unsuccessful. The title promises a history, not a recipe: the authors show how Google's attitude changed through cases (Ads MySQL, cluster turnup, Borg). The sub-headings first give the value (general, then Google-specific), then the use cases and a hierarchy of automation classes, then three stories, then the lesson on reliability, then advice.

Chapter opening — untitled section before the first sub-heading in the book

- Automation is "a force multiplier, not a panacea"; multiplying force does not make it accurate, so thoughtless automation "can create as many problems as it solves."
- Software automation beats manual operation in most circumstances, but better than either is "a higher-level system design requiring neither of them—an autonomous system."
- Value comes "from both what it does and its judicious application."

The Value of Automation

- **Consistency**: a human repeating an action hundreds of times will not do it the same way each time, which causes mistakes, data-quality issues and reliability problems. For well-scoped, known procedures, "the value of consistency is in many ways the primary value of automation."
- **A platform**: done properly, automation can be extended, applied to more systems or even spun out for profit. No automation is "a tax levied on the operation of a system." A platform centralizes mistakes (a bug is fixed once and forever) and exports metrics.
- **Faster repairs**: regular, successful automation lowers MTTR for common faults. Problems found late cost more, so catching them as they arise lowers total cost in a sufficiently large system.
- **Faster action**: humans are slower than machines for failover or traffic switching. Bad situations can get worse, hence automation should be scoped over well-defined domains. Many Google services "could not long survive without this automation."
- **Time saving**: the benefit is less immediately calculable than people think; once a task is encapsulated, anyone can run it. "Decoupling operator from operation is very powerful." Bironas: non-automatable processes mean "feeding the machines with the blood, sweat, and tears of human beings."

The Value for Google SRE

- Google's bias comes from planet-spanning scale, where "consistency, quickness, and reliability dominate."
- Google's production environment is complicated but uniform. Where vendors lacked an API, Google built its own, for long-term benefit. Owning the source code of everything SRE touches makes "own the product in production" easier.
- This is a "maximalist view": not every component should be automated and some systems began as prototypes. The chosen approach is platforms, seen as "necessary for manageability and scalability."

The Use Cases for Automation

- Automation is "metasoftware": software to act on software. Examples: account creation, cluster turnup and turndown, rollouts, runtime configuration changes.
- **Google SRE's Use Cases for Automation**: SRE's affinity is for running infrastructure and the lifecycle of systems, not the quality of the data passing over them (though it alerts on coarse changes such as half a dataset vanishing).
- Tools (Puppet, Chef, Perl) differ in abstraction level. The trade-off is classic: higher-level abstractions are easier to reason about, but a "leaky abstraction" makes you fail systemically and repeatedly. Example: a binary push is assumed atomic, but may leave binaries staged but not pushed, pushed but not restarted, or restarted but not verifiable. Most abstractions halt and call for intervention; "Truly bad automation systems don't even do that."

- Abstract deployment languages are more reusable and more of a common platform; generic rollout tools are sometimes the most tractable option.
- **A Hierarchy of Automation Classes**: ideally a system needs no external glue logic. Turnup automation maintained separately suffers "bit rot", and tight coupling fails because developers resist a test deployment for every change. Infrequently run automation (cluster failover) is fragile because of a long feedback cycle. The path: 1) no automation (manual master failover); 2) externally maintained system-specific (a script in an SRE's home directory); 3) externally maintained generic (a shared failover script); 4) internally maintained system-specific (the database ships its own script); 5) systems that need no automation (it notices problems and fails over itself).

Automate Yourself Out of a Job: Automate ALL the Things!

- Ads data lived in MySQL; by 2005–2008 it was mature and managed. The team wanted to migrate it to Borg to "completely eliminate machine/replica maintenance" and allow bin-packing of MySQL instances.
- A proof of concept ran in late 2008, but Borg tasks move once or twice a week: tolerable for replicas, unacceptable for masters. Master failover took 30–90 minutes. Manual failovers would give best-case 99% uptime, below the business requirement, and the error budget demanded under 30 seconds of downtime per failover, which no human-dependent procedure could reach. "Our only choice was to automate failover."
- In 2009 the "Decider" daemon did planned and unplanned failovers in under 30 seconds 95% of the time. This moved the team from optimizing for a lack of failover to "embracing the idea that failure is inevitable."
- Cost: applications needed far more failure-handling logic. Benefits: time on mundane operations fell by 95%, single-task outages no longer paged a human; schema changes were then automated and operational maintenance cost fell nearly 95%; about 60% of hardware was freed.
- Lesson: go "the extra mile to deliver a platform rather than replacing existing manual procedures."

Soothing the Pain: Applying Automation to Cluster Turnups

- Ten years earlier, the Cluster Infrastructure SRE team turned up a cluster about as often as it hired, so turnup served as training. Steps 4 (basic services, lock service, storage, compute) and 6 (resources for user-facing services) were extremely complex, with services of over a hundred subsystems. One misconfigured component is "a customer-impacting outage waiting to happen."
- A Bigtable cluster deliberately left its first (logging) disk unused; a year later automation took an unused first disk to mean no storage and wiped the machine, and all the data. Lesson: do not rely on implicit "safety" signals.
- **Detecting Inconsistencies with Prodtest**: hand-tuned flags caused hard-to-spot misconfigurations. Prodtest extended Python unit tests to real services, with dependent tests so one failure aborts the chain (Figure 7-1, a DNS service chain). Bugs extend others' Prodtests, and managers could predict "go live" dates (six or more weeks from network-ready to serving). Then came the mandate: five clusters network-ready on the same day, to be turned up in one week.
- **Resolving Inconsistencies Idempotently**: each test was paired with an idempotent fix, so fixes could run every 15 minutes (Figure 7-2). A small group could reach 1% of websearch and ads traffic in a week or two. In hindsight it was "deeply flawed": latency between test, fix and retest caused flaky tests, and non-idempotent fixes could leave inconsistent state.
- **The Inclination to Specialize**: automation varies in competence (accuracy), latency and relevance (proportion of the real process covered). It began competent, high-latency and relevant, run by service owners. A central turnup team cut latency, but with over a thousand changes a day the people hit by bugs were no longer domain experts, so it lost relevance and competence. "Automation code, like unit test code, dies when the maintaining team isn't obsessive about keeping the code in sync." Ugly incentives: the turnup team had none to reduce service owners' debt, a team not running automation had none to build systems easy to automate, and a product manager unaffected by poor automation prioritizes features. "The most functional tools are usually written by those who use them."
- A security mandate to cut SRE privileges replaced SSH with the authenticated, ACL-driven, RPC-based Local Admin Daemon (Admin Servers).
- **Service-Oriented Cluster-Turnup**: Admin Servers entered service teams' workflows, and SREs moved from home-directory scripts to peer-reviewed RPC servers. Each owning team supplies the contract (API) for turnup and may change its implementation; at network-ready, RPCs go to each Admin

Server. The result is low-latency, competent, accurate, and it held as the number of teams and services seemed to double each year. This repeated the five-step path of the hierarchy.

Borg: Birth of the Warehouse-Scale Computer

- Early clusters had "golden" binaries on master machines, then a descriptor file plus parallel SSH; Python scripts grew into a machine-state database with monitoring, managing machine lifecycle through repair.
- This automation was "profoundly limited" because abstractions were tied to physical machines. Borg treated a collection of machines as "a managed sea of resources," with cluster management an entity with API calls to a central coordinator, allowing batch and user-facing tasks to share machines.
- Results: continuous OS upgrades with constant effort, deviations fixed automatically; "Thousands of machines are born, die, and go into repairs daily with no SRE effort." Treating it as a software problem bought time to make cluster management "autonomous, as opposed to automated."
- Analogy: rescheduling resembles a process moving between CPUs, so it becomes an intrinsic feature, and turnup is just more schedulable capacity. The global computer "must be self-repairing" given constant failures. Moving up from manually triggered to automatically triggered to autonomous requires self-introspection.

Reliability Is the Fundamental Feature

- Introspection details should also be exposed to the humans troubleshooting. In aviation and industry, effective automation progressively removes humans from direct contact; when it fails, they cannot operate the system, their reactions lack practice and their mental models are wrong. This arises more in nonautonomous systems, where manual actions are presumed always available but eventually are not.
- Automation has been harmful at Google (the sidebar "Automation: Enabling Failure at Scale"): during rack decommissioning, a restarted run found the set of machines needing Diskerase empty, but the empty set was a special value meaning "everything," so almost all colo machines (the CDN) were wiped. Users were still served from Google datacenters (slightly higher latency, thanks to capacity planning). Reinstalling took two days; weeks went on sanity checks, rate limiting, and an idempotent decommission workflow.
- Yet more systems now need automation or autonomous behavior, at scale and otherwise: "Reliability is the fundamental feature."

Recommendations

- You need not be Google-scale: automation gives more than time saving, so it pays beyond a time-spent versus time-saved sum.
- The highest leverage is in the design phase; rapid shipping "rarely makes for a resilient system." Autonomous operation is hard to retrofit; decoupled subsystems, APIs and minimal side effects help.

How this chapter builds the authors' case

It follows Chapter 6 in Part II's order and cites Chapter 2 (uniform production environment) and Chapter 17 (cost of late-found problems), and it points forward to "Disaster Role Playing" as practice against lost skills. The three cases: MySQL on Borg a success, turnup a warning about where automation lives, Borg the move from automated to autonomous.

CHAPTER 8

Release Engineering

Place in the argument — why this chapter, why this title

Written by Dinah McNutt. Part II's introduction says most companies treat release engineering as an afterthought, though most outages result from pushing a change, and it is the best way to make releases consistent. The title names a job function. Sub-headings go from role to philosophy, pipeline, configuration, and lessons for non-Googlers.

Chapter opening — untitled section before the first sub-heading in the book

- Release engineering is "a relatively new and fast-growing discipline" that "can be concisely described as building and delivering software."
- Reliable services need reliable release processes: binaries and configurations built in a reproducible, automated way so releases aren't "unique snowflakes," and changes to the process are "intentional, rather than accidental."

The Role of a Release Engineer

- Release engineering is data-driven: tools report metrics such as release velocity (time for a change to reach production).
- They define best practices and make tools behave correctly by default, so teams avoid reinventing release "(poorly)."
- With SREs they develop strategies for canarying, pushing without interrupting services, and rolling back.

Philosophy

- Four principles.
- **Self-Service Model**: "teams must be self-sufficient." Teams control their own release process and frequency; releases are automatic and need engineers only when problems arise.
- **High Velocity**: frequent releases mean "fewer changes between versions," easing testing and troubleshooting. Some teams build hourly and select a build; others use "Push on Green" and deploy every build that passes all tests.
- **Hermetic Builds**: the same revision gives identical results on different machines; builds are "insensitive to the libraries and other software installed on the build machine" and use known versions of tools and dependencies. Old releases are rebuilt at the original revision plus chosen later fixes (**cherry picking**), with versioned build tools.
- **Enforcement of Policies and Procedures**: access control gates approving changes, creating and deploying releases, approving cherry picks and build-configuration changes. Nearly all changes need code review, and each release archives a report of its changes, which can expedite troubleshooting.

Continuous Build and Deployment

- **Rapid** is the automated release system delivering "scalable, hermetic, and reliable releases."
- **Building**: Blaze builds targets with their dependencies; every binary shows build date, revision and build identifier, tying it to a record of how it was built.
- **Branching**: major projects branch from the mainline at a revision and never merge back; fixes go to the mainline and are cherry picked, so "we know the exact contents of each release."
- **Testing**: continuous unit tests on the mainline; their targets should match those gating release, and releases should be cut at the last fully passing revision. Tests re-run on the release branch with an audit trail, because cherry picks may yield code that exists nowhere on the mainline.
- **Packaging**: MPM packages are named, hashed and signed; labels (dev, canary, production) move to the newest package, and Rapid's build-ID label makes name plus label unique.
- **Rapid**: configured by blueprints (build and test targets, deployment rules, owners) with role-based ACLs; workflows run serially or in parallel as Borg tasks (Figure 8-1). Typical release: branch at the integration revision; compile and test in parallel; system tests and canaries; log each step and report changes.
- **Deployment**: Rapid drives simple deployments; complicated ones use Sisyphus, an SRE-built rollout framework, given the build label by Rapid. "Our goal is to fit the deployment process to the risk

profile of a given service": hourly pushes in pre-production, exponential expansion from one cluster for large services, days across regions for sensitive infrastructure.

Configuration Management

- Close collaboration with SREs: it "may initially seem" simple but configuration changes are "a potential source of instability." All schemes keep configuration in the main repository with strict code review.
- **Mainline for configuration**: decouples binaries and configuration but causes skew between checked-in and running versions.
- **Same MPM package as binaries**: one package to install, but tight binding; suits few files or files changing every release.
- **MPM "configuration packages"**: the hermetic principle applied; two packages change independently (a flag first_folio changed to bad_quarto without a new binary build) and a label such as much_ado ties them together.
- **External store** (Chubby, Bigtable): for dynamic changes.

Conclusions

- Practices "can also be applied more widely"; with right tools, automation and policies releases can be "as painless as simply pressing a button."
- Google built its own tools because others "don't work at the scale we require"; but policies must be defined first, automated or not.
- **Start Release Engineering at the Beginning**: it "has often been an afterthought"; budget early since retrofitting costs more.

How this chapter builds the authors' case

Where Chapter 7 argues for automation platforms, this chapter shows one, with hermetic, repeatable, automated releases. Part II's introduction links it to stability, since most outages follow a change; the chapter itself ties frequent releases to fewer changes per release. Its closing plea to involve release engineering early is followed in Part II by Chapter 9 on simplicity.

CHAPTER 9

Simplicity

Place in the argument — why this chapter, why this title

Written by Max Luebbe. Part II's introduction calls simplicity a key principle of effective software engineering, hard to recapture once lost. The epigraph (Hoare): reliability's price is the pursuit of utmost simplicity. Sub-headings run from balance, to code, to structure.

Chapter opening — untitled section before the first sub-heading in the book

- Software is "inherently dynamic and unstable"; it is stable only in a vacuum, with no code, hardware, library or user-base change.
- So SRE's job is "to keep agility and stability in balance in the system."

System Stability Versus Agility

- Sometimes stability is rightly sacrificed: "exploratory coding" with a set shelf life can skimp on testing since it never ships.
- Otherwise SREs want balance, with little impact on developer agility. Reliable processes "tend to actually increase developer agility": rapid, reliable rollouts make changes easy to see, so bugs are found and fixed sooner.

The Virtue of Boring

- "Boring" is positive in software. "Surprises in production are the nemeses of SRE."
- **Essential complexity** cannot be removed from the problem; **accidental complexity** can be fixed with engineering effort. SRE should push back on it and eliminate it in systems they onboard.

I Won't Give Up My Code!

- Commenting code out or flag-gating it are "terrible suggestions": source control reverses changes, and flagged-off code is "a metaphorical time bomb" (Knight Capital).
- "Every new line of code written is a liability"; so scrutinize purpose, remove dead code, detect bloat in testing.

The "Negative Lines of Code" Metric

- Every changed line risks defects; a smaller project is easier to understand and test, so be wary of new features.

Minimal APIs

- Perfection comes when nothing is left to take away. Fewer methods are easier to understand and let us do the core problem better: "In software, less is more!"

Modularity

- Loose coupling between binaries, or binaries and configuration, promotes agility and stability: a bug can be fixed and pushed independently.
- Versioning API changes lets developers upgrade safely. Avoid "util" or "misc" binaries; each should have a clear, well-scoped purpose.

Release Simplicity

- It is easier to understand one change than a batch; with 100 unrelated changes, finding the performance culprit is costly. Small batches are faster and more confident, like gradient descent.

A Simple Conclusion

- "Software simplicity is a prerequisite to reliability." Saying "no" to a feature keeps focus on innovation.

How this chapter builds the authors' case

Its release section makes the small-change argument that Chapter 8 makes for frequent releases, though the text does not cross-reference it. Part II's introduction presents it as a principle of all

effective software engineering, not only reliability work.

PRACTICES

- The authors' own description: SREs "run services" and "are ultimately responsible for the health of these services"; the section "addresses the theory and practice of an SRE's day-to-day activity: building and operating large distributed computing systems."

The theme of the Part

- Service health as a hierarchy, "in much the same way that Abraham Maslow categorized human needs": from what a system needs "to function as a service at all" to the higher levels that allow "taking active control of the direction of the service rather than reactively fighting fires." It was made explicit when Google SREs helped with the launch of healthcare.gov. The Part walks up this Service Reliability Hierarchy (Figure III-1) "from most basic to most advanced."

How the chapters build the theme

- Monitoring (Chapter 10): "Without monitoring, you have no way to tell whether the service is even working."
- Incident Response (Chapters 11–14): on-call as a tool toward the larger mission; structured troubleshooting; resisting ad hoc response; managing incidents.
- Postmortem and Root-Cause Analysis (Chapters 15–16): blameless postmortems, and the outage tracker.
- Testing (Chapter 17): "an ounce of prevention is worth a pound of cure."
- Capacity Planning (Chapters 18–22): the Auxon case study, then load balancing at the frontend and in the datacenter, overload, and cascading failures.
- Development (Chapters 23–26): distributed consensus, Cron, data pipelines, data integrity.
- Product (Chapter 27): reliable launches "from Day Zero", the point of "a workable product."

What the Part contributes to the book

- The day-to-day practice behind the Part II principles, ordered so each level rests on the one below.

CHAPTER 10

Practical Alerting from Time-Series Data

Place in the argument — why this chapter, why this title

Written by Jamie Wilkinson. Part III puts Monitoring at the bottom of the Service Reliability Hierarchy, since without it you cannot tell whether the service even works, and names this chapter as its tools-and-philosophy chapter. Chapter 6 gave implementation-agnostic principles; the title promises the practical counterpart, a real system, Borgmon, turning time-series into alerts. Sub-headings follow the data from origin to ten years on.

Chapter opening — untitled section before the first sub-heading in the book

- Monitoring, "the bottom layer of the Hierarchy of Production Needs, is fundamental to running a stable service." It enables rational decisions about change, the scientific method in incident response, and measuring alignment with business goals (Chapter 6).
- Large systems are hard to monitor: many components, and a need for a low maintenance burden. Google needs the distribution of response times to find the latency tail.
- Alerting on single-machine failures is "unacceptable because such data is too noisy to be actionable"; systems should "aggregate signals and prune outliers" while retaining granularity to inspect components.
- Over 10 years Google moved from custom check scripts to making time-series collection "a first-class role," with a rich language for charts and alerts.

The Rise of Borgmon

- Borgmon was built shortly after Borg (2003) to complement it. Open source tools (e.g., Prometheus) are very similar.
- Instead of custom scripts, Borgmon uses a common data exposition format: mass collection with low overhead and no subprocess or connection-setup cost. This is **white-box monitoring**. Charts and alerts use simple arithmetic, and history can feed alert computation. This serves Chapter 6's goal of simplicity.
- Metrics are standardized as **varz**: all metrics from a target in one HTTP fetch (lines like http_requests 37).
- A Borgmon can collect from other Borgmon, building hierarchies following the service topology: typically one per cluster and a pair globally.

Instrumentation of Applications

- /varz lists exported variables as plain text key-value lines; map-valued variables add labels (http_responses map:code 200:25 404:0 500:12). Adding a metric takes one declaration in code.
- The schemaless textual interface makes instrumentation easy, but decoupling variable definition from use in rules needs careful change management; this was satisfactory because tools validate and generate rules.
- **Exporting Variables**: each major Google language has an implementation registering with the HTTP server built into every binary.

Collection of Exported Data

- Borgmon finds targets by name resolution (service discovery cuts maintenance), fetches /varz at intervals, stores values in memory, and spreads collection across the interval so targets aren't in lockstep.
- Synthetic variables per target (name resolved, responded to collection and health check) make rules for unavailable tasks easy.
- Scraping over HTTP seems at odds with SNMP's design to keep working when other network applications fail, but this is rarely an issue; collection failure itself can be a signal.

Storage in the Time-Series Arena

- Data is in an in-memory database, checkpointed to disk: (timestamp, value) points in chronological lists called time-series, each named by a unique set of name=value labels (Figure 10-1).

- Memory is a fixed block, the **time-series arena**, with a garbage collector expiring oldest entries; the span of data is the **horizon**. Datacenter and global Borgmon hold about 12 hours ("a magic number"). At about 24 bytes a point, 1 million series for 12 hours at 1-minute intervals fit in under 17 GB. Older data goes to the slower, cheaper TSDB.
- **Labels and Vectors**: series are vectors of numbers (Figure 10-2); a series' name is a **labelset**. Minimum labels: var, job, service, zone. Together they form the variable expression.
- A query need not name all labels; dropping instance returns a vector of matching series. Adding [10m] returns ten minutes of history.

Rule Evaluation

- "Borgmon is really just a programmable calculator, with some syntactic sugar that enables it to generate alerts"; collection and storage are "necessary evils."
- Centralizing rule evaluation, not forked subprocesses, runs computations in parallel and keeps configuration small.
- Rules are algebraic expressions computing series from series, over time (one series' history) and space (label subsets of many series). Exported runtime metrics allow "monitoring the monitoring."
- **Aggregation** is "the cornerstone of rule evaluation in a distributed environment": summing a job's task series to treat the job as a whole; total queries-per-second is the sum of the rates of all query counters (the sum of rates defends against counter resets).
- Counters only increase; gauges take any value. Counters are better because they don't lose meaning for events between samples, which a gauge is likely to miss.
- Worked example: alert when non-200 rates over request rates exceed a value. Aggregate response-code rates across tasks, sum the non-200 codes, divide by the request rate. Each output is appended as a new series whose history can be inspected.
- Rules: task:http_requests:rate10m = rate over [10m]; dc:http_requests:rate10m = sum without instance of it (rates 1, 0.9, 1.1, 0, 1 give 4). rate() is total delta over total time between earliest and latest values.
- Names are colon-separated triplets: aggregation level, variable, operation. The error rules give dc:http_errors:ratio_rate10m, "datacenter HTTP errors 10 minute ratio of rates" (0.15 in the sample), comparable with the SLO of Chapter 4.
- Results stay in the arena for ad hoc querying, useful when on-call.

Alerting

- A rule is true (triggers) or false. Alerts can "flap," so a minimum duration, typically at least two evaluation cycles, avoids false alerts from missed collections.
- Example: ratio above 1% and errors above 1, for 2m, ErrorRatioTooHigh, severity=page. With 0.15 but errors not above 1, it stays inactive; once above, it pends two minutes, then fires.
- The central **Alertmanager** routes notifications and can inhibit alerts when others are active, deduplicate alerts with the same labelsets, and fan in or out by labelset.
- Per Chapter 6, page-worthy alerts go to on-call, subcritical ones to ticket queues, the rest to dashboards. Chapter 4 covers alert design.

Sharding the Monitoring Topology

- Collecting from every task globally is a bottleneck and single point of failure, so Borgmon streams time-series between Borgmon, cheaper than text varz.
- Typical: two or more global Borgmon (diversity against maintenance and outages) and one per datacenter; larger deployments split a scraping layer from a DC aggregation layer (Figure 10-3). Upper tiers filter what they stream, building local caches that can be drilled into.

Black-Box Monitoring

- White-box rules know the internals, showing quickly what fails, which queues are full and where bottlenecks are. But "You only see the queries that arrive at the target"; those lost to DNS errors are invisible, those lost to a crash "never make a sound." "You can only alert on the failures that you expected."
- **Prober** runs a protocol check on a target, can validate payloads, extract values as time-series (e.g., response-time histograms by operation and payload size), and alerts via Alertmanager or exports varz.

Maintaining the Configuration

- Separating rules from targets lets the same rules apply to many targets; language templates form reusable libraries, reducing bugs.
- Borgmon supports unit and regression tests with synthesized series; the Production Monitoring team's continuous integration runs them and ships configuration to all Borgmon, which validate it before accepting.
- Two template classes: those codifying the emergent schema of a code library's variables, and aggregation libraries modeling service topology; groupings follow shared fate.

Ten Years On...

- Borgmon "transposed the model of check-and-alert per target into mass variable collection and a centralized rule evaluation." The monitored system scales independently of the size of alerting rules.
- "Ensuring that the cost of maintenance scales sublinearly with the size of the service is key to making monitoring (and all sustaining operations work) maintainable."
- Borgmon remains internal, but the idea is now accessible through open source tools.

How this chapter builds the authors' case

It builds on Chapter 6 (white-box versus black-box, simplicity, paging versus tickets) and points to Chapter 4 (SLOs, alert design) and Chapter 2 (BNS). In Part III it supplies the monitoring layer at the base of the Service Reliability Hierarchy, before Chapter 11 on on-call. Its closing point, that maintenance cost must scale sublinearly, is called a theme recurring in all SRE work.

CHAPTER 11

Being On-Call

Place in the argument — why this chapter, why this title

Written by Andrea Spadaccini. Part III says on-call is a tool for the larger mission and for staying in touch with how distributed systems work and fail, and that this chapter explains how on-call is balanced with other duties. It is the incident-response layer after monitoring (Chapter 10), which feeds the pager. Sub-headings go from what on-call is to balance, safety, overload and underload.

Chapter opening — untitled section before the first sub-heading in the book

- On-call is "a critical duty," but pitfalls in organizing it can seriously harm services and teams. The chapter gives the tenets Google SRE developed.

Introduction

- In IT, on-call was historically done by dedicated Ops teams. Google SREs are on-call for their services but differ from purely operational teams by emphasizing engineering, since the problems are at a scale "intractable without software engineering solutions."
- Purely operational work is capped at 50%; the rest goes to engineering projects that scale the team through automation.

Life of an On-Call Engineer

- On-call engineers manage outages and perform or vet production changes, available within minutes per agreed paging response times: typically 5 minutes for user-facing or time-critical services, 30 for less sensitive ones.
- Response time follows availability: 4 nines per quarter (99.99%) allows around 13 minutes of downtime, so reaction must take minutes; relaxed SLOs allow tens of minutes.
- Paging events take priority over almost everything, project work included; nonpaging events are less urgent (Chapter 29).

Balanced On-Call

- Quantity is the percent of time on-call; quality is the number of incidents per shift.
- **Balance in Quantity**: at least 50% engineering; of the rest, no more than 25% on-call, up to 25% other operational work. With primary and secondary and week-long shifts, a single-site team needs a minimum of eight engineers; a dual-site team, six each.
- Multi-site is preferred when work justifies growth: "follow the sun" avoids night shifts, which harm health, and a limited rotation keeps engineers in touch with production. It costs communication overhead.
- **Balance in Quality**: an incident is related events and alerts with one root cause and one postmortem. Handling one takes on average 6 hours, so a 12-hour shift allows at most 2 incidents per day. Pages should be flat, with a likely median of 0; a component paging daily will probably break something else. If exceeded for a quarter, corrective measures are needed (Chapter 30).
- **Compensation**: time-off or cash, capped; the cap limits an individual's on-call and burnout.

Feeling Safe

- Of two modes of thinking, intuitive and rapid or rational and deliberate, the second is more likely to produce good results in complex outages. So stress must be reduced: stress hormones (cortisol, CRH) impair decisions and push toward "unreflective and unconsidered (but immediate) action"; the fourth identical page tempts confirmation bias.
- Intuition "can be wrong"; habitual responses "can be disastrous."
- Key resources: clear escalation paths, well-defined incident-management procedures, and a blameless postmortem culture.
- For multi-team incidents or unbounded duration, a formal protocol (Chapter 14) is supported by a web tool.
- Postmortems focus "on events rather than the people"; recognizing automation opportunities is one of the best ways to prevent human errors.

Avoiding Inappropriate Operational Load

- **Operational Overload**: team and leadership set concrete quarterly objectives; loaning an experienced SRE helps (Chapter 30). Symptoms should be measurable (daily tickets < 5, paging events per shift < 2).
- Misconfigured monitoring is a common cause: paging alerts should match symptoms threatening SLOs, and "all paging alerts should also be actionable." Aim for 1:1 alert/incident.
- If developers' changes cause noise, set common goals. In extreme cases SRE may "give back the pager," making developers exclusively on-call until standards are met; rare, and temporary when some pages are routed to developers during multi-quarter fixes. This "attests to the balance of powers"; the tension between reliability and feature velocity (Chapter 1) is typically resolved to the benefit of the company.
- **A Treacherous Enemy: Operational Underload**: out of touch with production, engineers become over- or underconfident and gaps appear only in incidents. Size teams so each is on-call at least once or twice a quarter; "Wheel of Misfortune" (Chapter 28) and DiRT help.

Conclusions

- A guideline for all Google SRE teams, using engineering work as the primary means to scale production responsibilities.

> **How this chapter builds the authors' case**
>
> It carries forward the 50% cap on operational work and Chapter 1's tension between SRE and development, here resolved by giving back the pager. It points to Chapter 29 (interrupts), 30 (overload), 14 (incident management) and 28 (Wheel of Misfortune). Part III places it after monitoring and before troubleshooting (Chapter 12).

CHAPTER 12

Effective Troubleshooting

Place in the argument — why this chapter, why this title

Written by Chris Jones (sidebar by Randall Bosetti). Part III's introduction says that figuring out what's wrong is the first step of incident response and offers this chapter as the structured approach, before emergency response and incident management. The title promises effectiveness: troubleshooting as learnable, not innate. The order is theory, practice with a running example, a case study, then making it easier.

Chapter opening — untitled section before the first sub-heading in the book

- Troubleshooting is critical for operators of distributed systems but often viewed as innate, because for those who do it often it is ingrained and hard to explain, "much like explaining how to ride a bike." The authors believe it is "both learnable and teachable."
- Novices are tripped up because it depends on two factors: generic troubleshooting skill and solid knowledge of the system. First principles alone work but are less efficient and effective than understanding how things are supposed to work; "there's little substitute to learning how the system is designed and built."

Theory

- Troubleshooting is an application of the hypothetico-deductive method: given observations and a theoretical basis for system behavior, iteratively hypothesize causes and test them.
- In the idealized model (Figure 12-1): a problem report; look at telemetry and logs for current state; combined with knowledge of construction, operation and failure modes, identify possible causes. Test either by comparing observed state with theories for confirming or disconfirming evidence, or by actively treating the system, changing it in a controlled way and observing. Repeat until a root cause is found, then take corrective action and write a postmortem.
- **Common Pitfalls** (in Triage, Examine and Diagnose, often from lack of system understanding): looking at irrelevant symptoms or misunderstanding metrics; misunderstanding how to change the system safely to test; wildly improbable theories or latching on to past causes; spurious correlations.
- The first two are fixed by learning the system and common distributed-systems patterns. The third by remembering not all failures are equally probable, "when you hear hoofbeats, think of horses not zebras," and preferring simpler explanations.
- Correlation is not causation: packet loss and failed disks can share a power outage as cause; in large systems coincidental correlations are inevitable.

In Practice

- Practice is never as clean as the model, but some steps make it less painful.
- **Problem Report**: every problem starts with one, an automated alert or a colleague saying "The system is slow." It should give expected behavior, actual behavior and, if possible, how to reproduce. Ideally a consistent form in a bug tracker. Google opens a bug for every issue, creating a log; reporting directly to a person lowers visibility and quality and concentrates load on those reporters know.
- **Shakespeare Has a Problem** (running example): on-call for the Shakespeare search service, you get alert ShakespeareBlackboxProbe_SearchFailure: the prober hasn't found results for "the forms of things unknown" for five minutes.
- **Triage**: severity ranges from one user with a workaround to a global outage; respond in proportion (all-hands emergency for the latter is right, for the former overkill, Chapter 14). In a major outage "Ignore that instinct!" to find the root cause at once; instead make the system work as well as it can: divert traffic from a broken cluster, drop traffic to prevent cascading failure, disable subsystems. "Stopping the bleeding should be your first priority," while still preserving evidence such as logs. Like pilots taught to fly the airplane first, if a bug threatens unrecoverable data corruption, freezing the system may be better.
- **Examine**: use monitoring metrics (Chapter 10) and graphs of time-series to find correlations (beware false ones). Logging: text logs for real-time debugging, structured binary logs for retrospective analysis; verbosity levels adjustable on the fly; sampling; a selection language ("show me operations

that match X"). Tracing with Dapper follows requests through the stack. Exposing current state: endpoints show recent RPCs and error and latency histograms; Borgmon servers show their rules.

- **Debugging Shakespeare**: the prober sends an HTTP GET to /api/search with search_text and expects a 200 and an exact JSON payload. About half of the last 10 minutes' probes succeeded, with no pattern; the prober doesn't show failure output. Manual curl gets 502 Bad Gateway and an X-Request-Trace header listing backend servers, so you can examine those backends.
- **Diagnose**: design knowledge helps hypotheses, but generic practices help anyway.
- *Simplify and reduce*: components with well-defined interfaces can be checked by the data between them; inject known test data at each step (black-box testing). A reproducible test case speeds debugging and may run in nonproduction with riskier techniques. Divide and conquer: work from one end of a stack to the other, or bisect, splitting the system in half and repeating.
- *Ask "what," "where," and "why"*: a malfunctioning system is still trying to do something; find what, why, where resources go (like the "Five Whys"). Example: a Spanner cluster with timing-out RPCs; tasks use all CPU; profiling shows log sorting; the time goes to a regular expression. Solutions: avoid backtracking, search for similar patterns, consider RE2, which guarantees linear runtime growth.
- *What touched it last*: "a working computer system tends to remain in motion until acted upon by an external force," such as a configuration change or load shift. Log deployments and configuration changes at all layers and annotate graphs with them (Figure 12-2). In the example the 502 with backend list rules out the API frontend and load balancers, so focus on backends.
- **Test and Treat**: rule hypotheses in or out experimentally. Test design: mutually exclusive alternatives; decreasing order of likelihood, considering risk; beware confounding factors and side effects; some tests are only suggestive. Take clear notes of ideas, tests and results, and make changes systematically so you can restore the pre-test setup.
- **Cure** (in the book after the sidebar): ideally causes narrow to one, but proving it by reproducing at will is hard in production (joint causes, path dependence, downtime), so often only probable factors are found; a non-production copy helps at a cost. Then write a postmortem of what went wrong, how it was tracked, fixed and prevented.

Negative Results Are Magic

- A negative result is an experiment where the expected effect is absent. "Realizing you're wrong has much value." Negative results are conclusive: they tell us something certain about production, the design space or performance limits (a web server handling only ~800 of 8,000 needed connections), and later teams can start from that.
- Publishing negative results improves the data-driven culture and reduces bias in metrics; SRE learned this with postmortems. Be skeptical of any document that doesn't mention failure. "Above all, publish the results you find surprising so that others—including your future self—aren't surprised."

Case Study

- App Engine (part of Google Cloud Platform): an internal customer's content-management app showed dramatic increases in latency, CPU usage and number of running processes; no code change correlated, and traffic had not risen (Figure 12-3). Latency rose nearly an order of magnitude and CPU and processes nearly quadrupled (Figs 12-4 to 12-6).
- Traffic and configuration changes (the usual causes) were ruled out: the spike was brief, the change came on a Saturday with no pushes in flight, and no other apps were affected. App Engine's developers noticed a correlation with a datastore call, merge_join, suggesting poor indexing. Dapper traces were used to find properties to index. Slow static-content requests, which skip the datastore, showed the correlation was spurious.
- Requests spent about 250 ms before the first RPC doing something unknown; SRE does not profile app code, and Dapper traces only RPCs. They chose not to solve it yet: with a public launch the next week, they recommended the most CPU-rich instance type, reducing latency enough to launch and investigate later.
- A change in the type of work was suspected: a common antipattern where instances cache datastore objects in memory and request time scales with their number. The developers instrumented the app and found a per-request whitelist check using an in-memory cache.
- Root cause: a long-standing access-control bug created a whitelist object in the datastore whenever one path was accessed; a pre-launch security scanner generated thousands in half an hour, each

checked on every request, with no RPCs. Fixing the bug and removing the objects restored performance.

Making Troubleshooting Easier

- Build observability, white-box metrics and structured logs, into each component from the ground up; design systems with well-understood, observable interfaces.
- Keep information consistent, e.g., a unique request identifier across RPCs.
- Many troubleshooting needs come from misrepresenting reality in code or environment changes; simplifying, controlling and logging such changes reduces and eases it.

Conclusion

- A systematic approach, “as opposed to relying on luck or experience,” makes troubleshooting understandable to novices, can bound time to recovery and gives users a better experience.

How this chapter builds the authors’ case

It uses Chapter 10’s exported variables, Borgmon and monitoring as the telemetry for the Examine step, and cites Chapter 28 (first principles as a way to learn a system), Chapter 14 (all-hands emergencies) and Chapter 29. Part III places it as the first step of incident response after on-call; Chapters 13 and 14 add emergency response and incident management.

CHAPTER 13

Emergency Response

Place in the argument — why this chapter, why this title

Written by Corey Adam Baye. The Part III intro puts incident response after monitoring and warns against responding ad hoc on adrenalin; this chapter and Chapter 14 are its answer. The title names the moment itself. Sub-headings run from calm advice, through three real failures grouped by trigger (test, change, process), to the lessons and habits drawn from them.

Chapter opening — untitled section before the first sub-heading in the book

- "Things break; that's life." How people respond to an emergency sets organizations apart, and few respond well naturally: it takes preparation and "periodic, pertinent, hands-on training", backed by board and management.

What to Do When Systems Break

- "First of all, don't panic!" You are trained, and typically no one is in physical danger, "only those poor electrons". If overwhelmed, pull in more people, even the entire company, and follow your incident response process (Chapter 14).

Test-Induced Emergency

- SREs break systems on purpose to improve reliability; sometimes assumptions and results are "worlds apart".
- Details: to flush out hidden dependencies, access to one database out of a hundred in a large MySQL setup was blocked. Within minutes, internal and external users couldn't reach key systems. SRE aborted, but the permissions rollback failed; they restored permissions to replicas and failovers while developers fixed the flaw in the application-layer library. Access returned within an hour.
- Went well: affected services escalated at once; parallel efforts; quick follow-up and periodic retesting.
- Learned: the reviewed test reflected insufficient understanding of the dependencies. The incident response process, only weeks old, wasn't followed. Rollback procedures were untested, which lengthened the outage; now they must be tested before large tests.

Change-Induced Emergency

- Despite tests, scale makes it "impossible to anticipate every dependency or interaction".
- Details: a global Friday push to the abuse-protection infrastructure triggered a crash-loop bug across the externally facing fleet, and internal applications failed too.
- Response: alerts fired within seconds; engineers retreated to "panic rooms" with backup production access. Within five minutes the push engineer rolled back and services began to recover; an incident was declared within 10 minutes. Some services took up to an hour.
- Went well: fast detection (though alerts spammed on-calls); clear updates; out-of-band communications and command-line tools worked (engineers needed more familiarity); rate-limiting of full updates may have throttled the crash-loop; and luck, as the push engineer was watching real-time channels.
- Learned: an earlier thorough canary never exercised a rare keyword combined with the new feature; this change "wasn't considered risky" and got a lighter canary. Thorough canarying matters "regardless of the perceived risk". Also, "Google relies upon our own tools", whose stack sat behind crash-looping jobs.

Process-Induced Emergency

- Automation is so efficient it can be "a bit frightening"; here "moving fast was not such a good thing".
- Details: in routine testing, two consecutive turndown requests were sent for one installation; a bug on the second sent all machines in all such installations globally to the Diskerase queue (Chapter 7).
- Response: on-call drained traffic from wiped locations, then disabled all team automation when pagers fired worldwide. Within an hour all traffic was diverted. Recovery was the hard part: congested links were mitigated, the first installation was rebuilt within three hours, and a manual three-step reinstall restored most capacity in three days.

- Went well: large installations, managed differently, took the full load; turndown itself worked; incident protocols had "matured considerably in the year since the first outage".
- Learned: the turndown server lacked sanity checks; an empty rack response became an empty filter, wiping everything: "sometimes zero does mean all." Reinstalls were slow (TFTP at the lowest QoS, BIOS halting or rebooting) and the reinstall infrastructure, with a regression, couldn't handle thousands of simultaneous setups; owners retuned it.

All Problems Have Solutions

- A solution exists even if not obvious to the person "whose pager is screaming"; "cast your net farther", involve others, and use the person with the most state. Afterwards clean up and write up.

Learn from the Past. Don't Repeat It.

- Keep a History of Outages: be thorough and honest, seek strategic prevention, publish postmortems, follow up on actions.
- Ask the Big, Even Improbable, Questions: What If…? Power, flooding, a dark datacenter, a compromised server: "Who will write the check?"
- Encourage Proactive Testing: "theory and reality are two very different realms"; better a reviewed test with your best people close than 2 a.m. Saturday.

Conclusion

- Different triggers, shared traits: no panic, others pulled in, learning from earlier outages, documented failure modes, proactive testing. The cycle brings incremental improvement, and applies to any organization.

> **How this chapter builds the authors' case**
>
> It points forward to Chapter 14 (the incident response process) and Chapter 15 (postmortems), and back to the Diskerase example of Chapter 7. The test incident failed to follow the process; the automation incident showed it had matured.

CHAPTER 14

Managing Incidents

Place in the argument — why this chapter, why this title

Written by Andrew Stribblehill. Chapter 13 urged following an incident response process; this chapter supplies it. The order is a story told twice: the incident going wrong, why, the process elements that fix it, the same incident replayed well, then when to declare one and a summary.

Chapter opening — untitled section before the first sub-heading in the book

- Effective incident management limits disruption and restores operations fast; if you haven't "gamed out" your response, principled management "can go out the window".

Unmanaged Incidents

- Mary, on-call at The Firm, is paged at 2 p.m. Friday: a datacenter stops serving; then a second and a third of five; the rest overload.
- She reverts the release, to no effect, and calls Josephine (3:30 a.m. her time). Sabrina and Robin poke around, "just looking". Executives demand ETAs.
- Malcolm, called by Josephine, deploys a CPU-affinity change to production; the servers restart "and then died".

The Anatomy of an Unmanaged Incident

- Everybody was "doing their job, as they saw it". Three hazards: Sharp Focus on the Technical Problem (Mary couldn't see the bigger picture); Poor Communication (nobody knew what others did; helpers unused); Freelancing (Malcolm didn't coordinate, making things "far worse").

Elements of Incident Management Process

- Such skills exist "to channel the energies of enthusiastic individuals"; Google's system is based on the Incident Command System.
- Recursive Separation of Responsibilities: clear roles give more autonomy, since people needn't second-guess colleagues; overloaded people ask the planning lead for staff and delegate, perhaps via subincidents. Roles: Incident Command (holds high-level state, assigns roles, holds all positions not delegated); Operational Work (the operations team should be the only group modifying the system); Communication (public face, periodic updates, keeps the document current); Planning (bugs, handoffs, tracking divergence from the norm).
- A Recognized Command Post: a "War Room" or desks with email and IRC, which is reliable, logs communications, and takes bots.
- Live Incident State Document: the commander's "most important responsibility"; concurrently editable, templated, key information on top, messy but functional, retained for postmortems (Appendix C).
- Clear, Live Handoff: say "You're now the incident commander, okay?" and wait for firm acknowledgment; tell everyone.

A Managed Incident

- Mary asks Sabrina to take command. Sabrina emails a prearranged list, records Mary's assessment in a live document, keeps VPs informed without minutiae, asks communications to draft user messaging, loops in Josephine, and tells helpers to prioritize Mary's tasks and keep her informed.
- At 5 p.m. she arranges replacements, and at 6 p.m. hands off to the sister office, who mitigate and close the incident and begin the postmortem.

When to Declare an Incident

- Declare early. It is an incident if you need a second team, the outage is customer-visible, or it's unsolved after an hour's concentrated analysis.
- Proficiency "atrophies quickly": use the framework for other cross-team changes, in disaster-recovery testing, and in role-play.

In Summary

- A plan made in advance, scalable and regularly used reduced mean time to recovery and stress.

- Best practices: Prioritize (stop the bleeding, restore service, preserve evidence); Prepare; Trust; Introspect; Consider alternatives; Practice; Change it around (rotate roles).

How this chapter builds the authors' case

Chapter 13 told readers to follow the incident response process and showed an incident where it wasn't; this is that process. It cites [Kri12] disaster-recovery testing as practice and leads into Chapter 15, where the incident document feeds the postmortem.

CHAPTER 15

Postmortem Culture: Learning from Failure

Place in the argument — why this chapter, why this title

Written by John Lunney and Sue Lueder. The Part III intro makes a blameless postmortem culture the first step in understanding what went wrong, after Chapters 13 and 14. Sections go from philosophy to workflow, to introducing the culture, to ongoing improvements.

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph: "The cost of failure is education." Without a formalized process of learning, incidents "may recur ad infinitum". So postmortems are "an essential tool for SRE".
- A **postmortem** is "a written record of an incident, its impact, the actions taken to mitigate or resolve it, the root cause(s), and the follow-up actions to prevent the incident from recurring".

Google's Postmortem Philosophy

- Goals: document the incident, understand all contributing root causes, and put effective preventive actions in place.
- "Writing a postmortem is not punishment—it is a learning opportunity for the entire company." Common triggers: user-visible downtime beyond a threshold; data loss; on-call intervention such as rollback; long resolution time; a monitoring failure. Define criteria beforehand; any stakeholder may request one.
- **Blameless** postmortems are "a tenet of SRE culture": they assume everyone "had good intentions and did the right thing with the information they had". Otherwise people hide issues for fear of punishment. It came from healthcare and avionics; you can't "fix" people, but you can fix systems and processes.
- Examples: a finger-pointing rewrite demand versus a blameless action item that future on-callers will thank us for.
- Best Practice: Avoid Blame and Keep It Constructive. Removing blame lets people escalate without fear; blame leads to incidents swept under the rug.

Collaborate and Share Knowledge

- Postmortems are Google Docs with a template (Appendix D). Look for real-time collaboration, open commenting and email notifications.
- Review: senior engineers ask whether key data was collected, impact assessed, the root cause "sufficiently deep", and the action plan appropriate. Then it is shared as widely as it would help.
- Best Practice: No Postmortem Left Unreviewed. "An unreviewed postmortem might as well never have existed." Hold review sessions, then add it to a repository.

Introducing a Postmortem Culture

- It needs "continuous cultivation and reinforcement", with senior management participating. Activities: postmortem of the month, a Google+ group, reading clubs, and Wheel of Misfortune, reenacting a past postmortem.
- Against doubts about cost: a trial period, rewards, and leadership participation.
- Best Practice: Visibly Reward People for Doing the Right Thing. At a 2014 TGIF, an SRE whose release took down a critical service for four minutes, and who rolled back at once, got two peer bonuses and applause.
- Best Practice: Ask for Feedback on Postmortem Effectiveness. Survey teams. Now "any significant incident is followed by a comprehensive postmortem".

Conclusion and Ongoing Improvements

- The authors say continuous investment means Google "weathers fewer outages and fosters a better user experience". A "Postmortems at Google" working group coordinates templates and automates creation and data extraction for trend analysis.
- Aggregation tools find common themes. Future work: machine learning to predict weaknesses and reduce duplicate incidents.

How this chapter builds the authors' case

Chapter 13 urged writing up incidents and said postmortem writing is covered here; Chapter 14's incident document is kept for postmortem analysis. Chapter 16 adds the Outalator.

CHAPTER 16

Tracking Outages

Place in the argument — why this chapter, why this title

Written by Gabe Krabbe. The Part III intro pairs this with Chapter 15 as a tool to keep track of recent incidents, causes and actions. Postmortems cover individual big outages; this tool covers the rest. Sections introduce Escalator, then the Outalator built on it.

Chapter opening — untitled section before the first sub-heading in the book

- Improving reliability "is only possible if you start from a known baseline and can track progress." Outalator passively receives all monitoring alerts and lets users annotate, group and analyze them.
- Postmortems are only written for large-impact incidents, so frequent small issues fall outside them, as do changes with "large horizontal impact".

Escalator

- All SRE alert notifications share a central replicated system tracking whether a human acknowledged; with no acknowledgment after a configured interval it escalates, e.g. primary to secondary on-call.
- It began as a "largely transparent tool" receiving copies of on-call emails.

Outalator

- Following Escalator's example of adding features to existing infrastructure, it handles outages, "the next layer of abstraction". It shows a time-interleaved list of several queues (Figure 16-1), stores notifications and replies, and lets annotations be marked "important". Alerts can be grouped into one incident (Figure 16-2), allowing separate analysis of "incidents per day" and "alerts per day".
- Aggregation: one event triggers many alerts (a network failure alerts every team); multiple alerts are unavoidable given the false positive/negative trade-off. Grouping is "critical"; emailing each duplicate does not scale.
- Tagging: not every alert is an incident. General-purpose, mostly free-form tags; colons act as semantic separators ("cause:network:switch"), with suggested prefixes like "cause" and "action" from team history. Avoiding a predetermined list gives better data; tags are "probably one of the Outalator's most useful unique features".
- Analysis: historical data answers "what did we do last time?" but matters most for systemic problems. Layers: counts and aggregate statistics; comparison across teams and time (is "the third time this week" normal?); then semantic analysis, e.g. which component causes most incidents.
- Reporting and communication: items can be emailed to the next on-call, and a "report mode" serves weekly reviews.
- Unexpected Benefits: coinciding alerts speed diagnosis; seeing that the Bigtable SRE team has not been alerted means you should alert them. Some teams use dummy escalator configurations as a "system of record", e.g. to audit privileged accounts.

How this chapter builds the authors' case

It opens by saying postmortems (Chapter 15) are "only part of the answer", and extends learning from failure to small, frequent issues and cross-team patterns. It is built on the existing on-call alerting system, and the Part III intro places it with Chapter 15.

CHAPTER 17

Testing for Reliability

Place in the argument — why this chapter, why this title

Written by Alex Perry and Max Luebbe. The Part III intro places testing after postmortems: once we know what tends to go wrong, the next step is prevention. The title ties testing to reliability, not just correctness: testing is how SREs quantify confidence. Sections go from kinds of tests, to starting a test and build environment on a project already underway, to testing at scale in the SRE manner, to a conclusion.

Chapter opening — untitled section before the first sub-heading in the book

- Epigraph: “If you haven’t tried it, assume it’s broken.” An SRE responsibility is “to quantify confidence in the systems they maintain”, by adapting classical software testing to systems at scale. Confidence is measured by past reliability (monitoring data) and future reliability (predictions from it).
- Predictions are strong enough only if either the site stays completely unchanged, or you can confidently describe all changes so analysis can allow for the uncertainty of each.
- Testing is “the mechanism you use to demonstrate specific areas of equivalence when changes occur”: each test that passes before and after a change reduces the uncertainty. How much testing you need depends on the reliability requirements. More coverage means you can make more changes before reliability falls below an acceptable level. If you change too much too fast, pause while monitoring data accumulates; sampling statistics then show which new paths need better testing.
- Relationships Between Testing and Mean Time to Repair: passing tests doesn’t prove reliability, but failing tests “generally prove the absence of reliability”. Monitoring finds bugs only as fast as the reporting pipeline reacts. A test with **zero MTTR** is a system-level test applied to a subsystem that detects the same problem monitoring would, blocking the push. More such bugs raise MTBF, which encourages faster releases; new bugs then slow velocity.

Types of Software Testing

- Two categories: traditional (offline, during development) and production (on a live service).
- Traditional tests form a hierarchy (Figure 17-1) layered on unit tests. A **unit test** checks a separable unit such as a class or function, and also serves as specification. An **integration test** assembles passed units; dependency injection (e.g. Dagger) mocks complex dependencies, such as a lightweight mock replacing a stateful database. A **system test** is the largest test of an undeployed system, end to end: smoke tests (simple but critical behavior, short-circuiting costlier tests); performance tests (guarding against incremental slowdown, e.g. 8 GB growing to 32 GB, or 10 ms to 50 ms to 100 ms); regression tests (a “gallery of rogue bugs” documented as tests). Tests cost time: unit tests take milliseconds, full-server tests minutes to hours.
- Production tests interact with a live system, are like black-box monitoring (Chapter 6), and are “essential to running a reliable production service”.
- Rollouts Entangle Tests: production “usually isn’t hermetic” because rollouts are staged and not in source-control order; so a test may pair the latest configuration with an older live binary, and a hermetically valid result may not be valid operationally.
- Configuration test: a test per configuration file examines how production is actually configured and reports discrepancies; it is non-hermetic. The passing version shows how far production lags engineering. Simple when production exposes the file (query and diff); harder with built-in defaults, preprocessors or shared-runtime context.
- Stress test: components often “catastrophically fail” rather than degrade; stress tests find limits (how full can a database get before writes fail; how many queries per second before overload).
- Canary test: a subset of servers is upgraded and left in an incubation period (“baking the binary”); if no unexpected variances occur, rollout continues progressively, else revert. It “isn’t really a test; rather, it’s structured user acceptance”, exposed to less predictable traffic, so it can miss faults.
- Canary arithmetic: with an exponential rollout the cumulative reports are CU = RK (R the report rate, U the order of the fault, K the period over which traffic grows by a factor of e). Reports gathered during rollback estimate C and R, giving U. Orders: U=1 the request hit broken code; U=2 it randomly

damages data a later request sees; U=3 the damaged data is also a valid identifier to a previous request. Most bugs are order one and scale linearly with traffic; they can be tracked by turning logs of unusual responses into regression tests. Higher-order bugs can't, and must be caught during release, because operational workload grows very quickly. With exponential rollout, overlapping methods with the same K keep K small, minimizing user-visible variances. Footnote 4: start at 0.1% of traffic, then 1%, 10%, 100% on successive days.

Creating a Test and Build Environment

- SREs often join once a project is underway with a prototype codebase; testing every function is "completely overwhelming", so start with the most impact for least effort. Ask: can you stack-rank components? Which functions are mission- or business-critical (billing)? Which APIs do other teams integrate against?
- Cheap first step: smoke tests for every release ("Shipping software that is obviously broken is among the most cardinal sins of a developer"). Build a testing culture by documenting every reported bug as a test that fails at first and passes when fixed.
- Infrastructure: versioned source control, then a continuous build system running tests on every submission and notifying engineers at once. When a change breaks the build, drop everything and fix it because: it's harder to fix after more changes; breakage slows the team; nightly and weekly builds lose value; emergency releases (e.g. a security disclosure) become harder. Stability and agility, "traditionally in tension", align here: "When the build is predictably solid and reliable, developers can iterate faster!"
- Bazel builds dependency graphs and rebuilds only what depends on a changed file, so tests run cheaper and faster. Set explicit goals and deadlines rather than "We need more tests"; life- or revenue-critical systems need far higher coverage than a short-lived script.

Testing at Scale

- A systems perspective. A unit test's few dependencies each need their own coverage, or an unrelated change can leave it passing regardless of faults. A release test may depend on everything in the repository; practical environments choose branch points to resolve the most uncertainty in the fewest iterations.
- Testing Scalable Tools: SRE tools (e.g. propagating database metrics, predicting usage, refactoring replica data) keep side effects within the tested mainstream API and sit behind a validation and release barrier. Barrier Defenses Against Risky Software: software that bypasses the tested API (such as turning off transactions to shorten maintenance) can wreck a live service if launched against a user-facing replica. Design defense: a separate tool places a barrier so the replica fails its health check; the risky software checks for the barrier and touches only unhealthy replicas; the black-box monitoring health validator removes the barrier.
- Automation tools (index selection, datacenter load balancing, relay-log shuffling) operate against a tested API for an invisible side effect. Testing can show the other layer behaves the same before and after, but some invariants (a DNS cache holding until the TTL) may not survive. Automation can disturb its own environment (a container-shuffling tool shuffling itself, two tools changing each other's environment), which is fine only if restart semantics are tested and checkpoint health assured independently.
- Testing Disaster: offline recovery tools (checkpoint, loadable, barrier, clean start) are easy to test well; break these constraints and confidence is harder. Online repair tools operate outside the mainstream API and may race with eventually consistent behavior.
- Using Statistical Tests: Lemon (fuzzing), Chaos Monkey and Jepsen aren't necessarily repeatable, so rerunning after a fix doesn't prove it. Log the random actions, refactor the log into a release test and replay: the rate of nonfailure tells how hard the fault will be to prove fixed; later runs may show worse failures.
- The Need for Speed: each test gives pass or fail per version, possibly varying between identical runs; computing likelihoods for every test at every version is infeasible, so form hypotheses and repeat appropriately, estimating scenarios together.
- Testing Deadlines: simple hermetic tests give interactive feedback; orchestrated tests are batch tests, saying "this code is not ready for review". Example: a service with over 21,000 simple tests, comparing pass/fail vectors before and after a patch. Rejecting 1 in 10 good patches would bring complaints; 1 in 100 might pass unnoticed. So the 42,000th root of 0.99 (2 × 21000 tests) means individual tests must run correctly over 99.9999% of the time. "Hmm."

- Pushing to Production: configuration is often separate from developer source, and testing infrastructure ignores it. In the SRE model segregation is worse: it stops the production model being related to the application model, and "prevent[s] the system architecture from changing" because migration risk can't be eliminated. Even with unified testing, some failures indicate larger risk than others.
- Expect Testing Fail: when a product released once a year, MTBF was one year regardless of testing, and previous reliability data was irrelevant. Now new versions can be built every few minutes; testing intermediate versions maps problems back to underlying causes, so the shorter feedback cycle gives a higher-quality annual release. More versions to users lowers MTBF but reveals where tests are needed. Careful reliability management combines limits on uncertainty from test coverage with limits on user-visible faults to set release cadence. The reliability budget can be segmented by team, each with its own release-rate limit. To avoid scaling SREs linearly, production must run mostly unattended, and SRE tools must be tested or manual intervention reduces confidence in historical data.
- Configuration files exist because changing them is faster than rebuilding, which keeps MTTR low. Two kinds: one only edited on failures (release cadence slower than MTBF, so uncertainty about manual edits); one changed more than once per application release (a major risk unless tested and monitored better than the application, else it dominates site reliability). Categorize each file as only one, and enforce that. For the latter: enough coverage for routine edits; edits delayed for release testing; and a break-glass mechanism, made noisy (e.g. filing a bug). Break-Glass and Testing: don't disable tests; associate the early push with pending tests, back-annotate failures, and boost test priority.
- Integration: configuration content is potentially hostile to the interpreter. Interpreted languages such as Python give no upper limit on loading time, so apply deadline checks. Custom syntax needs all tests from scratch; YAML with a tested parser bounds loading; protocol buffers add a schema checked at load time with bounded runtime. SRE writes systems engineering tools with robust validation; use defense in depth. Example: an edit makes the parser stop at half of /etc/passwd; the home-directory tool should notice the mismatch and report it, not remediate by deleting user data.
- Production Probes: testing covers known data, monitoring unknown user data, but actual risk is more complicated. Request banks (known bad; known good replayable; known good not replayable) serve as integration tests, release tests and probes. Probes still add value: release tests used a fake backend, probes a load balancer and persistent backend, and frontends and backends release independently, so the probe configuration was untested. A failing probe means the environments aren't equivalent, so the site is likely broken. The production updater replaces probes with the application, so all four old/new combinations run; probes in the readiness check make a bad update fail safely with no user traffic.
- Fake Backend Versions: the peer team maintains the fake backend as a build dependency; ideally it cuts a release alongside the backend. Monitoring should know all release versions on both sides of an interface and run new combinations when either team releases; this need not block that release, but rollout automation should block production rollout until problematic combinations are impossible.

Conclusion

- "Testing is one of the most profitable investments engineers can make to improve the reliability of their product." It is continuous, costly in effort and infrastructure. "You can't fix a problem until you understand it, and in engineering, you can only understand a problem by measuring it."

How this chapter builds the authors' case

Footnote 1 says the scheduler and executors are shared infrastructure and ordinary SRE-supported services not discussed here. The chapter links to black-box monitoring (Chapter 6), builds on the Part III hierarchy (monitoring, response, postmortems, then testing as prevention), and recurs to canarying, and the Part III intro points on to capacity planning (Chapter 18).

CHAPTER 18

Software Engineering in SRE

Place in the argument — why this chapter, why this title

Written by Dave Helstroom and Trisha Weir, with Evan Leonard and Kurt Delimon. The Part III intro presents it as a case study of software engineering in SRE with Auxon, a tool for automating capacity planning, and its Development theme says significant large-scale system design and software engineering are key to Google's approach to SRE. The title says what the chapter claims: software engineering belongs inside SRE. Order: why it matters, the problem Auxon addressed, the solution and lessons, how to foster such work, conclusions.

Chapter opening — untitled section before the first sub-heading in the book

- Behind consumer products such as Gmail or Maps, "a massive amount of behind-the-scenes software engineering" goes unseen, and a number of those products are developed within SRE.
- Google production is "one of the most complex machines humanity has ever built", so SREs are uniquely suited to build tools for uptime and latency (binary rollout, monitoring, dynamic server composition). These are full-fledged software engineering projects with a product-based mindset: internal customers and a roadmap.

Why Is Software Engineering Within SRE Important?

- Scale needs internal software, as few third-party tools fit. SREs are well placed because they have Google-specific production knowledge (scalability, graceful degradation, interfacing), are embedded in the subject matter, and have direct, frank, high-signal feedback from fellow SREs, who tolerate minimal UI and alpha issues.
- Pragmatically: SRE-supported services grow faster than the organization; "team size should not scale directly with service growth". Linear team growth against exponential service growth needs perpetual automation.
- SREs benefit too: career development, a coding outlet, project work balancing interrupts and on-call, and attracting and keeping diverse skills ("a variety of backgrounds… can help prevent blind spots"), so teams mix software and systems engineers.

Auxon Case Study: Project Background and Problem Space

- Auxon automates capacity planning for services in production.
- Traditional Capacity Planning, a cycle: 1) collect demand forecasts (several quarters to years); 2) devise build and allocation plans; 3) review and sign off; 4) deploy and configure resources. It's "neverending": assumptions change, deployments slip, budgets are cut, each revision trickling into all later quarters. Demand is the key driver and supply is manually shaped to fit.
- Brittle by nature: any change (lower efficiency, faster adoption, a late cluster, a performance-goal change altering the service's footprint) forces cross-checking, or recreating, the whole plan; each quarter rests on earlier ones.
- Laborious and imprecise: forecast data collection is slow and error-prone; resources aren't equally suitable (North American capacity doesn't help a shortfall in Asia); constraints relate to intent. Bin packing by hand is tedious and NP-hard; requests arrive as inflexible "X cores in cluster Y", with the reasons and degrees of freedom lost. Spreadsheets scale poorly, so teams oversimplify. Result: approximate bin packing, brittle, with "no known bounds on an optimal solution".
- Our Solution: Intent-Based Capacity Planning: "Specify the requirements, not the implementation." Programmatically encode a service's dependencies and parameters (**intent**) and autogenerate an allocation plan; when demand, supply or requirements change, regenerate. Plans become nimble, toil drops, owners focus on SLOs, production dependencies and infrastructure requirements, and optimization gives precision and cost savings; today's algorithms can solve to a known optimal solution even though some bin packing is NP-hard.

Intent-Based Capacity Planning

- Intent "is the rationale for how a service owner wants to run their service", reached through layers of abstraction: 1) "50 cores in clusters X, Y, and Z for service Foo"; 2) a 50-core footprint in any 3 clusters in region YYY; 3) meet demand in each region with N + 2 redundancy; 4) run Foo at 5 nines. Higher

steps give more flexibility. Support all levels; services tend to gain most by step 3, and sophisticated ones may aim for step 4.

- Precursors to Intent: Dependencies (Foo needs Bar within 30 ms of network latency; Bar depends on Baz and Qux; nested, shared); Performance metrics, "the glue between dependencies", converting higher-level to lower-level resources, derived by load testing and monitoring; Prioritization, forcing trade-offs (N + 2 for Foo versus N + 1 for Bar) to be "transparently, openly, and consistently" made rather than ad hoc.
- Introduction to Auxon: built by a small group of software engineers and a technical program manager in SRE over two years; plans many millions of dollars of machine resources. Users state requirements ("My service must be N + 2 per continent") via a configuration language or API; they are prioritized and represented as a giant mixed-integer or linear program whose solution forms the allocation plan.
- Figure 18-1 components: Performance Data (scaling per unit of demand); Per-Service Demand Forecast Data (some services, like Colossus, derive demand from dependents); Resource Supply (the upper bound); Resource Pricing (the objective to minimize); Intent Config (human-readable wiring); Configuration Language Engine (produces a protocol buffer, light sanity checking); Solver (scales across hundreds or thousands of machines); Allocation Plan (resources per service per place, plus unsatisfied requirements).
- Requirements and Implementation: Auxon began with an SRE and a TPM who had planned capacity in spreadsheets. The team stayed in on-call rotations and was both consumer and developer, which gave ownership, credibility and legitimacy.
- Approximation: "Don't focus on perfection and purity of solution… Launch and iterate." Linear programming was uncharted, so they built a heuristic "Stupid Solver" behind an abstracted interface, later swapped out. Fuzzy requirements encourage general, modular design: the Allocation Plan was shaped to be universally useful so automation systems integrate on their own, an "agnostic" approach that eased onboarding; machine-performance data sat behind a swappable interface. Design flexible enough to avoid rework, yet with a real-world implementation.
- Raising Awareness and Driving Adoption: a single presentation or email isn't enough; you need a coherent approach, user advocacy and senior sponsorship. Provide documentation, since "if your solution is too difficult or confusing, they will write their own solution".
- Set expectations: separate aspirational goals from the Minimum Viable Product; don't promise too much too soon, yet promise enough. Auxon offered immediate relief from manual bin packing, with the same configuration files carrying over to later savings, and a roadmap.
- Identify appropriate customers: larger teams had home-grown tools and little pain, so early targets were teams with no process, who became advocates; a Business Area case study compared before and after.
- Customer service: offer white glove support, address fear of being replaced by a shell script (teams own configurations and results, not toil); early adopters are local experts across the globe.
- Designing at the right level: **agnosticism**, "come as you are; we'll work with what you've got"; avoid over-customizing for big users, and avoid defining success as 100% adoption, given diminishing returns on the long tail.
- Team Dynamics: a seed team of generalists and engineers of broad experience covers blind spots. They took the design to Google's Operations Research and Quantitative Analysis teams, later adding statisticians and optimization experts once the project was demonstrably successful.

Fostering Software Engineering in SRE

- Good candidates: engineers with firsthand domain experience, a highly technical user base giving high-signal bug reports, noticeable benefits (less toil), and a fit with organizational objectives, with cross-organizational review to avoid overlap.
- Poor candidates: touching many moving parts, all-or-nothing designs, overly specific work (teams are organized around services, so standardization comes second), or overly generic frameworks that fit no use case. A broad example: the layer-3 load balancer later offered as Google Cloud Load Balancer.
- Staffing and Development Time: SREs are generalists with strong coding, but may lack product-team experience ("I have a design doc; why do we need requirements?"), so partner with engineers, TPMs or PMs. Uninterrupted project time "must be aggressively defended". Most products start as side projects, then stay grassroots, become formal projects, or gain executive sponsorship; either way, SREs must keep working as SREs, being both creator and customer.

- Getting There: it is as much organizational change as technical, against an SRE's instinct to write code for immediate needs, which doesn't scale and duplicates effort. Decide whether you want better practice on one team or cross-team standards (years, higher payback). Guidelines: create and communicate a clear message (SREs are skeptical; benefits include faster ramp-up and portable knowledge; success is when they ask how, not if); evaluate capabilities (product manager, agile skills; borrow from product development, hire later); launch and iterate (straightforward, uncontroversial first products, a six-month release rhythm, some teams moving to push-on-green); and don't lower your standards (would you onboard it if a dev team made it? code review, integration testing, production-readiness review). "It takes a long time to build credibility… but only a short time to lose credibility."

Conclusions

- Hands-on production experience yields innovative approaches, as with Auxon. Such projects mean the SRE team doesn't have to scale linearly with the size of the services; benefits go to the company, SRE and the SREs themselves.

How this chapter builds the authors' case

It refers back to Chapter 2 for "service" and "cluster" and to [Hix15a] for capacity planning, and echoes the principle that team size should not scale with service growth. The Part III intro places it under capacity planning, which precedes load balancing in Chapters 19 to 21, and under Development as the first example of SREs doing software engineering, followed by Chapters 23 to 26.

CHAPTER 19

Load Balancing at the Frontend

Place in the argument — why this chapter, why this title

Written by Piotr Lewandowski. “Frontend” means the first of two levels, balancing user traffic between datacenters (the next chapter goes inside one). Sections follow a request in time: the DNS lookup before any connection, then the virtual IP once it starts. Each layer is useful but insufficient, which brings on the next.

Chapter opening — untitled section before the first sub-heading in the book

- Google serves “many millions of requests every second” on many computers; even a supercomputer would be a single point of failure, “a recipe for disaster”.

Power Isn’t the Answer

- Even with a powerful machine and a network that never fails, the speed of light in fiber bounds how fast data can be served over distance.
- “Optimal” has no single answer; it depends on the level (global versus local), the technical level (hardware versus software) and the nature of the traffic.
- Search cares about latency, so it goes to the nearest datacenter by round-trip time. Video upload cares about throughput and succeeding first time, so it may take an underutilized link at the expense of latency.
- Locally, machines are assumed equally distant, so the aim is utilization and protecting a server from overloading.
- Real cases add cache warmth and congestion avoidance; load balancing is “anything but straightforward and static”, so it is done at multiple levels.

Load Balancing Using DNS

- Simplest: return multiple A or AAAA records and let the client pick. Little control, since each attracts roughly equal traffic; SRV records could weight them but “have not yet been adopted for HTTP”.
- Closeness: anycast nameservers, or a network-location map, at the cost of a complex DNS server and a pipeline keeping the map current.
- The recursive resolver in the middle has three implications: recursive resolution, nondeterministic reply paths, caching complications.
- The nameserver sees the resolver’s IP, not the user’s; the EDNS0 extension carries the client subnet, and the biggest resolvers already support it.
- One nameserver may serve a whole continent; an ISP’s may sit in one datacenter and return what suits it “despite there being better network paths for all users!”
- A cached reply may reach one user or thousands, so Google tracks the approximate user base behind each resolver and estimates its geographic spread, optimizing for the majority.
- “Best” also means enough capacity and a healthy datacenter; the DNS server is integrated with global control systems that track these.
- Resolvers’ caches cannot be flushed, so TTLs are low, a lower bound on propagation.
- DNS is “the simplest and most effective way” to balance before the connection starts, but not sufficient: the 512-byte reply limit (RFC 1035) caps addresses below the number of servers. Next come virtual IPs.

Load Balancing at the Virtual IP Address

- A **VIP** is shared across many devices but looks like one IP, hiding machine counts and easing maintenance. The network load balancer forwards packets to a backend.
- Least loaded backend breaks down for stateful protocols (every connection must be tracked). Hashing, id(packet) mod N, is stateless, but removing a backend makes N-1 and “almost every packet suddenly maps to a different backend”, resetting connections.
- **Consistent hashing** (1997) stays “relatively stable” as backends change. Usually simple connection tracking, with consistent hashing as fallback under pressure such as a denial of service attack.

- Forwarding: NAT needs per-connection state. Rewriting the layer 2 MAC address allows **Direct Server Response**, where small requests and large replies mean little traffic crosses the balancer, but everything must share a broadcast domain; Google outgrew it.
- Current: GRE packet encapsulation; balancer and backend "can even be on separate continents". Price: 24 bytes overhead for IPv4+GRE, risking MTU overrun and fragmentation, avoided with a larger MTU in the datacenter.
- "Load balance early and load balance often", but the difficulty is in the details.

How this chapter builds the authors' case

The Part III intro places it right after capacity planning (Chapter 18): load balancing ensures the capacity built is properly used, and Chapters 19 to 21 are called essential for service reliability. The chapter itself ends by pointing to what happens once packets reach the datacenter, the subject of Chapter 20.

CHAPTER 20

Load Balancing in the Datacenter

Place in the argument — why this chapter, why this title

Written by Alejandro Forero Cuervo. After the frontend chapter, this one goes inside the datacenter and covers application-level policies for routing a stream of queries to backend tasks; networking and datacenter selection are out of scope. The sections go in the order a client must decide: what the ideal is, which backends are healthy, which few to connect to (subsetting), and finally which one gets the request (policies, from naive to good).

Chapter opening — untitled section before the first sub-heading in the book

- Setting: a stream of queries not exceeding the datacenter's resources, hitting services built from many homogeneous, interchangeable server processes: at least three (fewer loses "50% or more" of capacity with one machine), up to over 10,000, typically 100 to 1,000. These are **backend tasks**; **client tasks** hold connections to them and must pick a backend per query.
- It is a generalized scenario fitting no one service. The techniques apply across the stack, e.g. the GFE (Google Frontend) uses them along with Chapter 19's, and a single request can trigger a long transitive chain of dependent requests with high fan-out.

The Ideal Case

- Ideally load is spread perfectly so the least and most loaded tasks use the same CPU. Traffic can go to a datacenter only until the most loaded task reaches its limit (Figure 20-1).
- With a large spread, idle capacity of every task but the busiest is wasted: the sum over tasks of (CPU[0] – CPU[i]), "wasted" meaning "reserved, but unused" (Figure 20-2).
- Poor balancing "artificially limit[s] resource availability": 1,000 CPUs reserved, perhaps no more than 700 usable.

Identifying Bad Tasks: Flow Control and Lame Ducks

- First identify and avoid unhealthy tasks.
- Flow control: clients count active requests per backend and treat it as unhealthy at a limit (100 is reasonable). It doubles as simplistic balancing. But it only guards against extreme overload, can trip while backends have spare resources (long-lived requests), and has caused all backends to become unreachable; raising the limit does not tell "unhealthy" from "slow to respond".
- Robust approach, the **lame duck state**. Backend states from the client: healthy; refusing connections (starting, shutting down or abnormal); lame duck, listening and able to serve but "explicitly asking clients to stop sending requests."
- A lame duck task broadcasts to active clients; inactive clients send periodic UDP health checks, so it propagates in typically 1 or 2 RTT.
- Benefit: clean shutdown without errors for unlucky active requests. Steps: scheduler sends SIGTERM; task enters lame duck state via an RPC API call in the SIGTERM handler; ongoing requests run normally; active requests drop to zero; after a configured interval the task exits or is killed. Rule of thumb: 10s to 150s.
- The state also lets clients connect to tasks still doing long initialization, which signal explicitly when ready.

Limiting the Connections Pool with Subsetting

- **Subsetting** limits the backends a client interacts with. Clients keep long-lived connections (per-request connections cost resources and latency; idle ones go to a cheap "inactive" mode).
- Each connection costs memory and CPU at both ends, which adds up; subsetting avoids a client connecting to too many backends or a backend receiving too many clients.
- Picking the right subset: size and selection algorithm. Typical size 20 to 100. Larger when clients are far fewer than backends (else some backends never get traffic) or when client load is imbalanced, e.g. bursts concentrated in the assigned subset.
- Algorithm needs: uniform assignment (one backend overloaded by 10% means all overprovisioned by 10%); minimal churn on restarts and failures (replacement means new TCP connections and negotiation); handling resizes without knowing sizes in advance, notably during rolling pushes.

- Random subsetting: shuffle once, take healthy backends from the start. It handles failures with little churn but "works very poorly" since load is uneven. Simulation with 300 clients, 300 backends, subset size 30% (90 each): least loaded backend at 63% of average (57 connections vs 90), most loaded 121% (109) (Figure 20-3). At 10% (30 each): 50% (15) to 150% (45) (Figure 20-4). Even load would need subsets of 75%, "simply impractical".
- Deterministic subsetting, with code in the book. Clients are divided into "rounds" of subset_count clients (backends divided by subset size). Each round shuffles the backend list with its own seed and each client takes the subset at client_id % subset_count, so within a round each backend goes to exactly one client. Example: 12 backends, size 3, 4 subsets per round, 10 clients.
- The shuffle prevents subsets of consecutive backends that a gradual update takes down together. Different seeds per round matter: otherwise a failed backend's load falls only on its subset's remaining members and compounds as more fail ((subset_size - N) left).
- Result: clients per backend differ by at most 1; for 300 clients each using 10 of 300 backends, every backend gets exactly the same connections (Figure 20-5).

Load Balancing Policies

- Policies choose which backend in a client's subset gets a request. The complexity comes from deciding in real time with "only partial and/or stale backend state information". They range from stateless (Round Robin) to informed (Least-Loaded, Weighted).
- Simple Round Robin: each client cycles through connectable, non-lame-duck backends. Long the most common and much better than random, but can give up to 2x CPU spread. Causes:
- Small subsetting: clients issue requests at different rates, so backends in busy clients' subsets are more loaded.
- Varying query costs: the costliest requests can use 1000x or more CPU than the cheapest, unpredictably (e.g. "return all emails received by user XYZ in the last day"). Capping work per request, e.g. pagination, is hard: all client code changes, and consistency problems arise (repeated or skipped messages). Interfaces thus allow 100x to 10,000x spreads. One Java backend: about 15 ms CPU on average but up to 10 seconds for some; a large query raises load and hurts other requests' latency.
- Machine diversity: different CPUs. The scheduler adjusts reservations by equivalence factors (2 CPU units on slow machine X equal 0.8 on fast Y), which took significant effort; the virtual unit **GCU** (Google Compute Units) maps each architecture to its performance.
- Unpredictable performance: antagonistic neighbors (up to 20% differences, from shared caches or bandwidth) and task restarts (more resources for minutes, e.g. dynamically optimizing Java). Some servers stay in lame duck state and prewarm after start. Without adapting, "inherently" suboptimal at scale.
- Least-Loaded Round Robin: each client counts active requests per backend and round-robins among those with the minimum (worked example with t0 to t9). Loaded tasks tend to have higher latency, so load moves away from them.
- Pitfall learned "the hard way": an unhealthy task returns fast errors and **sinkholes** traffic. Fix: count recent errors as active requests.
- Two limits: active requests are a poor proxy for capability when requests mostly wait on the network (a twice-as-fast backend looks equally loaded); and each client sees only its own requests. Large services saw the busiest task at twice the CPU of the least, "about as poorly as Round Robin".
- Weighted Round Robin uses backend-provided information: clients keep a "capability" score per backend; each response (health checks too) carries queries and errors per second and utilization, typically CPU. Scores adjust periodically; failures are penalized. It "has worked very well", sharply narrowing the spread (Figure 20-6).

How this chapter builds the authors' case

It explicitly continues Chapter 19, which ends by saying packets reaching the datacenter bring their own difficulties; the GFE uses these algorithms together with Chapter 19's. The chapter's measure throughout is wasted reserved capacity, and the Part III intro presents this chapter and the next as essential for reliability.

CHAPTER 21

Handling Overload

Place in the argument — why this chapter, why this title

Written by Alejandro Forero Cuervo. Load balancing aims to avoid overload, but "eventually some part of your system will become overloaded", so this chapter covers what happens then. Order: measure load properly (not queries per second), decide who is rejected (customer limits, client throttling, criticality), the task's own signals, how clients react to errors, and load from connections.

Chapter opening — untitled section before the first sub-heading in the book

- Gracefully handling overload "is fundamental to running a reliable serving system." One option is degraded responses (search a small percentage of candidates; use a stale local copy); under extreme overload even that fails and errors are served.
- Balancing across datacenters so none exceeds capacity (100 tasks at 500 requests per second: at most 50,000 queries per second) can still be insufficient. Hence: "redirect when possible, serve degraded results when necessary, and handle resource errors transparently when all else fails."

The Pitfalls of "Queries per Second"

- Query cost varies by issuing client code or time of day. Modeling capacity as queries per second, or by static request features, was learned "the hard way" to be a poor metric: ratios change, drastically at times. "A moving target makes a poor metric".
- Better: measure capacity in resources (e.g. 500 CPU cores and 1 TB of memory). The **cost of a request** is a normalized measure of CPU time.
- CPU as the provisioning signal usually works: memory pressure shows up as CPU in garbage-collected platforms, and elsewhere other resources can be provisioned to outlast CPU; if too costly, each resource is counted separately.

Per-Customer Limits

- Global overload "occurs quite frequently", especially for internal services. Then only misbehaving customers should get errors, so owners set quotas from negotiated usage. For 10,000 CPUs: Gmail 4,000 CPU seconds per second, Calendar 4,000, Android 3,000, Google+ 2,000, every other user 500.
- The sum can exceed 10,000, relying on customers not all hitting limits together. Global usage is aggregated in real time and limits pushed to tasks; computing per-request CPU is tricky for servers without thread-per-request.

Client-Side Throttling

- Rejecting should be cheaper than serving, but not always (a simple RAM lookup costs almost as much), and a backend can be overloaded "even though the vast majority of its CPU is spent just rejecting requests!" So a client seeing many "out of quota" rejections caps its own traffic; excess fails locally.
- **Adaptive throttling**: each client keeps two minutes of history of requests (attempted by the application layer) and accepts (accepted by the backend). Once requests is K times accepts, new requests are rejected locally with probability max(0, (requests − K × accepts) / (requests + 1)) (Equation 21-1). Locally rejected requests still count in requests, which is preferred: the more is attempted relative to accepted, the higher the drop probability.
- It gives stable rates; in large overload backends reject one request per request processed. The decision is local, with no extra dependencies or latency.
- Where rejecting costs nearly as much as processing, lower K (e.g. 2 to 1.1) for more aggressive throttling. They prefer 2x: more waste, but backend state reaches clients faster.

Criticality

- Four values: CRITICAL_PLUS (serious user-visible impact if it fails); CRITICAL (default for production jobs; capacity provisioned for all of it); SHEDDABLE_PLUS (partial unavailability expected; default for batch jobs); SHEDDABLE (frequent partial, occasional full unavailability expected). Four "were sufficiently robust"; more would cost resources.

- First-class in RPC: a task out of quota for a customer rejects a criticality only if already rejecting all lower ones; an overloaded task rejects lower criticalities sooner; adaptive throttling keeps stats per criticality.
- It propagates automatically (B and C issued while executing A inherit A's), replacing incompatible ad hoc notions. It is set close to browsers and mobile clients, typically in HTTP frontends, overridden only where sensible.

Utilization Signals

- Task-level protection rests on utilization: often CPU rate over CPUs reserved, sometimes memory. Near thresholds, requests are rejected by criticality (higher thresholds for higher criticality). Signals use local state, since the aim is to protect the task.
- Most useful: **executor load average**, active threads (running or ready and waiting) smoothed by exponential decay; rejection starts when they exceed available processors. Brief fan-out spikes are swallowed.

Handling Overload Errors

- Two cases: a large subset of the datacenter's backends overloaded (would not occur if cross-datacenter balancing were perfect): do not retry, let errors bubble up. A small subset (typical, from imperfect internal balancing): retry immediately, since capacity likely exists elsewhere and the extra latency is a few round trips.
- Retries look like new requests to load balancing; the odds of a different task suffice. Backends reject both equally and quickly, making retries "a form of organic load balancing".
- Deciding to Retry: a per-request budget of three attempts; a per-client budget, retrying only while retries are below 10% of requests. Worst case retries alone approach 3X; the 10% budget cuts growth to 1.1x. Third, an attempt counter in metadata; backends keep histograms and, if many retries show other tasks overloaded, return "overloaded; don't retry" (Figure 21-1).
- In deep stacks retry only at the layer immediately above the rejecting one, avoiding "a combinatorial retry explosion". In Figure 21-2, Backend B retries against the overloaded DB Frontend, then returns "overloaded; don't retry" or a degraded response to Backend A, which does likewise.

Load from Connections

- Request-only models overlook the cost of large connection pools and churn, negligible when small but problematic at scale.
- Many clients with few requests: UDP health checks can cost more than serving; tune or create connections dynamically.
- Bursts of new connections from big batch jobs can overload backends. Either expose load to cross-datacenter balancing (base it on utilization), or force batch clients through batch proxies: "batch client → batch proxy → backend". The proxy acts "like a fuse" and cuts connections, allowing bigger subsets.

Conclusions

- Chapters 20 and 21's techniques depend on state propagating through a distributed system and sometimes work imperfectly, so it is "critical to ensure that individual tasks are protected against overload": a task provisioned for a rate should keep serving it without significant latency impact however much excess arrives, and not crash, up to somewhere above 2x or even 10x.
- Ignored degradation leads to memory exhaustion and thrashing, and failure may cascade to other components; see Chapter 22.
- Common mistake: an overloaded backend should not stop accepting all traffic; it should accept "only the requests that it can process and reject the rest gracefully".
- "There is no magic bullet".

How this chapter builds the authors' case

It opens from the load balancing of Chapters 19 and 20 ("Avoiding overload is a goal of load balancing policies"), reuses their ideas (subsets, inactive-connection health checks, cross-datacenter balancing), and its conclusion pairs Chapter 20 with this one before pointing to Chapter 22: unchecked overload of one task can cascade.

CHAPTER 22

Addressing Cascading Failures

Place in the argument — why this chapter, why this title

Written by Mike Ulrich. The title promises both halves: designing against cascading failures and acting when one starts. Chapter 21 ended on single-task protection against overload and pointed here. The order is causes and design, then triggers, then testing, then emergency steps, then a worked Shakespeare example. Two epigraphs, "If at first you don't succeed, back off exponentially" and a reminder about jitter, foreshadow the retry material.

Chapter opening — untitled section before the first sub-heading in the book

- A **cascading failure** is "a failure that grows over time as a result of positive feedback." A portion failing raises the chance others fail; e.g. a replica fails from overload, load rises on the rest, "a domino effect that takes down all the replicas for a service."
- Running example is the Shakespeare search service (Figure 22-1).

Causes of Cascading Failures and Designing to Avoid Them

- A well-thought-out design should consider a few typical scenarios behind most cascades.
- Server Overload, the most common cause; most others are extensions of it. Frontends in cluster A handle 1,000 QPS; if cluster B fails, A gets 1,200, runs out of resources, crashes or misses deadlines, and successfully handled requests dip well below 1,000. The load balancing controller then sends requests to other clusters, spreading overload service-wide, perhaps within a couple of minutes.
- Resource Exhaustion yields higher latency, errors or lower-quality results; "something eventually needs to give". It may also crash servers and shift problems to others.
- CPU: requests slow; more in-flight requests (using memory, threads, file descriptors, backend resources); queues saturate; thread starvation fails health checks; watchdogs kill servers; missed RPC deadlines waste work and invite retries; less CPU cache benefit.
- Memory: dying tasks; in Java, more GC and CPU use, the "GC death spiral" (less CPU, slower requests, more RAM, more GC); lower application cache hit rates mean more backend RPCs.
- Threads (RAM overhead, even running out of process IDs) and file descriptors (cannot open connections, health checks fail).
- Dependencies among resources: overload brings secondary symptoms that "can look like the root cause". Example: poorly tuned GC in a Java frontend, CPU exhausted, more in-flight requests, less RAM for cache, more misses, backend runs out of CPU or threads, health checks fail. Such a chain is unlikely to be fully diagnosed during an outage, especially with different owners.
- Service Unavailability: once a couple of servers crash, load on the rest snowballs into crash-looping, and returning servers are bombarded and fail. If healthy at 10,000 QPS but cascading at 11,000, dropping to 9,000 will not stop it; if 10% of servers can handle requests, load must fall to about 1,000 QPS. Servers failing health checks or in lame duck state have the same effect, and policies avoiding servers that served errors can worsen it.

Preventing Server Overload

- Strategies in rough priority order. First: load test capacity limits and the overload failure mode, "the most important important exercise", since which resource will be exhausted is hard to predict.
- Serve degraded results; instrument servers to reject when overloaded, failing "early and cheaply"; instrument higher-level systems (reverse proxies, load balancers, tasks) to reject. Rate limiting ignores service health, may not stop a started failure, and can leave capacity unused.
- Capacity planning, coupled with performance testing. If each cluster breaks at 5,000 QPS and peak is 19,000, about six clusters give N + 2. It reduces the probability but is not sufficient, since lost infrastructure, partitions or traffic surprises create pockets of high load.
- Queue Management. Thread-per-request servers queue before a thread pool. With constant rate and latency there is no reason to queue. Example: queue 10x threads and 100 ms handling means 1.1 seconds when full, mostly queued. For steady traffic keep queues small (50% or less of the pool), rejecting early; Gmail often uses queueless servers with failover. Bursty load may use queues sized to bursts.

- Load Shedding and Graceful Degradation. **Load shedding** drops a proportion of traffic near overload to keep doing useful work, e.g. HTTP 503 above a number of in-flight requests; LIFO or CoDel drops requests unlikely to be worth processing (a user who waited 10 seconds has refreshed), combined with deadline propagation. **Graceful degradation** reduces the work, e.g. searching only an in-memory subset or a less accurate ranking.
- Considerations: which metrics trigger it, what actions, which layer. Cautions: it should rarely trigger and stay simple; "the code path you never use is the code path that (often) doesn't work", so regularly run some servers near overload; alert when too many enter these modes; complex versions can misfire, so design a way to turn them off quickly (config in Chubby speeds this but risks synchronized failure).
- Retries. A naive frontend caps backend RPCs per request at 10 (the code uses 20 attempts, per the book). Backend limit 10,000 QPS per task; frontend sends 10,100; the 100 rejected are retried, backend gets 10,200, retries grow 100, 200, 300 QPS and less useful work gets done; the backend may crash and shift load to the others. Even if calls fall to 9,000 QPS the problem may persist, if failures are costly or backends unstable; load must be cut sharply until retries stop. It also happens with JavaScript XmlHttpRequest retries or offline sync protocols.
- Retry guidance: apply backend protections; always use randomized exponential backoff, since unrandomized retries "ripple" and amplify; limit retries per request; consider a server-wide retry budget (e.g. 60 retries per minute in a process); avoid retries at multiple levels (3 retries, 4 attempts, across backend, frontend and JavaScript layers gives 64, or 4^3, attempts on the database); separate retriable from nonretriable errors and return a specific overload status. In an emergency retry graphs may be mistaken for a symptom; mitigation means fixing retry code, cutting load or cutting requests off.
- Latency and Deadlines. Deadlines limit how long the backend may consume the frontend's resources. Picking one is "something of an art": none or very high leaves resources consumed; short ones fail expensive requests.
- Missing deadlines: work after the client has given up earns no credit ("you don't get credit for late assignments with RPCs"); check remaining deadline at each stage.
- Deadline propagation: a deadline is set high in the stack and the RPC tree shares the absolute deadline (A picks 30 seconds, spends 7, B gets 23, then 19 to C). Without it, hardcoded 20 seconds lets C work uselessly. Reduce outgoing deadlines a bit, consider upper bounds, mind the traffic mix. Exception: checkpointed catchup. Propagate cancellations to avoid RPC leakage.
- Bimodal latency: 10 frontends with 100 threads each, 1,000 QPS at 100 ms, use 100 threads. If 5% never complete (e.g. unavailable Bigtable row ranges) and the deadline is 100 seconds, they need 5,000 threads, so only 19.6% of requests are handled, an 80.4% error rate. Remedies: look at latency distributions not means; fail fast; avoid deadlines orders of magnitude above mean latency; limit in-flight requests per keyspace or client (e.g. 25% of threads).

Slow Startup and Cold Caching

- Processes are slower just after starting: initialization, and Java's Just-In-Time compilation and class loading. Caches matter too: with a cold cache 100% of requests are costly. Cold caches arise from a new cluster, a cluster returning from maintenance (stale), and restarts (a separate memcache binary may help).
- Strategies: overprovision, noting a **latency cache** (sustains load when empty) versus a **capacity cache** (cannot); caches added for performance can become hard dependencies. Use general prevention techniques and test large restarts. Add load to a cluster slowly, and keep caches warm with nominal load in all clusters.
- Always Go Downward in the Stack: storage problems affect backends and frontends and fixing storage repairs both. Backend-to-backend traffic risks distributed deadlock (a full thread pool in A holds B's threads) and sudden switches to heavy intra-layer proxying under load (primary proxying to secondary). Better: avoid cycles; the backend tells the frontend to retry on the correct backend instead of proxying.

Triggering Conditions for Cascading Failures

- Process Death: Query of Death or other crashes; a small event can break a service near the brink.
- Process Updates: pushing a binary or config to many tasks at once; account for overhead, push off-peak, adjust in-flight updates dynamically.

- New Rollouts: changes alter request profiles or resources; check recent changes and consider reverting; keep change logging.
- Organic Growth: usage grows without capacity adjustment.
- Planned Changes, Drains, or Turndowns: capacity lost to maintenance, or dependencies drained.
- Request profile changes: shifted traffic, changed payload cost, data growth (images per user).
- Resource limits: slack CPU from overcommitment is a dangerous safety net; another job's MapReduce can remove it. Stay within committed limits when load testing.

Testing for Cascading Failures

- Failure modes are "very hard to predict from first principles".
- Test Until Failure and Beyond: load test components until they break; at the limit a component should serve errors or degraded results without significantly reducing successful handling, rather than crash. It reveals the breaking point for capacity planning. Test gradual and impulse loads, return to nominal load (can it exit degraded mode unaided; how far must load drop after crashes), state and correctness for stateful services, each component separately.
- Real traffic tests in a small slice of production, with spare capacity: reduce task counts, lose a cluster's capacity, blackhole backends.
- Test Popular Clients: can they queue work while down, use randomized exponential backoff, be triggered externally to create load; ask internal clients about their failure handling.
- Test Noncritical Backends: their unavailability, or blackholing, should not harm critical components (e.g. spelling suggestions); long deadlines are a hazard.

Immediate Steps to Address Cascading Failures

- Use the incident management protocol (Chapter 14).
- Increase Resources if idle ones exist, though not enough in a death spiral.
- Stop Health Check Failures/Deaths: health checking can itself make a service unhealthy; temporarily disable. Process health checking ("is this binary responding at all?") serves the scheduler; service health checking serves the load balancer.
- Restart Servers: GC death spirals, requests without deadlines, deadlocks. Identify the source first, canary, go slowly; a cold cache could be amplified.
- Drop Traffic: "a big hammer". Fix the trigger, cut load enough to stop crashing (e.g. 1% through), let servers become healthy, ramp up gradually. Drop less important traffic first; root cause must be fixed or load returns the failure.
- Enter Degraded Modes, Eliminate Batch Load (index updates, data copies, statistics), Eliminate Bad Traffic (queries of death).
- Cascading Failure and Shakespeare: a documentary airs in Japan and Asian traffic surges beyond capacity during a major update. Safeguards from the Production Readiness Review (degradation dropping pictures and maps, retries with randomized exponential backoff or none) did not stop tasks failing one by one and Borg restarts. SRE added tasks. The postmortem led to GSLB redirecting overload to neighboring datacenters and autoscaling.

Closing Remarks

- When overloaded, "something needs to give"; past the breaking point it is better to let some errors or lower-quality results through than to try to fully serve every request. Knowing breaking points and behavior beyond them "is critical".
- Changes meant to improve the steady state (retrying, shifting load from unhealthy servers, killing unhealthy servers, adding caches) can raise the chance of large-scale failure; be careful "that one outage is not being traded for another."

How this chapter builds the authors' case

It builds on the preceding load balancing chapters, citing the lame duck state from Chapter 20 and the incident management protocol of Chapter 14, and Chapter 21's conclusion points here for the case where one task's failure triggers others. The Part III intro presents it as advice for cascading failures both in design and when caught in one. Its own thread: capacity planning, testing, degradation and retry discipline each reduce, but none alone prevents, the cascade.

CHAPTER 23

Managing Critical State: Distributed Consensus for Reliability

Place in the argument — why this chapter, why this title

Written by Laura Nolan. The title says the subject (critical state), the tool (distributed consensus) and the goal (reliability). The Part III intro places it under Development: consensus in the guise of Paxos is at the core of many of Google's distributed systems, including Cron (Chapter 24). The order: why consensus is needed (the CAP theorem and three real failures), how it works, patterns built on it, performance, deployment, monitoring.

Chapter opening — untitled section before the first sub-heading in the book

- Processes crash, disks fail, disasters hit regions, so systems run across sites. Distributing geographically is "relatively straightforward", but it brings the need for a consistent view of state, "more nuanced and difficult". Processes may need to agree on who the leader is, group membership, whether a message was committed to a queue, who holds a lease, or a value for a key.
- The **distributed consensus** problem is reaching agreement among processes connected by an unreliable network. It is fundamental to distributed computing and "one we rely on for virtually every service we offer." (Figure 23-1.)
- Recommendation: wherever there is leader election, critical shared state or distributed locking, use consensus systems "formally proven and tested thoroughly". Informal approaches cause outages and subtle consistency problems.
- CAP theorem: a system cannot have consistent views, availability and tolerance to network partitions at once. Partitions are inevitable, so the real question is how consistency and availability work for your application.
- ACID versus BASE (Basically Available, Soft state, Eventual consistency). BASE stores suit certain data and volumes but rely on multimaster replication with conflict resolution like "latest timestamp wins", which surprises with clock drift or partitions. Jeff Shute: developers spend "a significant fraction of their time" coping with eventual consistency, "an unacceptable burden".
- Systems cannot sacrifice correctness for reliability or performance, particularly with critical state (financial data).

Motivating the Use of Consensus: Distributed Systems Coordination Failure

- Failures are rare, rarely tested and hard to reason about. A seeming full partition might be a very slow network, some messages dropped, or throttling in one direction only.
- Case Study 1, the split-brain problem: a content repository uses pairs of file servers in different racks and must never write to both at once. Servers heartbeat, and on silence issue STONITH (Shoot The Other Node in the Head) and take mastership: an industry standard, "conceptually unsound". On a slow or lossy network, timeouts fire, some commands are lost, giving two active nodes or both down: corruption or unavailability. Leader election by timeouts is "a reformulation of the distributed asynchronous consensus problem, which cannot be solved correctly by using heartbeats."
- Case Study 2, failover requires human intervention: a sharded database promotes secondaries via an external health checker; if a primary cannot see its secondary it makes itself unavailable and escalates to a human. No data loss, but worse availability and operational load, and "human intervention scales poorly"; such events coincide with larger infrastructure problems, and if consensus cannot elect a master, "a human is likely not better positioned to do so."
- Case Study 3, faulty group-membership: a search component uses gossip to join and elects a leader; on a partition each side elects a master and accepts writes and deletions, so split-brain and corruption. Membership is another consensus instance.
- Many problems are versions of consensus: master election, group membership, locking and leasing, reliable queuing, critical shared state. Use only proven algorithms with extensively tested implementations; ad hoc means (heartbeats, gossip) "will always have reliability problems in practice."

How Distributed Consensus Works

- We care about asynchronous consensus (unbounded message delays). Algorithms are crash-fail or crash-recover (much more useful, as real problems are transient), and Byzantine or non-Byzantine (costly, less common).
- The FLP impossibility result: no asynchronous algorithm can guarantee progress with an unreliable network. In practice: enough healthy replicas and connectivity, and backoffs with randomized delays, which prevent retry cascades and the dueling proposers problem. “The protocols guarantee safety, and adequate redundancy in the system encourages liveness.”
- Paxos (Lamport) was the original solution; others are Raft, Zab, Mencius.
- Paxos Overview: a sequence of proposals accepted or not by a majority; each has a sequence number giving a strict ordering. Phase one: the proposer sends a sequence number; an acceptor agrees only if it has not seen a higher one; numbers must be unique per proposer. With majority agreement, the proposer sends a commit message with a value. Two values cannot commit for one proposal since “any two majorities will overlap in at least one node”. Acceptors journal to persistent storage when agreeing. Paxos alone only agrees on a value once, and a node may lack a complete view.

System Architecture Patterns for Distributed Consensus

- Consensus algorithms are low-level and primitive, agreeing on a value once. Usefulness comes from higher-level components (datastores, configuration stores, queues, locking, leader election), which also allow swapping algorithms. Many users are clients of services such as Zookeeper, Consul, etcd; Chubby fills this niche at Google. Offering consensus as a service rather than a library frees maintainers from deploying a highly available consensus service.
- Reliable Replicated State Machines: an **RSM** executes the same operations in the same order on several processes; consensus orders them; any deterministic program can be a highly available replicated service as an RSM (Figure 23-2). RSMs may need to sync state from peers.
- Reliable Replicated Datastores and Configuration Stores: consensus is in the critical path, so performance matters; read consistency semantics affect scaling. Timestamps used elsewhere are problematic as clocks cannot be synchronized; Spanner models worst-case uncertainty and slows down.
- Highly Available Processing Using Leader Election: leader election is equivalent to consensus. Write a simple program, replicate it, and elect one leader (Figure 23-3), as in GFS and Bigtable. Consensus is not in the critical path, so throughput is rarely a concern.
- Distributed Coordination and Locking Services: a barrier blocks processes until a condition is met, e.g. Map before Reduce in MapReduce (Figure 23-4). A single coordinator is an unacceptable single point of failure; use an RSM. Locks likewise; use renewable leases with timeouts instead of indefinite locks; most applications should use a higher-level system providing distributed transactions.
- Reliable Distributed Queuing and Messaging: queues distribute work; claimed tasks should use leases, not outright removal. Losing the queue stops the system, so implement it as an RSM. Atomic broadcast (same messages in the same order for all) is equivalent to consensus. Pub-sub keeps messages as a persistent ordered list and can implement coherent caches. Queues need throughput more than low latency.

Distributed Consensus Performance

- Conventional wisdom says consensus is too slow; “This conception is simply not true”, though implementations can be slow. Google’s scale is a disadvantage: large datasets and wide geography raise cost and latency. There is no one best algorithm; it depends on workload (throughput, request types, read semantics, sizes) and deployment (local or wide area, quorum, sharding, pipelining, batching).
- Distinguished leaders make perceived latency vary with distance (Figure 23-6).
- Multi-Paxos: a strong leader needs one round trip to a quorum. Initial Prepare/Promise establishes a numbered view (leader term); afterwards only Accept messages (Figure 23-7). Changing proposer costs a round trip and may cause **dueling proposers**, a livelock (Figure 23-8). Practical systems elect a proposer or rotate; tune election timeouts and backoff, with randomness “the best approach” (Raft).
- Scaling Read-Heavy Workloads: reads may go to any replica if strong consistency is not needed (Photon: compare-and-set absolutely consistent, reads from anywhere since stale data causes extra work, not wrong results). For up-to-date reads: read-only consensus operation, a replica guaranteed current (the stable leader), or quorum leases.

- Quorum Leases: a lease on part of the data granted to a quorum of replicas for a brief period; writes must be acknowledged by all of them, and if one is unavailable the data cannot change until expiry. Good for read-heavy, regionally concentrated reads.
- Network latency: the constraints are RTT and storage write time. In a datacenter RTT is about a millisecond; US typical 45 ms; New York to London 70 ms. Availability needs distant replicas. TCP needs a handshake and slow start; groups keep connections open, but many clients cannot, so use regional proxies (Figure 23-9).
- Fast Paxos: clients send Propose directly to acceptors, one parallel send replacing two. Not always faster: with a slow client link and fast acceptor links, N slow messages lose to one slow plus N fast, given the latency tail effect. It also hinders batching.
- Stable Leaders (Multi-Paxos, Zab, Raft): allow read optimizations, but all writes go through the leader (latency for distant clients), its outgoing bandwidth is a bottleneck because Accept carries the data, and a slow machine slows the system. Alternatives: rotating leadership (Mencius, Egalitarian Paxos), giving clients lower RTT to the nearest replica.
- Batching raises throughput but leaves replicas idle; pipelining allows multiple proposals in flight (like TCP “keep the pipe full”), keeping global order via view and transaction numbers.
- Disk Access: logging is required so a restarted node honors commitments. A write takes one to several milliseconds. A Multi-Paxos operation involves a proposer write, parallel messages, parallel acceptor writes, and replies; a variant makes the proposer write in parallel. At 10 ms per random write, consensus is limited to about 100 operations per minute (as the book states). Combine the consensus log with the RSM log; flush state writes later; batch client operations to amortize costs.

Deploying Distributed Consensus-Based Systems

- The critical decisions are replica number and location.
- Number of replicas: a majority quorum of 2f + 1 tolerates f failures (3f + 1 for Byzantine). Minimum three (two tolerate nothing). Most downtime is planned maintenance, which three replicas tolerate. An unplanned failure during maintenance makes it unavailable, so run five (up to two failures; with three left, add one or two).
- Without a quorum the system is in theory unrecoverable, since a missing replica may have seen a decision; forced reconfiguration risks data loss. The replicated log is “not always a first-class citizen in distributed consensus theory” but matters in production, and lagging replicas must catch up. Raft defines gap-filling: a five-instance system left with its leader still knows all committed decisions.
- Slow replicas can lag outside the quorum; tolerating more failures generally yields better performance. Cost matters for sharded systems like Photon. Trade-off: reliability, maintenance frequency, risk, performance, cost.
- Location of replicas: failure domains versus latency. A **failure domain** is components that one failure can make unavailable: a machine, a rack on one power supply, racks on one network equipment, a datacenter, datacenters in one region. Greater distance raises RTT and tolerated failure size. Group membership or leader election services may not need low latency; batch systems less affected. Do not expand domains needlessly: if all clients sit in New York, surviving Hurricane Sandy is pointless as the clients are down too.
- Take disaster recovery into account and back up snapshots elsewhere; two domains cannot be escaped: software bugs and human error. Client perception matters most; leaderless protocols may have an edge over WANs.
- Capacity and Load Balancing: add shards, or replicas for reads, with a cost: more load on the leader (strong leader) or on all processes (peer-to-peer). Adding a replica can lower availability: five replicas need three for quorum (40% can be down); six need four (33%). With five datacenters and one replica each, a sixth in one datacenter means an outage there removes both spares, cutting capacity 33% (Figure 23-10).
- Nearest-replica routing can cascade under a regional spike (Chapter 22), often from batch jobs (Figure 23-11). Leaders use more resources, especially outgoing bandwidth, so balance leaders across datacenters for highly sharded systems. If leaders are colocated and their datacenter fails, links between others see sudden traffic (Figure 23-12). This can arise unintentionally from algorithms that site leaders near clients, on the fastest machines, or favor longer-running processes.
- Quorum composition: even spacing gives consistent performance, but geography (transatlantic versus intracontinental) prevents it. Five replicas, two central US, one east coast, two in Europe (Figure 23-13): the east coast replica is a linchpin where quorums overlap. Losing it makes EU to central RTT around 50% higher than EU to east coast (Figure 23-14), “a key weakness of the simple

majority quorum". Hierarchical quorum: nine replicas in three groups of three; a majority of groups, each counted if a majority of its members are up, so a central replica can be lost without large impact (Figure 23-15), at resource cost, offset by fewer consensus groups.

Monitoring Distributed Consensus Systems

- Consensus is at the core of critical Google systems, and certain aspects need special attention: members running per group and each process's health (a process can run yet not progress); persistently lagging replicas (recovering, lagging, up-to-date, leader); whether a leader exists (none means totally unavailable); number of leader changes (term or view number; too rapid means flapping, a decrease signals a serious bug); consensus transaction number (should increase); proposals seen and agreed; throughput and latency.
- Also: proposal acceptance latency distributions, inter-location network latencies, acceptor logging time, bytes accepted per second.

Conclusion

- It deliberately avoids in-depth discussion of specific algorithms, which "would rapidly become out of date, unlike the fundamentals".
- "If you remember nothing else": know the problems consensus solves and what goes wrong with ad hoc methods such as heartbeats. Whenever you see leader election, critical shared state, or distributed locking, think consensus: "any lesser approach is a ticking bomb waiting to explode in your systems."

How this chapter builds the authors' case

The Part III intro lists it under Development and presents consensus (as Paxos) as core to Google's distributed systems, leading to Chapter 24 on Cron. Within the chapter it cites Chapter 22 for cascading failure when read load overwhelms the nearest replica, and echoes that chapter's use of randomized backoff to avoid dueling proposers and retry cascades. Its thread is the case for proven consensus over heartbeats and gossip, then the performance and deployment trade-offs that make it practical.

CHAPTER 24

Distributed Periodic Scheduling with Cron

Place in the argument — why this chapter, why this title

Written by Štěpán Davidovič. The title names the problem (periodic scheduling), the setting (distributed) and the familiar tool (cron). The authors say Google's cron serves "the vast majority of internal teams that need periodic scheduling of compute jobs" and that "what might seem like a basic service" taught many lessons. The order runs from small to large: single-machine cron, the jobs themselves (idempotency decides how failure must be handled), what changes at scale, Google's design built on those requirements, then a summary.

Chapter opening — untitled section before the first sub-heading in the book

- The aim is to show how cron can work in a distributed environment "in order to increase the reliability of the system against single-machine failures"; it runs on a few machines but launches jobs across a datacenter, with a scheduler such as Borg.

Cron

- Cron lets users specify commands and when they run (garbage collection, data analysis), in **crontab** format; usually one daemon, **crond**, loads the job list and launches jobs on time.
- Reliability: "Cron's failure domain is essentially just one machine." With a separate worker machine there are two failure domains. Only the crontab must persist; launches are "fire-and-forget" and untracked. anacron is the exception: it retries daily-or-rarer jobs missed while down, using a last-launch timestamp file.

Cron Jobs and Idempotency

- Jobs differ. Garbage collection is **idempotent**, safe to launch repeatedly; an email newsletter should not run twice. A five-minute job may skip a launch; a monthly payroll job should not.
- So "there is no single answer that fits every situation", but "we favor skipping launches rather than risking double launches, as much as the infrastructure allows": a skipped launch is recoverable, since owners can monitor and act, while a double launch may be "entirely impossible" to undo. Hence "fail closed" to avoid systemically creating bad state.

Cron at Large Scale

- Extended Infrastructure: one machine among 1,000 means a 1/1000th failure could kill cron. So "we decouple processes from machines": the (reliable) datacenter scheduler picks machines and handles deaths; a launch becomes RPCs to it.
- This takes time (health check timeouts, installing software, starting), local state can be lost, and delay may exceed the one-minute minimum interval. Persisting state on a distributed filesystem such as GFS is too slow for a five-minute job (one to two minutes of delay); "hot spares" can resume quickly.
- Extended Requirements: containers enforce isolation, so resources must be known up front, jobs may be delayed, and the full state of each launch is tracked from scheduled launch to termination. Launches need multiple RPCs, so cron faces **partial launch failure** and its recovery must handle it. Replicas sit in diverse locations so one power distribution unit cannot take out all of cron.
- Cron runs per datacenter: low latency, and it "shares fate with the datacenter scheduler, cron's core dependency".

Building Cron at Google

- Tracking state: "the consistency of that state is paramount" (payroll, newsletter are not idempotent). They chose state held by the cron service itself, not external storage: GFS/HDFS suit huge files and make small writes slow, and "Base services for which outages have wide impact (such as cron) should have very few dependencies."
- The Use of Paxos: replicas use Paxos (Chapter 23); a majority suffices to process state changes. One leader alone modifies state and launches jobs; it is also the Fast Paxos leader (Figure 24-1). Failure is noticed within seconds, a ready process is elected, then a cron-specific takeover protocol runs, staying within a tolerable one-minute failover. A quorum is told synchronously of the start and end of each launch.

- The leader keeps jobs ordered by launch time like crond, announces a launch, computes the next one, and keeps specification changes synced. A launch is identified by job and start time, else ambiguity arises for every-minute jobs (Figure 24-2).
- Paxos must stay synchronous: no launch until a quorum confirms, or after failover followers might repeat a launch they never knew happened. Completion is also announced; success or failure for external reasons is irrelevant, only that the attempt happened.
- A leader losing leadership "must immediately stop interacting with the datacenter scheduler", because "Holding the leadership should guarantee mutual exclusion of access to the datacenter scheduler".
- Followers mirror state to take over "at a moment's notice", update the next launch time synchronously and track open launches. Election must converge in under a minute; then open launches must be concluded.
- Resolving partial failures: two synchronization points (about to launch, finished). If the leader died between them, one condition must hold: external operations are idempotent, or their state can be looked up. Otherwise launches are missed or doubled. A reasonable solution is to precompute datacenter job names, give them to all replicas, and let the new leader launch the missing ones. Names must include the scheduled launch time: else a short job that finished during a long failover would be seen as complete and launched again.
- Storing the State: Paxos is a log needing compaction and storage. A **snapshot** replaces many entries (a thousand increments become "Set counter to 1,000"). Lost logs cost only recent state, but "Snapshots are in fact our most critical state".
- Three replicas keep logs and snapshots on local disk; snapshots are also backed up to a distributed filesystem. Logs are not, since frequent small writes are costly; losing changes since the last snapshot is a conscious, acceptable risk. A new replica can fetch state from a running one, so rescheduling a replica is "essentially a nonissue".
- Running Large Cron: the **thundering herd**: 30 teams each running a midnight daily job that spawns a MapReduce with thousands of workers. Fix: a crontab question mark lets the system pick the value by hashing the job configuration over the range (e.g. 0..23 for hour). Load stays spiky (Figure 24-3), often because jobs need specific times.

Summary

- Datacenters as the smallest effective hardware unit change large parts of the stack, cron included. The design needs strong consistency, so its core is Paxos, "a commonplace algorithm to reach consensus in an unreliable environment"; with analysis of new failure modes it gives a "robust cron service that is heavily used in Google".

How this chapter builds the authors' case

It relies on Chapter 23 for Paxos, applying consensus to launch notices, leader exclusion and snapshots, and on the datacenter scheduler (Borg) as cron's core dependency. The Part III introduction places it as the system that scales cron to whole datacenters and beyond, which it calls no easy task, and next turns to data processing pipelines, which Chapter 25 says are typically scheduled by periodic schedulers such as cron and share its thundering herd problem.

CHAPTER 25

Data Processing Pipelines

Place in the argument — why this chapter, why this title

Written by Dan Dennison. The title names a pattern; the case is that its frequency matters. The chapter treats "the frequency continuum between periodic pipelines that run very infrequently through to continuous pipelines that never stop running" and the discontinuities that cause operational problems, then offers "a fresh take on the leader-follower model" as a more reliable, better-scaling alternative. Sections run from the pattern's problems to Workflow and its guarantees.

Origin of the Pipeline Design Pattern

- Classic approach: a program reads, transforms and outputs data, usually scheduled by a periodic scheduler such as cron. This is a **data pipeline**.

Initial Effect of Big Data on the Simple Pipeline Pattern

- Big Data programs are "simple, one-phase pipelines"; given scale and complexity they are chained, each output feeding the next: **multiphase pipelines**, designed "for ease of reasoning about the system". The number chained is the **depth**, one if shallow, tens or hundreds if deep.

Challenges with the Periodic Pipeline Pattern

- They are stable with enough workers, demand within capacity and uniform chained jobs. They run at Google, but "the periodic pipeline model is fragile": carefully tuned parameters work at first, then organic growth causes missed deadlines, resource exhaustion and hanging chunks.

Trouble Caused By Uneven Work Distribution

- "Embarrassingly parallel" algorithms cut work into machine-sized chunks, but chunks can need uneven resources. Partitioned by customer, runtime is capped by the largest customer.
- "Hanging chunk": user code waits for total computation before the next stage (e.g. sorting), so completion is blocked on worst-case performance.
- The "sensible" response, kill and restart, worsens matters: pipelines usually lack checkpointing, so all chunks restart, wasting time, CPU cycles and human effort.

Drawbacks of Periodic Pipelines in Distributed Environments

- Periodic pipelines run as lower-priority batch jobs, which Borg assigns to available machines to control cost, giving "open-ended startup delays". Overuse risks preemptions. A well-tuned pipeline is "a delicate balance between high resource cost and risk of preemptions".
- Frequency has a floor: the minimum time between runs reaches the minimum average delay, a lower bound on latency. Below it runs stack up, or a nearly finished run is killed. Figure 25-1: below about 40 minutes for a ~20-minute job risks overlap.
- The fix is sufficient capacity, but teams resist acquiring resources for a shared pool.
- Monitoring: metrics usually arrive only on completion, so a failed job gives none. Periodic pipelines "shouldn't have inherent monitoring problems, but we have observed a strong association".
- "Thundering herd" (Chapter 24): thousands of workers start each cycle; misconfiguration or faulty retry overwhelms servers, cluster services and network. Naive retry compounds it; inexperienced engineers add workers. Nothing is harder on infrastructure and SREs "than a buggy 10,000 worker pipeline job".
- Moiré load pattern: pipelines whose runs occasionally overlap, consuming a shared resource together (Figures 25-2, 25-3: peak on-call pain near 1.2M aggregate load).

Introduction to Google Workflow

- Demand for continuous results forces refactoring or going continuous, usually at the least convenient time; so scope growth, resources and latency requirements at the outset.
- Google developed Workflow in 2003, using the leader-follower (workers) pattern and system prevalence: very large-scale transactional pipelines with "exactly-once semantics".
- Workflow as Model-View-Controller Pattern: the distributed equivalent of MVC (Figure 25-4). The model is the **Task Master**, holding job states in memory and synchronously journaling mutations to

disk. The view is the stateless, discardable workers updating state transactionally. Best performance: only pointers to work there, data in a common filesystem. An optional controller handles scaling and rollback (Figure 25-5).

Stages of Execution in Workflow

- Depth grows by subdividing processing into task groups, each a stage; workers consume units from the previous stage. It is easy to guarantee work is executed "exactly once".
- Correctness: the Task Master (RAM-limited) holds pointers to uniquely named data, and each work unit has a uniquely held lease; workers commit only with a valid lease. Unique output file names mean orphaned workers can neither commit nor destroy valid work: "the output files are always unique, and the pipeline state is always correct".
- Tasks are versioned; a worker must cite the task ID of the configuration it used, so after a configuration change old workers cannot commit despite leases. Result: a triple guarantee of configuration, lease ownership and filename uniqueness.
- A rogue Task Master at the same address, or a load balancer fronting independent ones, is prevented by a server token checked on every operation.
- Why not Spanner or another database: each task is unique and immutable; a database journal would need every read in a long-running transaction, "terribly inefficient".

Ensuring Business Continuity

- Pipelines without system prevalence often enter an undefined state when datacenters fail. The Task Master journals to Spanner (global, consistent, low-throughput) and elects its writer via Chubby.
- Because Spanner is slow, a global Workflow runs two or more local Workflows in distinct clusters plus reference tasks: a "stage 1" helper inserts them as work is consumed, "stage n" removes them transactionally, and the local Workflow blocks if it cannot (Figure 25-6). The helper, acting as MVC controller, updates a heartbeat task; if it times out, the remote helper seizes the work in progress.

Summary and Concluding Remarks

- "Periodic pipelines are valuable." But if a problem is continuous or will grow to be, "don't use a periodic pipeline"; use technology like Workflow, which scales well and is stable for SREs.

> **How this chapter builds the authors' case**
>
> It builds on Chapter 24: pipelines are scheduled by cron-like schedulers and inherit the thundering herd problem, which it cross-references. The Part III introduction frames it as showing how different pipeline architectures lead to surprising, counterintuitive challenges.

CHAPTER 26

Data Integrity: What You Read Is What You Wrote

Place in the argument — why this chapter, why this title

Written by Raymond Blum and Rhandeev Singh. The title is the definition in miniature: a read must return what was written, and the user must be able to read it. The chapter opens by redefining the word from the user's side, then sets the requirements and the objectives (SRE wants measurable ones), then shows how Google meets them with layered defenses, proves the layers with two real recoveries (Gmail 2011, Google Music 2012), and finally gathers the SRE principles the cases illustrate.

Chapter opening — untitled section before the first sub-heading in the book

- "When users come first, data integrity is whatever users think it is." Defining it as accessibility and accuracy of datastores is "insufficient": if a Gmail interface bug shows an empty mailbox for too long, users believe data is lost even if none was, and the world would question Google as steward of data.
- How long is "too long"? In a real 2011 Gmail incident, four days was perhaps too long; they believe 24 hours is a good starting point for Google Apps. The same applies to Photos, Drive, Cloud Storage and Cloud Datastore, since users do not distinguish products. "Data loss, data corruption, and extended unavailability are typically indistinguishable to users." So data integrity applies to all data in all services, and user access to data is especially important.

Data Integrity's Strict Requirements

- Uptime may seem stricter than integrity, but consider the numbers. A 99.99% uptime SLO leaves about an hour of downtime a year. A 99.99% good bytes SLO on a 2 GB artifact would leave up to 200 KB garbled, catastrophic: executables with random opcodes, unloadable databases. Each service has independent uptime and integrity requirements, even implicit ones; "The worst time to disagree with users about these requirements is after the demise of their data!"
- Revised definition: services in the cloud remain accessible to users, and access should remain in perfect shape. An artifact corrupted once a year and unrecoverable loses the year's uptime; if corruption is detected before users are affected and fixed within half an hour, it is 99.99% available with integrity near 100%. "The secret to superior data integrity is proactive detection and rapid repair and recovery."
- Choosing a Strategy for Superior Data Integrity: strategies trade uptime against integrity; the answer depends on your computing paradigm. Cloud applications optimize some mix of uptime, latency, scale, velocity and privacy (here, deletion within a reasonable time), on ACID and BASE APIs. BASE gives higher availability for softer consistency.
- Scenario: when velocity trumps other needs, apps use whatever APIs developers know: blobs in Blobstore (cheap, weakly consistent), authoritative metadata in a costlier Paxos-based service like Megastore, client caches. Result: challenges such as referential integrity between datastores, and "an environment riddled with complex relationships between various pieces of data that no single engineer fully groks". It needs out-of-band checks and balances (see the Third Layer). Uncoordinated backups of several datastores complicate recovery.
- Backups Versus Archives: "No one really wants to make backups; what people really want are restores." The key difference: backups can be loaded back into an application, archives cannot. Archives keep data long-term for audit, discovery and compliance and need not recover within service uptime (financial data for seven years; a week to retrieve logs may be acceptable). Real backups must restore within uptime needs and may run daily, hourly or continuously; consider how quickly you must recover and how much recent data you can lose.
- Requirements of the Cloud Environment in Perspective: mixed transactional and nontransactional restores may be incorrect; services evolving without downtime run different business-logic versions in parallel; independently versioned services may interact incompatibly. Providers offer few APIs, simple yet robust enough for locality, distribution, consistency and backup and recovery; problems arise in some combinations, and unless the provider solves them, applications must.

Google SRE Objectives in Maintaining Data Integrity and Availability

- "Maintaining integrity of persistent data" is a good vision, but SRE wants concrete objectives with measurable indicators.

- Data Integrity Is the Means; Data Availability Is the Goal. Integrity means accuracy and consistency over a data's lifetime. Story: an email provider's weeklong outage ended with an announcement that the data was gone, users left, and days later it proved recoverable. But it was inaccessible for too long. Moral: "data integrity without expected and regular data availability is effectively the same as having no data at all."
- Delivering a Recovery System, Rather Than a Backup System: backups are classically neglected; the dangers are low risk but high impact. Motivate with the visible payoff, the restore. "Backups are a tax", paid for the service of guaranteed data availability. So teams define SLOs for data availability under various failure modes, and practice and demonstrate meeting them.
- Types of Failures That Lead to Data Loss: three factors give 24 combinations (Figure 26-1). Root cause: user action, operator error, application bugs, infrastructure defects, faulty hardware, site catastrophes. Scope: widespread or narrow. Rate: big bang (1 million rows replaced by 10 in a minute) or creeping (10 rows deleted per minute over weeks).
- A study of 19 data recovery efforts at Google found the most common user-visible losses were deletion or loss of referential integrity from software bugs; the hardest was low-grade corruption or deletion found weeks to months later. Recovering means retrieving data for perhaps millions of users, each artifact to its own point in time: "point-in-time recovery", "timetravel" inside Google. Doing that across ACID and BASE stores within strict goals "is a chimera today". Many projects choose a tiered backup strategy without it; trade between them if necessary, but use both if you can.
- Challenges of Maintaining Data Integrity Deep and Wide: "replication and redundancy are not recoverability". "We have something even better than a backup—replication!" fails because synced replicas push a corrupt row or errant delete to all copies. Non-serving copies such as database exports guard against user and application errors but not lower-layer loss (a zero-day in the filesystem or driver). So "diversity is key": protecting against failure at layer X needs diverse components at that layer (media isolation; tape versus disk).
- Freshness competes with protection: the lower the snapshot is pushed in the stack the longer the copy takes. A transaction replicates in seconds, a database export may take 40 minutes, a filesystem backup hours; restore takes about as long. Retention: gradual loss may take days to notice, needing older snapshots merged with current state, which complicates restore.

How Google SRE Faces the Challenges of Data Integrity

- They assume protection mechanisms can fail as underlying systems do, so need "a number of complementary but uncoupled practices". There is no silver bullet for the 24 combinations; use **defense in depth**, layers covering progressively less common loss (Figure 26-2): first soft deletion, second backups and recovery, third regular data validation; replication is occasionally useful but plans should not rely on it.
- First Layer: Soft Deletion. With high velocity and privacy, application bugs cause most loss. Soft deletion marks data deleted at once, unusable except by administrative paths (legal discovery, hijacked account recovery, support), then destroys it after a delay. A trash folder reduces but cannot eliminate the support burden. Gmail's trash keeps messages under 30 days. Hijackers also delete users' data. Common delays are 15, 30, 45 or 60 days; most hijacking and integrity issues surface within 60 days.
- The most devastating acute deletion cases come from developers unfamiliar with the code working on deletion code, especially batch pipelines; design interfaces so they cannot circumvent soft deletion, and enable built-in soft deletion: "Even the best armor is useless if you don't put it on." For cloud offerings add **lazy deletion**, controlled by the storage system, keeping deleted data up to a few weeks; it is costly for short-lived data and impractical under privacy guarantees. Summary: trash is the primary defense against user error; soft deletion against developer error (secondary for user error); lazy deletion against internal developer error. Revision history often treats deletion as removing prior states, so apply lazy or soft deletion to it too.
- Second Layer: Backups and Their Related Recovery Methods: "backups don't matter; what matters is recovery"; recovery scenarios should dictate methods, frequency, location and retention. Less tolerable loss means incremental backups. Take full backups off-peak. Faster rescue means more local backups: costly snapshots briefly within the storage instance, then distributed storage nearby, then nearline or offline for site failures. Low-grade bugs need reach back months, but code and schema change make old backups costly or unusable, and restoring subsets to different points is hard. Google drew the line between 30 and 90 days for many services.
- Tiers: first, frequent quick backups closest to live data, kept hours to single-digit days, restore in minutes; second, fewer backups on local distributed filesystems, kept single-digit or low double-digit

days, restore in hours; later tiers on tape or offsite media protect against site-level issues; they are taken less often and kept longer.

- Overarching Layer: Replication: ideally every instance including backups is replicated; otherwise stagger backups across sites and use RAID, Reed-Solomon codes or GFS-style replication, choosing a common scheme in continual use rather than one tested only by your own recoveries.
- 1T Versus 1E: validating 700 petabytes at an ideal 300 MB/s would take 8 decades; a restore nearly a century, of a backup up to 80 years old. The technique: “trust points”, data verified once immutable, with incremental backups of the rest. But restores matter: three years of daily incrementals means a chain of over 1,000 interdependent backups, each adding risk. Also shard data to run N tasks in parallel; with time slices this cuts wall time by several orders of magnitude.
- Third Layer: Early Detection: “Bad” data propagates, so the sooner you know, the more complete recovery. Developers face referential integrity, schema changes, aging code, zero-downtime migrations and evolving integrations. Novices delegate integrity to a consistent API (Megastore, Paxos), but implementations are “riddled with hacks, optimizations, bugs, and educated guesses”. If true of Paxos, more so of eventually consistent Bigtable: “trust storage systems, but verify!”
- Out-of-band data validation: pipelines, usually MapReduce or Hadoop, often added late. They slow velocity short-term but give developers courage to move faster, like unit tests. Gmail validators catch bugs within 24 hours, enabling code changes more than weekly. Too strict and engineers abandon it; too lax and corruption slips through: “only validate invariants that cause devastation to users”. Drive validates that file contents match folder listings and fixes inconsistencies automatically, turning a 2013 emergency into “let’s go home and fix the root cause on Monday”.
- Validators are expensive: a significant portion of Gmail’s compute footprint, lowering cache hit rates; knobs and refactoring cut disk spindle contention 60%; the largest validator is split into 10–14 shards, one per day. Tiered validation: sacrifice rigor in daily validators but keep catching the most disastrous cases within 24 hours, with rigorous checks less often. Troubleshooting needs audit logs, playbooks, tools and dashboards. Because small teams cannot build all this, a central infrastructure team should provide the framework while product teams maintain the validator’s business logic.
- Knowing That Data Recovery Will Work: like a bulb that broke before the switch was flicked, recovery may be latently broken. Continuously test recovery and alert when it fails to give a heartbeat. “The only test that should let you sleep at night is a full end-to-end test”; “you only know that you can recover your recent state if you actually do so.” Automate and run continuously. Check that backups are valid and non-empty, resources suffice, wall time is reasonable, progress is monitorable, and no dependencies are outside your control (an offsite vault not open 24/7). Found in an emergency, such failures might have sunk some successful products.

Case Studies

- Gmail, February 2011, restore from GTape: coincident failures, and the largest use of the last line of defense, the GTape offline backup system, which was the first large-scale use to restore live data. On Sunday, February 27, the backup pager fired: Gmail had lost a significant amount of user data. Because such restores had been simulated many times, they delivered an estimate, restored all accounts within several hours of it and recovered 99%+ before the estimated time. This was “the fruit of planning”, Defense in Depth and Emergency Preparedness.
- Tape? Google has disks and networks, but defense in depth means layers against failure of internal redundancy (this case) and against a zero-day in a driver or filesystem affecting disk. Many teams pitched in under a central plan from dress rehearsals. They view failures as inevitable and plan for “random undifferentiated breakage”, too.
- Google Music, March 2012, runaway deletion detection: the challenge was logistics, over 5,000 tapes. A user reported skipped tracks on March 6; on March 7 an engineer found track metadata missing the audio reference, removed by the privacy deletion pipeline, redesigned for efficiency and first run February 6 with its second stage deleting audio. A hasty MapReduce on March 8 found about 600,000 audio references wrongly removed, affecting 21,000 users (possibly worse).
- Resolving: fix the root cause or recovery is in vain (re-enabling would hurt innocent users: a Catch-22), so work in parallel. Audio was on tape, but trucked offsite. Experienced SREs recovered while developers hunted the bug. Recent DiRT testing meant the tape team knew its tools. Only 436,223 of about 600,000 tracks were on tape; about 161,000 were deleted before backup. First wave: 1.5 PB, split into 5,475 restore jobs (by hand, over three days, so SRE built a programmatic solution); requested by midnight March 9; 5,337 tapes recalled; technicians loaded tapes by hand, hundreds of times faster than vendor robots in DiRT. It ran past the two-day estimate (a vendor omitted 1,862 tapes; 17 were bad, covered by redundant encoding); 99.95% was done by March 11; after an

unrelated outage, on March 13 all 436,223 tracks were accessible, in just short of 7 days (5 of recovery).

- Second wave: most of the 161,000 were store-bought or promotional and were reinstated from store copies; user uploads were re-requested from clients from March 14, over a week. Root cause: pipelines work in stages on short-lived copies to limit load, and unless relative age is coordinated they create race conditions. Stages designed three hours apart break as data grows, probabilistically; optimizations and then the refactor raised the probability of wrongful deletion. Fixes: redesign to eliminate the race and better monitoring and alerting for runaway deletion (footnote: alert on the global deletion rate, e.g. 10x the 95th percentile, not per user).

General Principles of SRE as Applied to Data Integrity

- Beginner's Mind: complex services have bugs no one fully groks; never think you understand enough to say it won't fail in a certain way. Trust but verify, with defense in depth.
- Trust but Verify: any API has defects; check critical data with out-of-band validators. "Perfect algorithms may not have perfect implementations."
- Hope Is Not a Strategy: unexercised components fail when needed; prove recovery with regular exercise. Automation helps, though staffed by engineers with competing priorities it may yield stopgaps.
- Defense in Depth: even bulletproof systems have bugs and operator error; detect quickly; "Every strategy eventually fails in changing environments"; the best strategies are multitiered.
- Revisit and Reexamine: "was safe yesterday" does not help tomorrow. Example: the Shakespeare service needed no index backups because it could be re-created by MapReduce; when annotations arrived, data could not be re-created, so replication for integrity, a tested backup and restore, and DiRT testing against the SLO were needed.

Conclusion

- Data availability must be a foremost concern. Like test-driven development, SRE proves systems can maintain availability with a predicted maximum down time; means are "necessary evils", avoiding "The operation was a success, but the system died."
- Recognize that "everything will go wrong". A matrix of disasters with plans lets you sleep one night; keeping plans exercised, the other 364. As recovery time N shrinks through faster, finer detection, approach N = 0, then plan prevention for "all the data, all the time".

How this chapter builds the authors' case

The Part III introduction says that making sure stored data is still there when you want to read it is the heart of data integrity, and that this chapter explains how to keep data safe. The chapter leans on the Paxos of Chapter 23 (what it guarantees in theory versus in implementation), on pipelines in the sense of Chapter 25 (the Music deletion pipeline was a staged data pipeline with a race), and on DiRT exercises (the footnote and the Shakespeare example). Its principles (trust but verify, hope is not a strategy) restate SRE principles from earlier in the book, and the next chapter takes the reliability pyramid to the product launch.

CHAPTER 27

Reliable Product Launches at Scale

Place in the argument — why this chapter, why this title

Written by Rhandeev Singh and Sebastian Kirsch with Vivek Rau. The title joins two demands: launches must be reliable, and they happen at scale (up to 70 a week). The Santa story shows a risky launch; the answer follows in the order who (the LCE team), how (the process), what (the checklist), which techniques, and how the team arose and fell short. The authors state SRE's role: "to enable a rapid pace of change without compromising stability of the site."

Chapter opening — untitled section before the first sub-heading in the book

- Internet companies launch "in far more rapid iterations than traditional companies". Google created a dedicated team of Launch Coordination Engineers and a "launch checklist" of common questions and recipes, "a useful tool for ensuring reproducibly reliable launches".
- Example: Keyhole, serving satellite imagery to Maps and Earth, normally serves up to several thousand images per second; on Christmas Eve 2011 NORAD Tracks Santa's "virtual fly-over" brought 25 times its normal peak, over one million requests per second. It had a hard deadline, publicity, an audience of millions, a steep ramp-up. The kill switches protecting services were nicknamed make-children-cry switches.
- A **launch** is any new code that introduces an externally visible change to an application; up to 70 a week. This rate is "both the rationale and the opportunity" for a streamlined process.

Launch Coordination Engineering

- Good engineers may not know the pitfalls of launching to millions. Google made a consulting team within SRE to guide developers toward robust, scalable, reliable products. LCE audits for compliance with reliability standards, liaises between teams, drives the technical side so tasks keep momentum, acts as gatekeeper signing off on launches judged "safe", and educates developers.
- Audits mostly precede launch; even products with strong SRE support engage LCE for critical launches.
- The Role of the Launch Coordination Engineer: hired directly or SREs with hands-on experience, same bar as any SRE, plus communication and leadership.
- Advantages: breadth of experience (knowledge transfer); a cross-functional perspective for launches spanning half a dozen teams; objectivity between SRE, developers, product managers and marketing. As an SRE role, LCEs are incentivized to prioritize reliability.

Setting Up a Launch Process

- After more than 10 years, a good process is: lightweight (easy on developers), robust (catches obvious errors), thorough (consistent and reproducible), scalable (many simple and fewer complex launches), adaptable (common and new types, like Chrome). Some conflict, e.g. lightweight and thorough, so balancing takes continuous work.
- Tactics: **simplicity** ("Get the basics right. Don't plan for every eventuality."); a **high touch approach** (experienced engineers customize the process to each launch); **fast common paths** for classes of launches with a common pattern, such as a new country.
- Engineers sidestep burdensome processes, so LCE must continuously balance cost and benefit.
- The Launch Checklist: checklists reduce failure in aviation and surgery; LCE's (Appendix E) helps assess the launch and gives action items. Examples: new domain name? persistent data? implement backups; user abuse? rate limiting and quotas.
- Questions are near-infinite, so the list can grow unmanageable; at one point adding a question needed a vice president's approval. Now: every question's importance substantiated, "ideally by a previous launch disaster"; every instruction concrete, practical and reasonable. Continuous curation, plus a full review once or twice a year.
- Driving Convergence and Simplification: engineers unaware of existing infrastructure reimplement it. Since almost all groups share the launch process, the checklist drives convergence on hardened common infrastructure. Long sections shrank to one line: "Implement rate limiting using system X." LCEs see the stumbling blocks across launches, so they can simplify approvals and escalate pain points to infrastructure owners.

- Launching the Unexpected: in a new product space an LCE may create a checklist from scratch with domain experts, around themes like reliability, failure modes and processes. Before Android, Google rarely shipped client logic it did not control; a Gmail bug is fixed in hours by pushing JavaScript, not on mobile. Keep each question's intent in mind to avoid "mindlessly applying" irrelevant items: return to first principles, then respecialize.

Developing a Launch Checklist

- Details differ for every company, so the chapter extracts themes from Google's checklists.
- Architecture and Dependencies: an architecture review checks that shared infrastructure is used correctly and identifies its owners as stakeholders; the dependency list feeds capacity planning. Actions: isolate user-facing requests; validate volume assumptions, since one page view can become many requests.
- Integration: internal ecosystems have idiosyncrasies, so this varies by company. Actions: DNS name, load balancers, monitoring.
- Capacity Planning: launch spikes differ in mix from steady state, skewing load tests; some products handled spikes up to 15 times the estimate; launch one region at a time to build confidence. If three deployments serve peak, keep four or five. Datacenter and network resources have long lead times.
- Failure Modes: examine each component and dependency for the impact of failure (machines, datacenters, network, bad input, DoS, degraded mode). Actions: request deadlines; load shedding to reject new requests early in overload.
- Client Behavior: on a traditional website, load is limited by how fast users click. That fails for clients that act without user input (app syncing, auto-refreshing site), where abusive behavior threatens stability. Actions: exponential backoff on failure; jitter automatic requests.
- Processes and Automation: some processes need humans (a release, moving datacenters, restoring backups), but humans are single points of failure, so document processes before launch so "any team member can execute a given process in an emergency".
- Development Process: development on mainline, releases on separate branches so bugs can be fixed without unrelated changes; config files also versioned (history, attribution, review). Actions: check in all code and config; cut each release on a new branch.
- External Dependencies: third-party code, services or data. Planning averts damage from vendor outages, bugs or scalability limits; Google used proxies, transcoding pipelines and caches.
- Rollout Planning: few events in large systems are instantaneous; configuration changes may take hours, and order matters. Marketing and PR add constraints; contingency can be a backup slide deck saying "We will be launching this feature over the next days". Actions: a launch plan with owners and contingencies.

Selected Techniques for Reliable Launches

- Gradual and Staged Rollouts: "never change a running system"; any change is risk, doubly so for globally distributed systems. Very few launches are "push-button". A new server goes on a few machines, then a datacenter, then globally, with observation between. First stages are **canaries**, after the miners' birds; tools roll back automatically if validation fails. It applies to Android app versions, offered to a growing percentage of installs, and to invite systems and rate-limited signups.
- Feature Flag Frameworks: slow rollout under real workloads pays for itself in reliability, velocity and time to market, especially where realistic test environments are impractical. They allow testing small UI tweaks in parallel, and a prototype on a small sample before months of hardening for a feature that may flop. Requirements: many changes in parallel; ramp to usually 1 to 10 percent; route by user, session, object or location; handle new-path failure without affecting users; independent immediate revert; measure improvement. Two classes: UI improvements and server-side or business-logic changes. The simplest is an HTTP payload rewriter at frontend servers keyed to cookies; stateful services reroute requests by user or entity ID.
- Dealing with Abusive Client Behavior: a client syncing every 60 seconds, not 600, causes 10 times the load. Retries add load to an overloaded service, so use exponential delay and retry only suitable errors (network error yes, 4xx usually no). Synchronized requests form a thundering herd (Chapters 24 and 25): so clients choose times randomly; retries at 1, 2, 4 seconds repeat a spike unless each delay is jittered. Server-side control of clients (a downloaded configuration file) is an important tool; dormant code for new functionality reduces launch risk, avoids a combinatorial explosion of versions, and lets you switch a feature off instead of shipping a new app.

- Overload Behavior and Load Tests: overload has many causes beyond runaway success. The naive linear model of CPU versus load fails: services are often slower unloaded because of caches; after a linear window comes nonlinearity, from rising response times to complete lock-up. Example: a service logging debug data on backend errors found logging costlier than normal handling, so it ground to a halt; the JVM analog is “GC (garbage collection) thrashing”. As behavior is hard to predict, “load tests are an invaluable tool”, required for most launches.

Development of LCE

- When the engineering team doubled every year, small teams on experimental products risked repeating predecessors’ mistakes. Volunteer “Launch Engineers” wrote checklists and held “Launch Reviews” days to weeks before launches.
- Requirements grew long, and inexperienced SREs were “overly cautious and averse to change”, risking slower launches. So in 2004 SRE staffed a small full-time LCE team to accelerate launches while ensuring high availability and low latency, and keeping stakeholders informed when corners were cut.
- Evolution of the LCE Checklist: in 3.5 years one LCE ran 350 launches; with an average of five engineers, over 1,500 launches. A new LCE needs about six months of training. Low-risk launches (no new server executables, traffic increase under 10%) got an almost trivial checklist: 30% of reviews by 2008. Google’s scaling removed constraints so small products “fit within the cracks”.
- Problems LCE Didn’t Solve: by 2009 launching a small new service was legendary for difficulty. Scalability changes: usage up two orders of magnitude forces design changes, then rearchitecture. Growing operational load: SRE keeps operational work below 50% (Chapter 5). Infrastructure churn: owners “running fast just to stay in the same place”; the fix bars backward-incompatible features until client migration is automated. These need company-wide efforts (platform APIs and frameworks, Chapter 32; automation; standardization), beyond LCE.

Conclusion

- A Launch Coordination role suits companies with rapid growth and a high rate of change: doubling developers every one or two years, hundreds of millions of users, reliability despite change. The LCE team was “Google’s solution to the problem of achieving safety without impeding change”, drawn from a 10-year experience.

How this chapter builds the authors’ case

The Part III introduction places it last on the reliability pyramid, once the product is workable: it is about giving users the best possible experience from Day Zero. It reuses earlier chapters: the thundering herd of Chapters 24 and 25, the cap on operational work from Chapter 5, and Chapter 32 for what LCE could not solve. It points to Appendix E (checklist) and Appendix B (rollout patterns).

PART IV

MANAGEMENT

- The authors' own description: "Our final selection of topics covers working together in a team, and working as teams. No SRE is an island, and there are some distinctive ways in which we work."

The theme of the Part

- How SRE people and teams are trained, protected from overload, and organised to collaborate and engage with services.

How the chapters build the theme

- Chapter 28: a structured training program can instil "best practices within a new hire's first few weeks or months that otherwise would take months or years to accumulate."
- Chapters 29–30: managing interrupts under turbulent conditions, then recovering from operational overload.
- Chapter 31: roles, cross-team and cross-site communication, production meetings, collaboration case studies.
- Chapter 32: the production readiness review (PRR), "a crucial step in onboarding a new service", and how to move beyond that model.

What the Part contributes to the book

- Turns the technical practices into a way of working as people and teams.

CHAPTER 28

Accelerating SREs to On-Call and Beyond

Place in the argument — why this chapter, why this title

Written by Andrew Widdowson (edited by Shylaja Nukala). It opens Part IV, Management, whose intro says a well-executed training program can instil in a new hire's first weeks or months practices that would otherwise take months or years to accumulate. The title gives the arc: hire, going on-call, then learning that continues. Sub-headings follow it: why structure matters, what kind of engineer to build, five practices leading to on-call, then the rite of passage and continuing education.

You've Hired Your Next SRE(s), Now What?

- Up-front investment in education makes better engineers and brings proficiency faster, with a "more robust and balanced" skill set.
- Teams rest on trust: fellow on-callers must know how the system works, diagnose atypical behaviour, ask for help and react under pressure. "What does a newbie need to learn to go on-call?" is therefore "essential, then, but not sufficient". Also ask how on-callers can assess readiness, how to harness new hires' curiosity so existing SREs benefit, and which activities benefit everyone.
- Learning preferences are mixed, so "there is no style of education that works best". Table 28-1 pairs recommended patterns with anti-patterns, e.g. concrete sequential experiences vs. "trial by fire"; reverse engineering vs. training only through checklists and playbooks; reading postmortems vs. treating outages as secrets; realistic breakages vs. a first chance to fix only once on-call; early shadowing vs. pushing students to primary too soon; nontrivial project work vs. giving all new projects to seniors.
- Figure 28-1 has an x-axis from abstract to applied work and a time axis top-down: postmortems and reverse engineering first, then shadowing on-call and mending documentation. Going on-call is a milestone after which learning is "nebulous, undefined, and self-directed" (dashed lines); project work is a triangle that starts small and grows. Activities should be paced: some now, some just before on-call, some continual.

Initial Learning Experiences: The Case for Structure Over Chaos

- Teams aim to contain reactive work through proactivity, and onboarding should too. In the all-too-common example John is given all incoming tickets on day one and told "We're throwing you in the deep end of the pool here." This "trial by fire" comes from ops-driven teams and may alienate capable engineers; it presumes things can be taught "strictly by doing, rather than by reasoning". If a ticket queue trained for the job, "this is not an SRE position."
- Students ask what they are working on, how far they have come, and when they can go on-call; unclear answers sap confidence and can cause retention problems. The remedy is concrete but sequential.
- **Learning Paths That Are Cumulative and Orderly**: frontload recurring abstract concepts and give hands-on work early. Example order for a serving stack: how a query enters; frontend serving; mid-tier services; infrastructure; tying it together (debugging, escalation, emergencies).
- Google Search SRE's **on-call learning checklist** lists per component the frontends, backends, SRE experts and developer contacts, docs to read, "Know before moving on" items and comprehension questions. It encodes no playbooks, so it stays "future-proof". Mentor-checked homework gauges retention; a tiered access model (read-only first, later mutation) gives "powerups" on the route to on-call.
- **Targeted Project Work, Not Menial Work**: ownership aids learning and trust; time split between learning and projects gives purpose. Starter projects: a trivial user-visible change shepherded to production (empathy for developers); monitoring for blind spots; automating a not-quite-painful-enough pain point (appreciation for removing toil).

Creating Stellar Reverse Engineers and Improvisational Thinkers

- The key question is what kind of engineer to create. SREs cannot be merely traditional operations-focused system administrators; they need reverse engineering for unseen systems, statistical rather than procedural thinking for hard-to-detect anomalies, and improvisation when procedures break down.

- **Reverse Engineers**: dig into debugging tools, RPC boundaries and logs; practice inference so it is reflexive in outages.
- **Statistical and Comparative Thinkers**: incident response is pruning a massive decision tree quickly, using experience and hypotheses. Breakages are a game of "which of these things is not like the other?" (kernel, CPU, binary version, traffic mix); architecture must allow factors to be controlled and compared.
- **Improv Artists**: when a fix fails and developers are unreachable, improvise. Several tools give defence in depth; being too procedural, or holding untested assumptions, gets you stuck; teach "zooming out" early.
- **Tying This Together**: the class "Reverse Engineering a Production Service (without help from its owners)". The Google News team is lost on a Bermuda Triangle cruise for 30 days; students, the new News SRE team, trace a query inbound, finding multiple ways to discover connectivity, using self-reporting RPCs and monitoring, and find a second endpoint that shows the first assumption too narrow. Afterwards each student diagrams a stack they will be on-call for and presents it to a senior, who may also learn, exposing drift. Teams should learn "from the newest, rather than oldest, members".

Five Practices for Aspiring On-Callers

- "Able to take on-call" is a proxy for "knows enough and can figure out the rest."
- **A Hunger for Failure**: postmortems (Chapter 15) are blame-free routes to root causes. Collect and curate "teachable postmortems" for newbies; run "postmortem reading clubs" or "tales of fail" sessions. They build a mental map of production and fuel disaster scenarios.
- **Disaster Role Playing**: the weekly "Wheel of Misfortune" or "Walk the Plank"; humorous names make it less intimidating. A game master picks a primary and secondary on-caller and announces a page; scenarios come from past outages, hypothetical breakages or newly found ones. Over 30-60 minutes the GM supplies context, plays other teams, and steers away from red herrings. Everyone learns something, and newbies see they could have solved it.
- **Break Real Things, Fix Real Things**: hands-on experience is better still, and should come before on-call. "Realism is paramount": a diverted multihomed instance or staging instance, under synthetic load, where proctors alter the job set for newbies to diagnose. Or start from a known good configuration and impair bottlenecks, as in Search SRE's "Let's burn a search cluster to the ground!": discuss expected changes, poll predictions, validate. Done quarterly, it shakes out bugs.
- **Documentation as Apprenticeship**: the checklist must be internalized before shadowing. It sets system boundaries for students, shows progress to mentors and managers, and is for all "a social contract". Newbies need current docs most, so Search SRE sorts sections by staleness and assigns newbies one or two to overhaul, peer-reviewed by the listed experts.
- **Shadow On-Call Early and Often**: no hypothetical replaces real outages. Copy pages to the newbie, at first in business hours; with no appointed responsibility there is no time pressure, and mentor and student review afterwards. It builds trust and lets seniors detach, avoiding burnout. Make the newbie a coauthor of postmortems, not the sole writer. Some teams add "reverse shadow": newbie primary, expert diagnosing without modifying state.

On-Call and Beyond: Rites of Passage, and Practicing Continuing Education

- Go on-call when a student can reason through most of the stack and improvise the rest, gated by an exam or the completed checklist; it is "a rite of passage" to celebrate.
- Learning continues because the stack changes, leaving knowledge "historic at best": hold a regular learning series, recorded for a training library, and give talks to developer counterparts.

Closing Thoughts

- Up-front training is "absolutely worthwhile"; the charge is "you have to scale your humans faster than you scale your machines."

How this chapter builds the authors' case

It points back to Chapter 15 for postmortems and says only that SRE teams mix proactive and reactive work "as discussed elsewhere in this book"; its starter projects value removing toil. As the first of the Part IV chapters, it prepares the on-call SRE for the interrupts of Chapter 29 and the team-level topics of Chapters 30 to 32.

CHAPTER 29

Dealing with Interrupts

Place in the argument — why this chapter, why this title

Written by Dave O'Connor (edited by Diane Bates). The Part IV intro presents it as a necessary skill, since responsibility for any significant service brings interruptions. The title names the problem rather than a solution; the chapter first defines operational load and how Google manages it, then the factors, then the human side ("Imperfect Machines"), which is where the practical advice comes from.

Chapter opening — untitled section before the first sub-heading in the book

- "Operational load" is the work that must be done to maintain a system in a functional state, like servicing a car. "Any complex system is as imperfect as its creators", who are "also imperfect machines."
- Three categories: **pages** (production alerts, always with an expected response time (SLO), sometimes in minutes); **tickets** (customer requests needing action; SLO more likely hours, days or weeks); **ongoing operational responsibilities** ("toil", see Chapter 5: rollouts, ad hoc time-sensitive questions), which may lack an SLO but "can interrupt you". Much of the load is unplanned, so the person must decide whether it can wait.

Managing Operational Load

- Pages: usually a dedicated primary on-call, a single person, to minimize interruption and "avoid the bystander effect"; may escalate. A secondary backs up the primary (sometimes only contacting the primary if pages fall through, possibly from another team).
- Tickets: handled by the primary or secondary on-call, or by a dedicated non-on-call person; randomly autodistributed or serviced ad hoc.
- Ongoing responsibilities: done by the on-call, assigned ad hoc, or picked up as lasting beyond a shift week.

Factors in Determining How Interrupts Are Handled

- Useful metrics: interrupt SLO, backlog size, severity, frequency, and the number of people available (some teams require ticket work before on-call).
- All suit the lowest possible response time, "without factoring in more human costs", which are hard to measure.

Imperfect Machines

- Humans "get bored" and "aren't very efficient"; the human element is "Working as Intended".
- **Cognitive Flow State**: being "in the zone" raises productivity and creativity and encourages mastery; interruption can kick you out, so maximize time in it. Flow needs clear goals, immediate feedback and a sense of control, and can come from low- or high-skill tasks. In the zone state, many SREs spend their time failing to get in, never trying, or "languishing in the interrupted state". In the "Angry Birds" state, full-time interrupt work gives clear goals and feedback (close X bugs, or stop getting paged) and can be fulfilling; stress comes from pager volume or from treating on-call as an interrupt while also coding, i.e. constant "interruptability". In a mixed environment people are happier with a balance, which varies by engineer, and they may not know their best one.
- **Do One Thing Well**: advice is for managers, directing team structure "so that people aren't set up for failure".
- Distractibility: Fred, not on-call, can still be hit by a randomly assigned due-today ticket, a colleague's page about his component, a raised ticket priority, a flag rollout going wrong, or a user question. Teams can manage it so more people feel "undistractible".
- Polarizing time: assign a cost to context switches; a 20-minute interruption means two switches and "a loss of a couple hours of truly productive work". Aim for polarized time, ideally a week, or a day or half-day, so each day is just projects or just interrupts (the idea of make time).
- **Seriously, Tell Me What to Do**: if one person cannot handle a class of interrupt volume, add another person (or downgrade pages to tickets). On-call: the primary focuses solely on on-call and that week is "written off" for projects; if a project cannot slip, they should not be on-call. A secondary who

actually helps should do interrupt work. Tickets: random assignment is “extremely disrespectful of your team’s time”; make it a full-time role, with two people if needed, not the whole team. Ongoing responsibilities: define roles anyone can take up, such as a push manager, with a formal handover. Be on interrupts, or don’t be: people who take tickets when unassigned skew the numbers and hide an unmanageable queue.

- **Reducing Interrupts**: rotations like a gauntlet leave root causes uninvestigated. Hand off tickets as for on-call; regularly scrub tickets and pages for root causes, and silence a class while a fix is expected, which also enforces a deadline.
- **Respect yourself, as well as your customers**: “Your team sets the level of service”; “It’s OK to push back some of the effort onto your customers.” “Policy can be as powerful a tool as code.” If a flaky component gets no attention, consider giving back the pager or deprecating it. Push back steps that need no privileges. Balance: the request should be meaningful, rational and include the legwork; in return the response should be helpful and timely.

How this chapter builds the authors’ case

It ties ongoing responsibilities to toil (Chapter 5) and follows Chapter 28 by describing what a newly on-call SRE meets. The Part IV intro says that if turbulent conditions persist, a team must recover from operational overload, which is the subject of Chapter 30.

CHAPTER 30

Embedding an SRE to Recover from Operational Overload

Place in the argument — why this chapter, why this title

Written by Randall Bosetti (edited by Diane Bates). The Part IV intro says that when turbulent conditions persist, a team must recover from operational overload, and offers a flight plan here. The sub-headings are three phases in order: learn the service, share context, drive change; each depends on the one before.

Chapter opening — untitled section before the first sub-heading in the book

- Standard policy splits time evenly between projects and reactive ops work, but rising ticket volume can upset this for months; then the team may burn out and "scalability and reliability suffer."
- Remedy: temporarily embed one SRE who improves the team's practices rather than just emptying the queue, giving a fresh perspective the team cannot give itself. Two may cause problems if the team reacts defensively.

Phase 1: Learn the Service and Get Context

- Articulate why habits help or hurt scalability: "more tickets should not require more SREs."
- **Ops Mode Versus Nonlinear Scaling**: **ops mode** answers growth with more administrators; SRE writes software so headcount does not rise with load. Shadow on-call to test "my service is tiny": if tiny but important, focus on what blocks reliability; "your job is to make the service work, not to shield the development team from alerts." If just starting, prepare for growth (100 to 10k requests/second in a year).
- **Identify the Largest Sources of Stress**: teams fall into ops mode by addressing emergencies instead of reducing them; rank outages by stress, since small problems can cause outsized stress.
- **Identify Kindling**, emergencies waiting to happen: services not designed to be self-managing; overspecialization knowledge gaps; SRE-developed services quietly growing in importance; reliance on "the next big thing"; undiagnosed common alerts; complaints with no SLI/SLO/SLA (Chapter 4); capacity plans that merely "Add more servers"; postmortems only rolling back the change; components where SREs say "the devs own it".

Phase 2: Sharing Context

- **Write a Good Postmortem for the Team**: commenting on old postmortems puts the team on the defensive, so take ownership of the next one, blameless, with the on-call, to show the SRE model makes fixes more permanent. "Why me?" reflects the **Bad Apple Theory**, "demonstrably false". Say: "Mistakes are inevitable in any system with multiple subtle interactions", and ask them to write what they were thinking, to find where the system misled them.
- **Sort Fires According to Type**: some fires "shouldn't exist" (toil, Chapter 5); others are part of the job. Sort, present, and explain each to the team.

Phase 3: Driving Change

- Team health is a process, not heroic effort; create the right initial conditions and teach a few principles.
- **Start with the Basics**: work from Chapters 1 and 6. First write an SLO: "probably the single most important lever" away from reactive ops; without it "no other advice in this chapter will be helpful."
- **Get Help Clearing Kindling**: resist fixing issues yourself, which bolsters "making changes is for other people." Give one team member useful work that permanently addresses a postmortem issue, review it, repeat two or three times.
- **Explain Your Reasoning**: explain every decision so the team can predict your review comments later; e.g. "I'm pushing back because the error budget we set for releases is exhausted."
- **Ask Leading Questions**: not loaded ones; they model principle-based reasoning, e.g. "How does this impact the SLO?" of a frequently firing, ignored alert.

Conclusion

- The team gains a reason to change, an example of change, an explanation of "folk wisdom", and principles for novel situations.

- End with an after-action report (possibly a postvitam) with action items; stay available for reviews and watch the team for months.

How this chapter builds the authors' case

It relies on the Introduction and Chapters 1, 4, 5 and 6 (SLOs, toil, monitoring) and on blameless postmortems. It follows Chapter 29 by addressing what happens when interrupts persist; Chapter 31 turns to communication and collaboration.

CHAPTER 31

Communication and Collaboration in SRE

Place in the argument — why this chapter, why this title

Written by Niall Murphy with Alex Rodriguez, Carl Crous, Dario Freni, Dylan Curley, Lorenzo Blanco and Todd Underwood (edited by Betsy Beyer). The Part IV intro says it covers the different roles within SRE; cross-team, cross-site and cross-continent communication; running production meetings; and case studies of good collaboration. The sub-headings move outward: communication within a team (production meetings), collaboration within SRE (with the Viceroy case), then collaboration outside SRE (with the DFP-to-F1 case).

Chapter opening — untitled section before the first sub-heading in the book

- SRE's organizational position shapes how it communicates. There is "tremendous diversity": infrastructural, service and horizontal product teams; product development counterparts many times the size, about the same size, or SRE as the developer. "We don't have just one model", which suits the "ultimately pragmatic nature."
- SRE is "not a command-and-control organization": teams work with the product developers for their service, and are accountable for it, but reporting lines run through SRE as a whole. Shared culture gives "strongly homogeneous approaches to problems. This is by design."
- Communication is like data flow, which must flow reliably around a team. The interface a team presents is like an API: good design is crucial and a wrong API is painful to correct. The API-as-contract metaphor applies to collaboration too. The promise of SRE: an organization charged with reliability, "with the same skills as the product development teams, will improve things measurably"; someone in charge of reliability without the complete skill set is not enough.

Communications: Production Meetings

- Few are lucky enough to have only useful meetings, but the **production meeting** is more useful than average: the team articulates to itself and invitees the state of its services, aiming for everyone to leave with "the same idea". It is service-oriented, not individual status updates. The second goal: bring "the wisdom of production" to bear, relating operational performance to design, configuration or implementation. "An immensely powerful feedback loop."
- Weekly, 30-60 minutes. Shorter suggests cutting things or a small portfolio; longer suggests mire in detail or sharding the team or services. A chair, often rotated, gives everyone a stake and teaches chairing skills useful in incident coordination. On a video call between teams of unequal size, put the chair on the smaller side.
- **Agenda** (not prescriptive; Appendix F has an example): Upcoming production changes (default is enabling change, tracking start time, duration, expected effect); Metrics (core metrics, Chapter 4; trends build a feeling for the performance envelope; resource usage shows insidious change); Outages (postmortem-sized problems, Chapter 15); Paging events (should that alert have paged in that way, or at all? remove unactionable pages); Nonpaging events (issues that should have paged, so fix monitoring; non-pageable issues needing attention; and non-pageable ones needing none, which should be removed as noise); Prior action items (assign and track; consistent delivery builds credibility and trust).
- **Attendance**: compulsory for the team, especially when spread across time zones, plus major stakeholders and partner product development teams. If you cannot invite your developers, "you need to fix that relationship", or much of the value of an SRE team, the operational feedback loop, is lost. Techniques for too many teams: a single representative, nominated subsets, advance feedback or the prefilled agenda.
- A well-known Google Doc enables "bottom up" pre-populated agendas prepared in parallel; real-time collaboration makes more people feel they own a slice of the team's work.

Collaboration within SRE

- A distributed organization, separated by time zones for pager rotations, makes "team" fluid: local, site, cross-continental and virtual teams. The interesting case is cross-team, cross-site and virtual collaboration.

- Technical mastery is hard, so teams specialise on a related subset of systems to decrease cognitive load. That improves mastery but leads to "siloization"; a crisp team charter defines what a team will and won't support.
- **Team Composition**: diversity improves collaboration; "diverse teams are simply better teams". Formal roles: tech lead (TL), manager (SRM), project manager. Well-defined roles allow quick safe decisions; fluid roles suit developed teams adapting to new situations but cost more communication. The TL owns technical direction; the manager adds performance management and being the catchall. Great ones have a complete skill set.
- **Techniques for Working Effectively**: "singleton projects fail unless the person is particularly gifted or the problem is straightforward." Outside the local team, you need excellent written communication or lots of travel: "over time you decay into just being an email address until you turn up in the flesh again."

Case Study of Collaboration in SRE: Viceroy

- Viceroy is a monitoring dashboard framework and service. SRE's structure produced duplicate work; each team was rewarded for its own solution, cross-team work was hard, and SRE-wide infrastructure was "closer to a toolkit than a product", leaving "smoldering, abandoned hulks" of frameworks.
- **The Coming of the Viceroy**: in 2012 teams considered moving from Borgmon (Chapter 10) to Monarch, but consoles were cumbersome in Borgmon and Monarch's console support did not scale to complex consoles nor support the legacy system. Teams (Spanner, Ads Frontend, and others; one effort was Consoles++) duplicated work for 12-18 months until engineers discovered each other and Viceroy was born in mid 2012. Teams with large existing projects had little incentive to move. Requirements shared two points (complex curated dashboards; support both systems) but differed in others (JavaScript use above all). A first look at merging in 2013 failed; by the end of 2013 differences had narrowed as Viceroy adopted JavaScript, prototypes appeared in early 2014 and the combined system was complete by the end of 2014. Benefits: data sources, modular JavaScript compilation, cache and background pipeline, and "development velocity on one solution was much larger than the sum" of the duplicates. "Ultimately, the common future vision was the key factor." Viceroy was declared the general monitoring solution, recommended rather than required.
- **Challenges**: first coordination among remote members was hard (misread cues, no water-cooler talk); extended contributors churned, needing training, though returning SREs became local experts; casual contributions diluted ownership and features were dropped; scope grew; distributed owners defaulted to local decisions, causing conflict.
- **Recommendations**: develop cross-site only when you have to (higher latency and communication cost, much higher throughput); check contributors' commitment; leaders provide vision, and agree a decision-making way that favours local decisions where trust is high; divide into components, each assigned to a small group preferably at one site (watch Conway's law); orient goals to functionality; use design docs and reviews, since "writing things down" offsets distance; argue standards with a strict time limit, then pick, document, move on; meet in person, with summits at a neutral location; match project management to the project's stage.

Collaboration Outside SRE

- Collaboration with product development is best early in design, ideally "before any line of code has been committed", since SRE recommendations on architecture are hard to retrofit (Chapter 32 discusses this). Tracked via OKRs; for some service teams it is their mainstay.

Case Study: Migrating DFP to F1

- Migration of DoubleClick for Publishers' main database from MySQL to F1 (Spanner is also named). The authors ran part of the serving system, using about 1,000 CPUs and 8 TB of RAM to index 100 TB daily (Figure 31-1). The schema was refactored, the output had to be perfectly identical so serving was untouched, and migration had to be live without disruption.
- Developers know business logic (BL); SREs know infrastructure from reuse across services. They met weekly. Because BL depended on infrastructure, they began with the infrastructure design, driven by SREs (extract tables, filter and join, extract only changed data, survive machine loss, linear resource growth, capacity planning). The design resembled other F1 extractors, so monitoring and tooling could be reused. Two SREs wrote the design doc; both teams reviewed it. Defined interfaces let developers work on BL independently.

- SREs deployed to a testing environment to measure performance and resources while developers validated that the new index matched the old, fixing discrepancies iteratively. Meanwhile SREs prepared production, monitoring, on-call training and a basic release process, then a rollout plan with developers. Launch was smooth, with no visible user impact.

Conclusion

- Given globally distributed teams, communication has always been a high priority. Collaboration between SRE teams has challenges but great rewards, including common platform approaches that free teams for harder problems.

How this chapter builds the authors' case

It uses the metrics of Chapter 4, the postmortems of Chapter 15 and Borgmon from Chapter 10, and cites Chapter 33 for SRE backgrounds from other industries. It says early collaboration with developers is discussed in Chapter 32, which the Part IV intro presents as the production readiness review and how to move beyond it.

CHAPTER 32

The Evolving SRE Engagement Model

Place in the argument — why this chapter, why this title

Written by Acacio Cruz and Ashish Bhambhani (edited by Betsy Beyer and Tim Harvey). The Part IV intro calls the production readiness review (PRR) a cornerstone of SRE operation and a crucial step in onboarding a new service, and says the chapter shows how to conduct PRRs and how to move beyond this successful but limited model. The title says the model evolves; sub-headings follow that evolution: the classic Simple PRR, then Early Engagement, then Frameworks and an SRE Platform.

SRE Engagement: What, How, and Why

- The rest of the book covers what happens when SRE already runs a service. Few services start with SRE support, so there must be a process to evaluate a service, negotiate how to fix deficits that bar support, and institute it: **onboarding**. Working through a prioritized queue of existing services is reasonable, but there are two better ways.
- First, as with bugs, "the earlier an SRE team consultation happens, the better the service will be"; engaged at design, onboarding time falls and the service is more reliable "out of the gate", since no suboptimal design must be unwound.
- Second, perhaps the best: short-circuit the process by which individually varied systems arrive at SRE's door, giving product development "a platform of SRE-validated infrastructure", reliable and scalable, which avoids classes of cognitive load problems and lets developers focus on innovation at the application layer.

The PRR Model

- The **Production Readiness Review** identifies a service's reliability needs from its specifics; SREs apply what they have learned. It is a prerequisite for SRE accepting responsibility for the production aspects of a service.
- It can start at any point in the lifecycle (Figure 32-1); engagement stages have expanded: the Simple PRR Model, then the Extended Engagement Model and the Frameworks and SRE Platform, which let SRE scale.

The SRE Engagement Model

- SRE seeks responsibility for important services where it can make concrete contributions. "Production" covers: system architecture and interservice dependencies; instrumentation, metrics and monitoring; emergency response; capacity planning; change management; performance (availability, latency, efficiency). SRE aims to improve a service along all of these.
- **Alternative Support**: many services do not need high reliability, and by design requests exceed SRE bandwidth (Chapter 1). Alternatives: documentation (Google's Production Guide) and consultation (the Launch Coordination Engineering team, Chapter 27; one or two SREs study a design for a few hours). Consultation is necessarily broad and insufficient for services that grew by orders of magnitude or that many others rely on; those need long-term engagement.

Production Readiness Reviews: Simple PRR Model

- When developers request takeover, SRE weighs the service's importance and SRE availability; if it merits support and staffing is agreed, SRE starts a PRR. Objectives: verify the service meets accepted standards of production setup and operational readiness and that owners can work with SRE; and improve reliability, minimizing the number and severity of incidents. Then an SRE team takes over. The Simple PRR usually targets an already launched service.
- **Engagement**: one to three SREs conduct the PRR and discuss with developers an SLO/SLA, disruptive design changes, and planning and training.
- **Analysis**: reviewers gauge the service's maturity along SRE's axes, using a service-specific PRR checklist based on domain expertise, similar systems and the Production Guide. Items include whether updates hit an unreasonably large share at once, whether the service uses the appropriate serving instance of dependencies (user requests should not depend on a batch system), and whether user-visible failures are monitored. SREs also review recent incidents and postmortems.
- **Improvements and Refactoring**: prioritize by importance to reliability, agree a plan with developers, and refactor together; this phase varies most in effort.

- **Training**: the lead reviewers train the whole team (design overviews, request-flow deep dives, hands-on exercises).
- **Onboarding**: progressive transfer of operations, change management and access; developers stay available to back up the SRE team.
- **Continuous Improvement**: services keep changing; SRE learns by operating, reviewing changes and postmortems, advises developers and feeds the Production Guide.
- **Engaging with Shakespeare**: developers carried the pager until growth made SRE support desirable; the PRR found dashboards not covering some SLO metrics; once fixed, SRE took the pager, with two developers still in the rotation.

Evolving the Simple PRR Model: Early Engagement

- Limitations include communication overhead and reviewer availability, but the main limit is that engagement starts very late, after launch at scale; earlier would markedly increase SRE's chance to remedy issues.
- **Candidates for Early Engagement**: services with significant new functionality joining a system SRE manages; significant rewrites of existing systems; services whose developers sought SRE advice or takeover at launch. SRE joins at Design and takes over during or after Build.
- **Benefits**: Design ("The best production incidents are those that never happen!"; SREs are party to trade-offs). Build (recommend libraries, build in controls, gain operational experience). Launch (e.g. a **dark launch**, where some traffic goes to the new service and responses are thrown away). Post-launch (fewer conflicts between reliability and features; SRE can take over sooner; a lasting collaborative relationship).
- **Disengaging from a service**: needing no full SRE management is a positive outcome, as the service was engineered to be reliable. If usage falls short, the effort is "part of the overall business risk".

Evolving Services Development: Frameworks and SRE Platform

- Early engagement still left room to scale.
- **Lessons Learned**: onboarding took two or three SREs and two or three quarters, so takeovers were serialized and strictly prioritized; production features were reimplemented per service (e.g. repeated logging frameworks in one language); fixes for patterns like overload could not be replicated; SRE contributions were local, so lessons could not reach onboarded services.
- **External Factors**: microservices raise the number of services and requests; each has a base fixed operational cost and expects lead times shorter than months. Hiring experienced SREs is difficult and costly.
- **Toward a Structural Solution: Frameworks** has four principles: codified best practices (services become "production ready" by design); reusable solutions; a common production platform with a common control surface (uniform interfaces, controls, monitoring, logging, configuration); easier automation and smarter systems (a single view for an outage). Frameworks exist for Java, C++ and Go. The shift: developers design with the SRE-blessed framework, rather than retrofitting to SRE specifications or adding SREs for markedly different services. SRE concerns sit mostly in infrastructure beneath business logic; framework modules (instrumentation and metrics, request logging, traffic and load control) are canonical solutions, so developers focus on business logic. Implementations in each language expose the same API, behavior, configuration and controls.
- **New Service and Management Benefits**: lower operational overhead (conformance tests, built-in deployment, monitoring and automation; deployment "in a matter of days"); universal support by design, which "effectively breaks the SRE staffing barrier"; faster, lower overhead engagements (a single SRE during one quarter); and **shared responsibility**: previously only full support or approximately none, now SRE supports the platform infrastructure (load shedding, overload, automation, traffic management, logging, monitoring) while developers provide on-call for application bugs. This brings a new relationship model and a new staffing model, with platform teams staffed by platform need, not service count.

Conclusion

- The Simple PRR standardized engagement but applied only after launch; Early Engagement involved SRE earlier to "design for reliability"; frameworks standardized best-practice code to meet growing demand. All three models are still practiced, but frameworks are becoming prominent, expanding SRE's contribution, lowering overhead and improving baseline quality.

How this chapter builds the authors' case

It contrasts with the rest of the book, which covers services SRE already runs, and refers back to Chapter 1 (demand exceeds SRE bandwidth) and Chapter 27 (the Launch Coordination Engineering team). Chapter 31 says early collaboration with developers is discussed here. It closes Part IV's management theme by moving from the PRR to engagement that scales.

PART V

CONCLUSIONS

- The authors' own description: having covered how SRE works at Google and how its principles and practices "might be applied to other organizations in our field", the section turns to other industries and to a closing view.

The theme of the Part

- SRE compared with other industries "where reliability is critically important", and SRE's evolution seen through aviation.

How the chapters build the theme

- Chapter 33: how SRE practices compare with other reliability-critical industries.
- Chapter 34: Benjamin Lutch, "Google's VP for Site Reliability Engineering", writes about SRE's evolution over his career, "through the lens of some observations on the aviation industry."

What the Part contributes to the book

- Places SRE in a wider context and closes the book.

CHAPTER 33

Lessons Learned from Other Industries

Place in the argument — why this chapter, why this title

Written by Jennifer Petoff (edited by Betsy Beyer). The Part V intro says that, having covered how SRE works at Google and might apply elsewhere, it is time to compare SRE's practices with other industries where reliability is critically important. The chapter asks whether SRE principles matter outside Google, how they are manifested, what drives differences, and what Google can learn. It distils the book into four themes, and the sub-headings follow: the veterans interviewed, then one section per theme in that order, then conclusions.

Chapter opening — untitled section before the first sub-heading in the book

- The authors interviewed Google engineers about previous work in other high-reliability fields to answer five comparative questions: are SRE principles important outside Google; how are they manifested; what are the similarities and differences; what drives them; and what can Google and tech learn.
- Four key themes: Preparedness and Disaster Testing; Postmortem Culture; Automation and Reduced Operational Overhead; Structured and Rational Decision Making.

Meet Our Industry Veterans

- Twelve veterans, each from a field where lapses are costly: defense GPS and guidance (Dahl); lifeguarding (Doherty); laser eye surgery (Gross); telecom and E911 (Hartmann, Greer); medical devices and automotive (Heiby); military aircraft and UK railway signaling (Hilton); synthetic diamond manufacturing (Kennedy); proprietary trading (Li); UK civil nuclear (Sheridan); US Navy nuclear submarines (Stevenson); air traffic control (Toia). Consequences range from financial loss to "fatalities".

Preparedness and Disaster Testing

- "Hope is not a strategy." SRE asks what could go wrong and what to do before it causes an outage or data loss. Annual **DiRT** (Disaster and Recovery Testing) drills inflict actual outages to ensure systems react as expected, find unexpected weaknesses and make systems robust.
- Strategies from other industries: relentless organizational focus on safety; attention to detail; swing capacity; simulations and live drills; training and certification; detailed requirements gathering and design; defense in depth.
- **Relentless Organizational Focus on Safety**: in manufacturing, "every management meeting started with a discussion of safety"; highly defined processes are strictly followed and workers feel empowered to speak up. Nuclear, military aircraft and railway signaling have detailed software safety standards and levels (Safety Integrity Level 1-4).
- **Attention to Detail**: in the Navy, a lapse on small tasks such as lube oil maintenance could cause major failure, so routine maintenance stops small issues snowballing.
- **Swing Capacity**: telecom loads are unpredictable (disasters, the Olympics); a SOW (switch on wheels), a mobile telco office, provides excess capacity.
- **Simulations and Live Drills**: the consequences of an outage decide which. Aviation cannot test "in production", so it uses realistic simulators; telecom drills for hurricanes; the nuclear Navy mixes "what if" exercises with live drills ("actually breaking real stuff but with control parameters", two to three days a week) because thought exercises are "useful, but not sufficient"; lifeguards face "mystery shopper" mock drownings.
- **Training and Certification**: matter when lives are at stake, e.g. lifeguard certification and recertification.
- **Detailed Requirements Gathering and Design**: for medical devices, users and maintainers are not designers, so requirements come from surgeons and technicians. A defense system took a year of design and three weeks of code, unlike Google's "launch and iterate" culture, because pressures and risk appetites differ.
- **Defense in Depth and Breadth**: nuclear reactors have redundancy, fallback systems and layered protection, due to zero tolerance for failure.

Postmortem Culture

- **Corrective and preventative action** (CAPA) is systematic investigation of root causes to prevent recurrence; SRE embodies it in blameless postmortems: what happened, how effective the response was, what to do differently, what actions prevent recurrence. "Dwelling on who might have caused the outage is counterproductive." They are published across SRE teams.
- Many industries do a version; "the motivation behind these exercises appears to be the main differentiator." Motives: regulation (FCC, FAA, OSHA, FDA, EU authorities); and safety. At Alcoa, the CEO required notification within 24 hours of any lost-worker-day injury. Manufacturing scrutinizes "near misses", which are "disasters waiting to happen" ("Latent error, plus an enabling condition, equals things not working quite the way you planned").
- Lifeguarding: "If a lifeguard's feet go in the water, there will be paperwork!" Serious incidents are examined end to end, leading to operational changes and training; it embraces blameless analysis.

Automating Away Repetitive Work and Operational Overhead

- SREs are software engineers with a low tolerance for repetitive reactive work; automation frees time to improve services.
- Industries are mixed. The nuclear Navy eschewed automation for interlocks and administrative procedures (a valve needed an operator, a supervisor and a crew member on the phone to the engineering watch officer) and a trusted human decision chain, fearing automation commits large irreparable mistakes quickly. Proprietary trading has grown cautious: Knight Capital lost $440M in a few hours in 2012 and the 2010 Flash Crash cost trillions in 30 minutes. Manufacturing embraces automation for efficiency and repeatability; in UK nuclear, a response needed in under 30 minutes must be automated. In laser eye surgery, a sanity check on entered data, then matching the patient's iris to a photo, eliminated "an entire class of medical errors."

Structured and Rational Decision Making

- SRE aims for decisions where the basis is agreed in advance, inputs are clear, assumptions explicit, and data beats feelings or the "HiPPO" (Highest-Paid Person's Opinion); everyone has users' interests at heart and can proceed from data.
- Other industries: "if it ain't broke, don't fix it…ever" (1980s telecom switches "bulletproof and massively redundant"; nuclear); playbooks in "the binder", suited to slowly evolving industries or limited worker skill; data-driven experimentation in research and manufacturing; and divided decision making in trading, where a separate enforcement team can halt trading and only it can restart: "If we aren't trading, we aren't losing money."

Conclusions

- Many SRE principles appear across industries, probably inspiring Google's practices. Main takeaway: Google has "a higher appetite for velocity"; the ability to change quickly must be weighed against failure implications, and where lives are at risk a conservative approach is warranted.
- Google walks a tightrope between reliability expectations and rapid change, deciding how reliable "reliable enough" is (as in earlier chapters). With lives not directly at risk, it can use error budgets to "fund" innovation and calculated risk, adapting known principles to balance scale, complexity and velocity with reliability.

How this chapter builds the authors' case

It condenses principles "discussed throughout this text" into four themes, linking to "Hope is not a strategy" and DiRT, blameless postmortems, automation and data-driven decisions, and ends on the error budget ("Motivation for Error Budgets" on page 33). Chapter 31 also cited it for the varied backgrounds of SREs. As the Part V intro says, Chapter 34 then closes the book.

CHAPTER 34

Conclusion

Place in the argument — why this chapter, why this title

Written by Benjamin Lutch, Google's VP for Site Reliability Engineering (edited by Betsy Beyer). The Part V intro frames it as SRE's evolution seen through aviation, following Chapter 33's comparison with other industries. He asks what let SRE evolve so well, and answers: "the nature of the principles by which it operates."

Chapter opening — untitled section before the first sub-heading in the book

- Engineers split time between on-call, observing how systems break, and reflecting on what to build to ease management; SREs play both pilot and designer, and experience is codified in code, packaged as a product.
- A foundation should be rules general enough to be useful but relevant later; the Introduction's responsibilities remain "spot-on 10 years after they were conceived." Concerns stay constant while activities evolve, e.g. from a dashboard for 20 machines to automation over tens of thousands.
- Aviation analogy: early planes had a pilot doing everything and simple, fragile, non-redundant systems. A 747 has redundant systems, yet the cockpit has "just two pilots", thanks to well-designed, approachable interfaces, flexible enough and with trained operators for emergencies.
- SRE's goal: a team "as compact as possible", at a high level of abstraction, with failsafes and thoughtful APIs, yet with comprehensive knowledge of how systems operate and fail from running them day-to-day.

How this chapter builds the authors' case

It returns to the Introduction by Ben Treynor Sloss and restates the book's system properties, extending Chapter 33's industry comparison to close the book.